

第九章 查找表（集合）

查找运算是计算机中最常用的操作之一，计算机大约有40%的运算与查找有关。被查找的数据对象逻辑结构可认为是**集合**，选取不同的存储结构，查找以及其它操作的效率也会有所不同。

9.1 概念和术语

9.2 静态查找表

9.3 动态查找表

9.4 哈希表

本章学习要点与习题



9.1 相关概念和术语

记录（数据元素）

[术语]

数据项1	数据项2	
------	------	-------

查找表 同一类型的记录(数据元素)的**集合**。

数据元素之间存在着松散的关系。为了提高查找

的效率，可在元素之间**人为地附加**某种确定的关系。

关键字 记录(数据元素)中的某个数据项的值。

主关键字 该关键字可以唯一地标识一个记录。

次关键字 该关键字不能唯一标识一个记录。

查找 指定某个值，在查找表中确定是否存在一个记录，该记录的**关键字**等于给定值。

静态查找表 对查找表的查找仅是以查询为目的，不改动查找表中的数据。

动态查找表 在查找的过程中同时插入不存在的记录，或删除某个已存在的记录。

查找成功 查找表中存在满足查找条件的记录。

查找不成功 查找表中不存在满足查找条件的记录。

内查找 整个查找过程都在内存中进行，即全部数据元素均在内存。

外查找 在查找过程中需要访问外存，即数据元素需要分段读入内存查找。

平均查找长度ASL (**A**verage **S**earch **L**ength) 查找方法时效的度量
为确定记录在查找表中的位置，需将关键字和给定值比较次数的期望值。

在无法确定查找成功和查找不成功的概率分别为多少时，按**查找成功时的ASL**和**查找不成功时的ASL**分别计算和讨论。

查找成功时的ASL计算方法： $ASL = \sum_{i=1}^n p_i c_i$ 其中： $\sum_{i=1}^n p_i = 1$

n ：记录的个数

p_i ：查找 n 个不同记录中第 i 个记录的概率，
(不特别声明时认为等概率 $p_i = 1/n$)

c_i ：找到第 i 个记录所需的比较次数

```
typedef int KeyType;
typedef struct {
    KeyType key;
    .....
}ElemType;
```

查找失败时的ASL计算方法：

按查找失败时实际的比较次数算。

本章约定：关键字
的类型为整型

[查找表上的基本操作]

- 1) 查找表初始化: `InitTable(&T)`
- 2) 销毁查找表: `DestroyTable(&T)`
- 3) 检索某个特定的数据元素: `SearchTable(T, key)`
- 4) 插入一个数据元素: `InsertTable(&T, e);`
- 5) 删去某个数据元素: `DeleteTable(&T, key);`
- 6) 遍历: `TraverseTable(T, Visit());`

9.2 静态查找表

数据稳定，变动很少的查找表

9.2.1 顺序表的查找

9.2.2 有序表的查找

9.2.3 静态树表的查找

9.2.4 索引顺序表的查找

9.2.1 顺序表的查找

静态查找表以顺序表表示

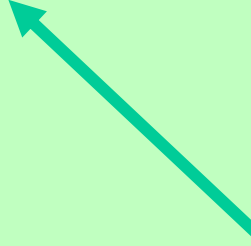
【算法9-1(a)】顺序查找

```
int Seq_Search_1 (datatype data[ ], keytype kx, int n)
{ /*查找表数据存放在data[1]至data[n]中, n是当前表中数据元素个数*/
    /*在表data中查找关键码为kx的数据元素, 若找到返回该元素在数组中的下标,
    否则返回0*/
```

```
    int i=1;
    while(i<=n && data[i].key!= kx )
        i++;    /* 从表头端向后查找 */

    if (i>n)
        return 0;
    else
        return i;
}
```

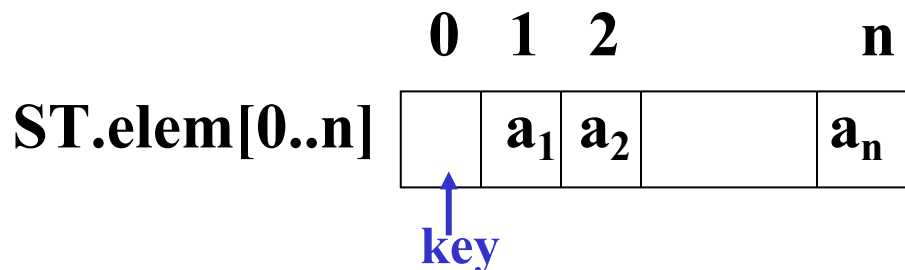
遍历（扫描）到
线性表的一个数
据元素时，要进
行两次比较操作



在上述查找中，每次循环条件要做两个比较和一个关系运算，如果我们稍加修改，会使**比较减少一半**。

实际查找时，通常将给定值(被查找值，key)放在第 0 个记录处，然后从后向前查找，直到找到所查记录为止，找不到则返回结果 0。因为记录 0 像哨兵一样看守着查找表下界，所以不会越界，无需进行越界检查了。

```
typedef struct {  
    ElemType *elem;  
    int length;  
}SSTable;
```



```
int Seq_Search_2 ( SSTable ST, KeyType key)  
{  
    ST.elem[0].key=key;  
    i=n;  
    while(data[i].key!= kx ) i--;  
    return i;  
} //Search_Seq2
```


[程序设计技巧] 设置监视哨，查找时每一步不必检测位置是否越界，提高算法效率。

[性能分析]

- 空间：一个辅助空间。

- 时间：

- 1) 查找成功时的平均查找长度

设表中各记录查找概率相等

$$ASL_s(n) = \sum_{i=1}^n P_i C_i = (1+2+\dots+n)/n = (n+1)/2$$

- 2) 查找不成功时的平均查找长度 $ASL_f = n+1$

[算法特点]

- 算法简单，对表中元素排列顺序无任何要求

- n很大时查找效率较低

- 思考一个改进措施：非等概率查找时，可将查找概率高的记录尽量排在表前部，如汉字输入是的同音字选择列表。

- 逐个查找的方法也可用于链表表示的静态查找表

思考：对于采用链式存储的线性表，为避免每一步的越界检测，哨兵放在哪里？

用什么形式的链表为好：单链表、单循环链表、双循环链表？

【算法9-1(c)】加监视哨后的顺序查找，

```
Int Seq_Search_2(datatype data[ ], keytype kx, int n)
{ /* 查找表数据存放在data[0]至data[n-1]中，n是当前表中数据元素个数 */
  /* 在表data中查找关键码为kx的数据元素，若找到返回该元素在数组中的下标，否则返回-1 */
```

```
  // 假设 n 小于等于 data数组长度 减一；在表尾设置哨兵
```

```
  data[n].key=kx;
```

```
  i=0; //从0向后查找
```

```
  while(data[i].key!= kx ) i++; /* 从表头向尾端查找 */
```

```
  if( i==n) return -1;
```

```
  else return i;
```

```
}
```

data[n]分量起到了监视哨的作用。

这样一个小小的设计技巧，会大大提高查找效率。

利用data[n]位置的数据，设置监视哨，**从前向后**比较，这样无需比较下标的值是否为超过数组长度，到达data[n]时，一定会跳出循环的。

9.2.2 有序表的查找

满足 $r[i].key \leq r[i+1].key$, $1 \leq i < n$

[顺序查找]

仍可用顺序查找，除了哨兵外，还可以利用有序表中存在的大于或小于key的 $r[i]$ 来起到哨兵作用。

- 1) 从前向后查找，哨兵设在尾部，当 $r[i] \geq key$ 时结束；
- 2) 从后向前查找，哨兵设在头部，当 $r[i] \leq key$ 时结束；

```
int seqsearch(sstable R, keytype K )
{
    R.elem[0].key = K;
    for( i = R.length ; R.elem[i].key > K ; i -- ) ;
    if(R.elem[i].key==K) return i;
    else return 0;
}
```

9.2.2 有序表的查找

满足 $r[i].key \leq r[i+1].key$, $1 \leq i < n$

[折半(对半/二分)查找法]

0	1	2	3	4	5	6	7	8	9	10	11
	05	13	19	21	37	56	64	75	80	88	92
↑low						↑mid					↑high

$$= \lfloor (low + high) / 2 \rfloor$$

K=21 l h m

1	11	6
1	5	3
4	5	4

K=85 l h m

1	11	6
7	11	9
10	11	10
10	9	

将给定值与中间的记录进行比较，若找到则查找成功；否则若给定值比中间记录小，则对前一半子表进行折半查找，反之则对后一半子表进行折半查找。若在查找过程中，遇到查找子表的上界小于下界，则表示查找失败。

折半查找（非递归算法）

```
int Search_Bin ( SSTable ST, keytype key)
{
    low= 1; high = ST.length;           // 置区间初值
    while( low <= high )
    {
        mid = ( low+high )/2;
        if(key==ST.elem[mid].key)
            return mid; // 找到待查元素
        else if ( key< ST.elem[mid].key)
            high=mid-1; // 继续在前半区间进行查找
        else
            low=mid+1; // 继续在后半区间进行查找
    }
    return 0; // 顺序表中不存在待查元素
} // Search_Bin
```

折半查找（递归算法）

```
int Search_Bin (SSTable ST, keyType key, int low, int high)
{
    if(low>high)
        return 0;    //查找不到时返回0
    mid=(low+high)/2;
    if( key == ST.elem[mid].key)
        return mid;
    else if(key < ST.elem[mid].key)
        .....//对前半部进行递归折半查找
    else
        .....//对后半部进行递归折半查找
}
```

折半查找（递归算法）

```
int Search_Bin (SSTable ST, keyType key, int low, int high)
{
    if(low>high)
        return 0;    //查找不到时返回0
    mid=(low+high)/2;
    if( key == ST.elem[mid].key)
        return mid;
    else if(key < ST.elem[mid].key)
        return Search_Bin(ST, key, low, mid-1); //前半部
    else
        return Search_Bin(ST, key, mid+1, high ); //后半部
}
```

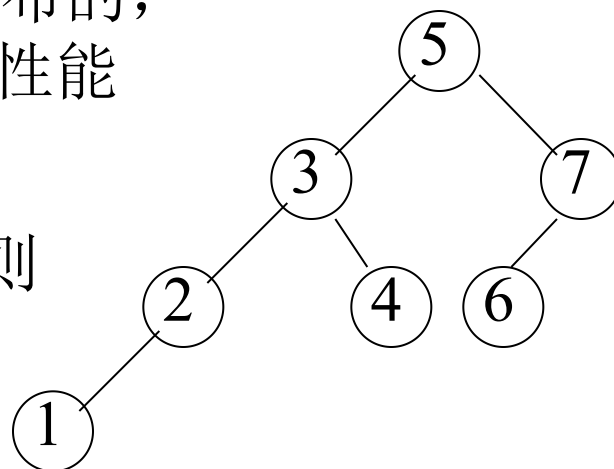
有序表的查找也可用其它的查找方法，如费波拉契查找（Fibonacci search）、插值查找等。

设查找表记录数为某个费波拉契数减一，即 $n = F_u - 1$ ，则可将给定值与第 F_{u-1} 个记录比较，若比其小，则在 $1 \sim F_{u-1} - 1$ 记录间查找，否则在 $F_{u-1} + 1 \sim F_u - 1$ 记录间查找。如下图所示：

若有序表的关键字是均匀分布的，则还可用插值查找方法，其平均性能在记录数较多时比折半查找好：

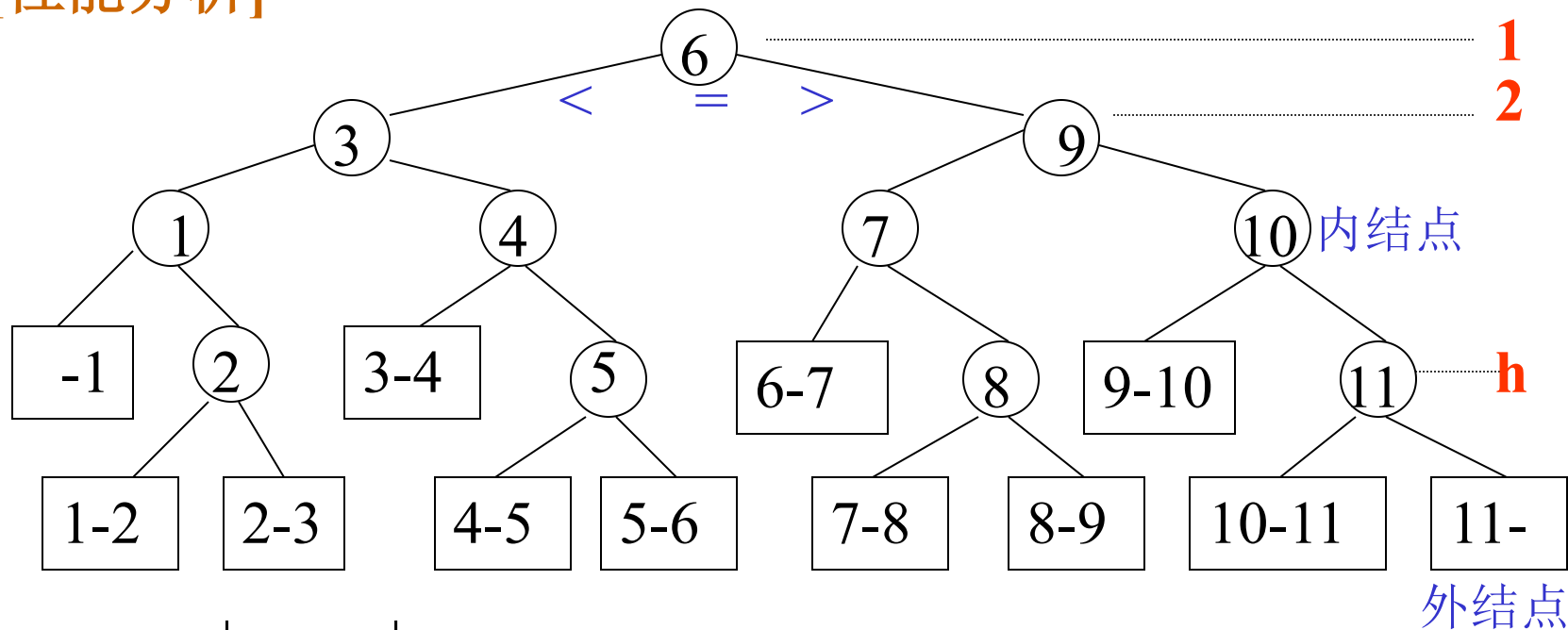
将给定值先与第 i 个记录比较，则

$$i = \frac{K - r[L].key}{r[H].key - r[L].key} n。$$



[性能分析]

判定树（描述查找过程的二叉树）



$$h = \lfloor \log_2 n \rfloor + 1 \quad \{ \text{同完全二叉树, 二叉树性质4} \}$$

成功查找时的平均查找长度:

($n > 50$)

$$ASL_s = \frac{1}{n} \sum_{j=1}^h j \cdot 2^{j-1} = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

例: $ASL_s = (1*1 + 2*2 + 3*4 + 4*4) / 11 = 3$

不成功查找时的查找长度: $h-1$ 或 h 次

三、静态树表的查找：

前面介绍的有序表查找都是在“等概率”情况下的，若在非等概率情况下应该怎样查找才能使查找效率更高了？

查找效率最高即平均查找长度最小，根据前面所学知识，我们可以给出有序表在非等概率情况下应遵循的两个原则：

- 1、最先访问的结点应是访问概率最大的结点；
- 2、每次访问应使结点两边尚未访问的结点的被访概率之和尽可能相等。

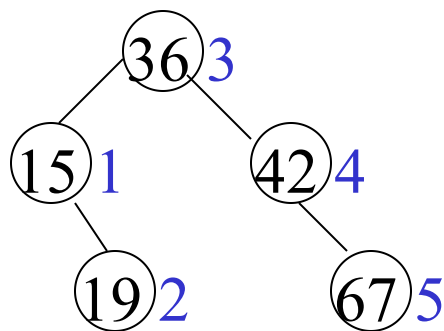
9.2.3 静态树表的查找

[问题出发点]

讨论有序表中各记录查找概率不等时，查找成功时的ASL。

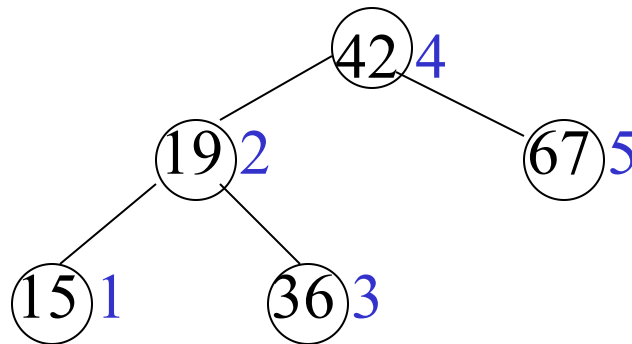
	1	2	3	4	5
关键字	15	19	36	42	67
查找概率	0.1	0.2	0.1	0.4	0.2

折半查找



$$ASL = \sum_{i=1}^n P_i C_i = 2.3$$

高概率为根+折半



$$ASL = \sum_{i=1}^n P_i C_i = 1.8$$

[静态最优查找树]

定义：查找性能最佳的判定树。

性质：带权内路径长度之和PH为最小值。

$$PH = \sum_{i=1}^n w_i h_i \quad \text{与ASL成正比}$$

其中n：二叉树上结点的个数(有序表长度)

h_i ：第i个结点在二叉树上的层次数

$w_i = c p_i$ ：c为某个常量；

p_i ：第i个结点的查找概率

最优查找体现的原则：

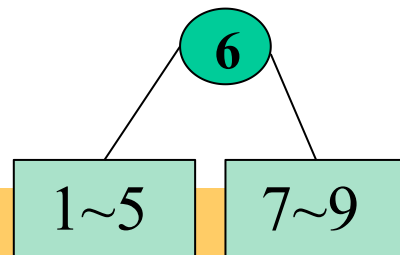
- 1) 最先访问的结点应是访问概率最大的结点；
- 2) 每次访问应使结点两边尚未访问的结点的被访概率之和尽可能相等。

[静态最优查找树---构造方法]

- 暴力求解
 - 生成n个结点对应的所有的二叉查找树，计算上面生成的每个二叉树的每个节点的带权路径长度，进而计算其PH
 - 在所有生成的二叉查找树的中找出PH最小的树
 - 算法复杂度为 $O(4^n/n^{1.5})$

构造这样一棵树的时间代价太大，亦即时间复杂度很大，因此我们通常是构造次优查找树（nearly optimal search tree），构造它的时间代价远远低于构造最优查找树，但查找性能只比最优查找树差1%~2%，很少差3%以上。

[次优查找树的构造方法]



例子: ΔP_i

i	1	2	3	4	5	6	7	8	9	
关键字	A	B	C	D	E	F	G	H	I	有序
权 值	1	1	2	5	3	4	4	3	5	
ΔP_i	[2 7	2 5	2 2	1 5	7]	0	[8	1 5	2 3]	

($r_l, r_{l+1}, \dots, r_{i-1}, \mathbf{r_i}, r_{i+1}, \dots, r_{h-1}, r_h$)

1)选取记录 r_i 作为根结点:

计算所有 $\Delta p_i (l \leq i \leq h)$

$$\Delta p_i = \left| \sum_{j=i+1}^h w_j - \sum_{j=l}^{i-1} w_j \right|$$

找出最小 Δp_i 的对应的 i ;

若相邻关键字的权值大, 则对上述的 i 结果进行调整:

以权值大的记录位置 i 替代最小 Δp_i 的对应的 i 。

2)递归处理 r_i 的左子树: $(r_1, r_{1+1}, \dots, r_{i-1})$ 构造的次优查找树

3)递归处理 r_i 的右子树: $(r_{i+1}, \dots, r_{h-1}, r_h)$ 构造的次优查找树

j	1	2	3	4	5	6	7	8	9	
关键字	A	B	C	D	E	F	G	H	I	有序
权 值	1	1	2	5	3	4	4	3	5	
SW _j	1	2	4	9	12	16	20	23	28	
ΔP _j	[2 7	2 5	2 2	1 5	7]	0	[8	1 5	2 3]	

左子树再次计算:

ΔP_j 1 1 9 6 1 9

右子树再次计算:

ΔP_j 8 1 7

简化计算 Δp_i 的方法:

设定累计权值和 $sw_i = \sum_{j=1}^i w_j$

则 $\sum_{j=l}^{i-1} w_j = sw_{i-1} - sw_{l-1}$ $\sum_{j=i+1}^h w_j = sw_h - sw_i$

$$\begin{aligned}\Delta p_i &= |(sw_h - sw_i) - (sw_{i-1} - sw_{l-1})| \\ &= |(sw_h + sw_{l-1}) - sw_i - sw_{i-1}| \\ &\text{设 } sw_0 = 0\end{aligned}$$

把到第i个位置的累计权值和装入数组了，计算一次用多次。

[次优查找树上的查找步骤]

设给定的查找值为K

将根结点作为当前考察的结点

- 1) 若当前结点指针为NULL，则返回空指针
- 2) 将当前结点的关键字key与K比较，
 若 $K=key$ ，则返回当前结点的指针；
 若 $K>key$ ，将其右孩子作为当前结点，转1)；
 若 $K<key$ ，将其左孩子作为当前结点，转1)；

[查找性能]

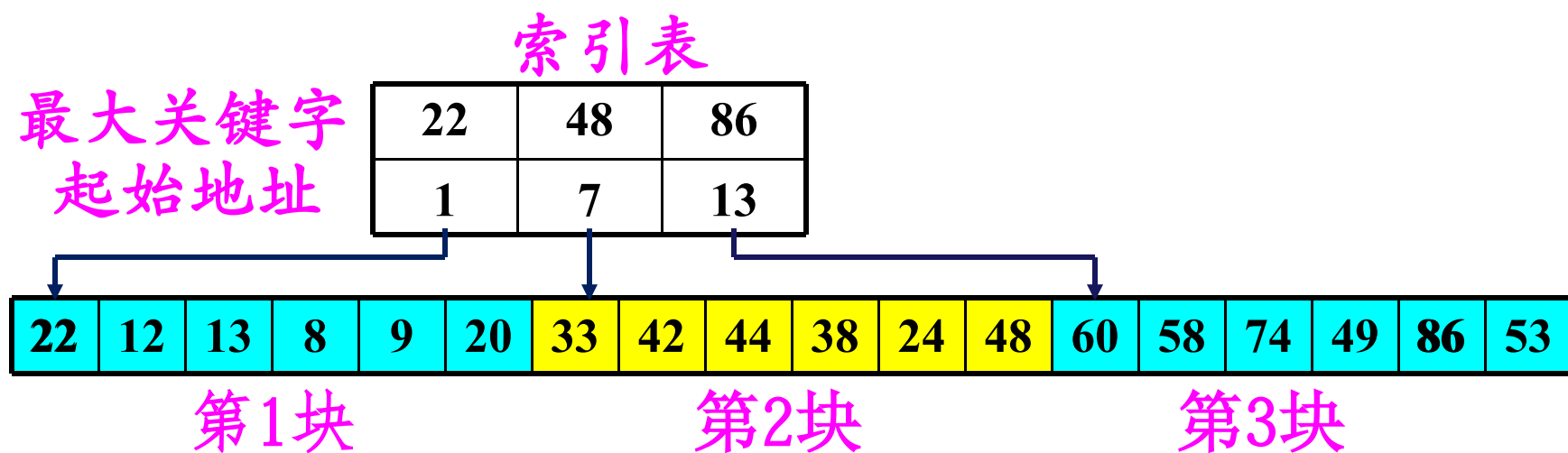
比较次数不超过树的深度， $O(\log_2 n)$



分块查找（块间有序，块内无序）-索引顺序表

分块有序，即分成若干子表，要求每个子表中的数值都比后一块中数值小（但子表内部未必有序）。

然后将各子表中的最大关键字构成一个**索引表**，表中还要包含每个子表的起始地址（即头指针）。



- ① 对索引表使用折半查找法（因为索引表是有序表）；
- ② 确定了待查关键字所在的子表后，在子表内采用顺序查找法（因为各子表内部是无序表）；

9.2.4 索引顺序表的查找

[表的构造]

索引表 $\text{index}[1..b]$

最大关键字值

起始地址

22	42	86
1	5	9

主表 $r[1..n]$ (分块有序/有序)

key
其它项

12	22	13	8	28	33	38	42	86	76	50	63
1	2	3	4	5	6	7	8	9	10	11	12

[索引表类型定义]

```
typedef struct {
    KeyType maxkey;
    int     stadr;
}indexItem; // 索引项
typedef struct {
    indexItem *elem;
    int       blocknum;
}indexTable; // 索引表
```

[分块查找步骤]

- 1) 折半或者顺序查找索引表，确定所在块
- 2) 在已确定的块中顺序查找/折半查找

[性能分析]

索引顺序查找的平均查找长度 =
查找“索引”的平均查找长度
+ 查找“块”的平均查找长度

设 b 为索引表长度， s 为块中记录个数

- 顺序查找索引表+顺序查找被确定的块

$$ASL_{bs} = \frac{1}{b} \sum_{j=1}^b j + \frac{1}{s} \sum_{i=1}^s i = \frac{1}{2}(b + s) + 1 = \frac{1}{2} \left(\frac{n}{s} + s \right) + 1$$

当 s 取 $\sqrt{n}$ 时， ASL_{bs} 取最小值 $\sqrt{n} + 1$

- 折半查找索引表+顺序查找被确定的块

$$ASL_{bs} = [\log_2(b+1) - 1] + (s+1)/2 \approx \log_2(n/s+1) + s/2$$

- 分块查找效率介于顺序查找和折半查找之间

分块查找优缺点

优点：插入和删除比较容易，无需进行大量移动。

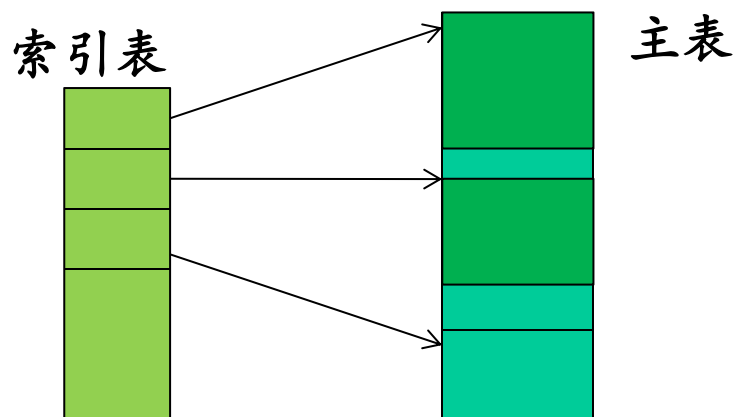
Why?

缺点：要增加一个索引表的存储空间并对初始索引表进行排序运算。

适用情况：如果线性表既要快速查找又少量动态变化，则可采用分块查找。

[算法特点]

- 实用中，各块大小不一定相等
- 分块查找的代价是附加索引表的存储空间开销和建立索引表的时间开销
- 可以在主表的每块后预留空闲位置，以便进行插入和删除操作时只涉及相应的块和该块的索引项。
- 主表中各块可以集中顺序存储，也可以按块分别顺序存储；主表中的记录还可以采用单链表，或每块组成一个单链表。



9.3 动态查找表

频繁进行插入、删除的查找表

9.3.1 二叉排序树

9.3.2 平衡二叉树

9.3.3 B-树

9.3.4 B+树

9.3.5 键树

9.3.1 二叉排序树BST(二叉查找/搜索树)

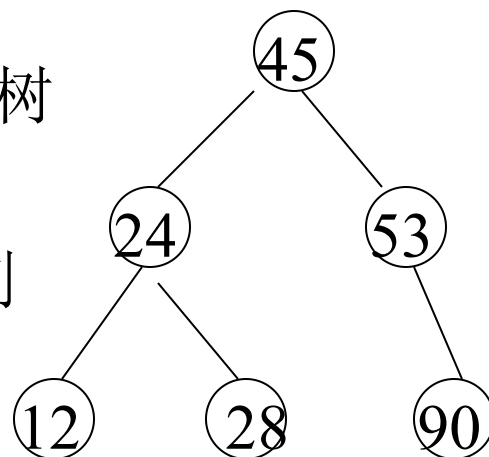
1. 定义

二叉排序树或者是空树，或者是满足如下性质的二叉树：

- 1)若其左子树非空，则左子树上所有结点的值均小于根结点的值；
- 2)若其右子树非空，则右子树上所有结点的值均大于根结点的值；
- 3)其左右子树本身又各是一棵二叉排序树

2. 性质

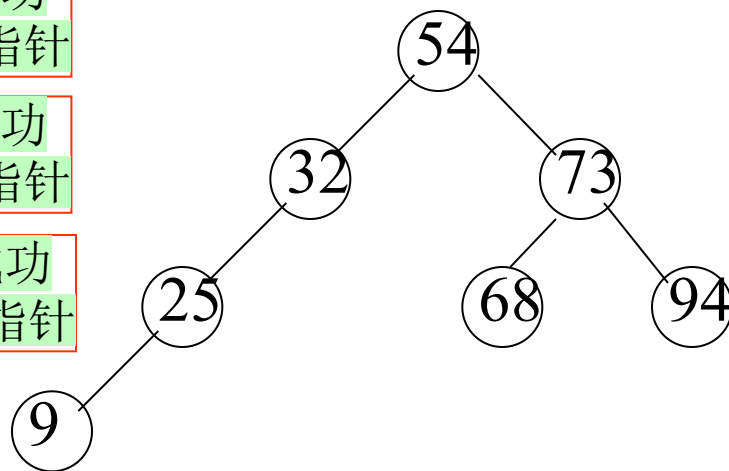
中序遍历一棵二叉排序树，将得到一个以关键字递增排列的有序序列



二叉排序树是动态查找树，即在查找时将未找到的给定值的记录插入到查找树中，查找树是动态生成的。

例： 有查找请求 {54、32、54、25、73、25、94、68、32、73、9}，查找结果如下：

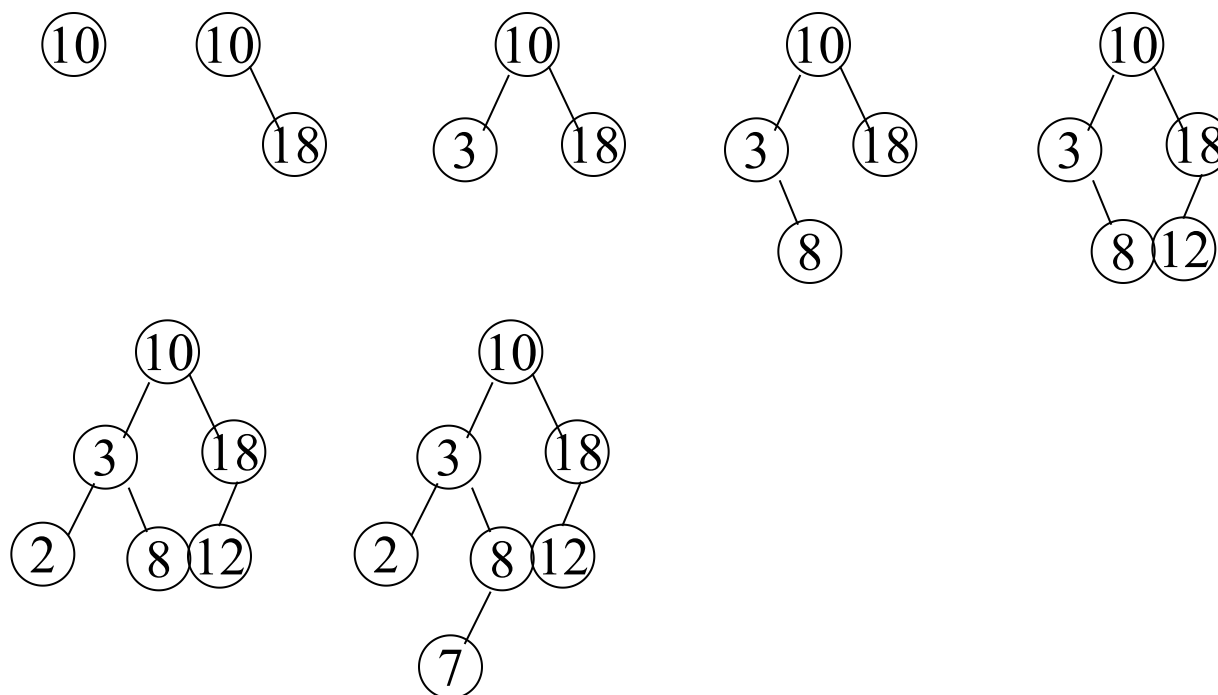
54查找失败 插入54	32查找失败 插入32	54查找成功 返回结点指针
25查找失败 插入25	73查找失败 插入73	25查找成功 返回结点指针
94查找失败 插入94	68查找失败 插入68	32查找成功 返回结点指针
73查找成功 返回结点指针	9查找失败 插入9	



3) 生成(查找并插入)

从空树出发，反复调用插入操作InsertBST（或带插入的SearchBST）即可。

[例] {10, 18, 3, 8, 12, 2, 7}



3. 在二叉排序树上的操作

通常，取二叉链表作为存储结构

1) 查找（只查找，不插入）- 递归

```
Bitree SearchBST ( BiTree T, KeyType key )
```

```
//在二叉排序树T中查找关键字值为 key 的结点，
```

```
//找到返回该结点的地址，否则返回空。
```

```
{
```

```
    if( (!T) || EQ(key, T->data.key))
```

```
        return T;
```

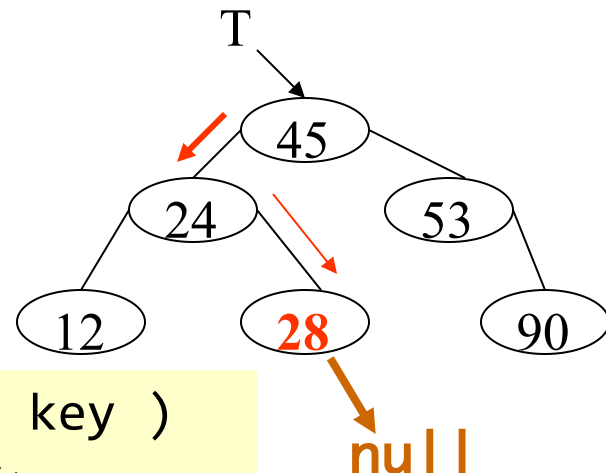
```
    else if ( LT(key, T->data.key))
```

```
        return (SearchBST(T->lchild, key));
```

```
    else
```

```
        return (SearchBST(T->rchild, key));
```

```
} //SearchBST P228-9.5 (a)
```



key=28

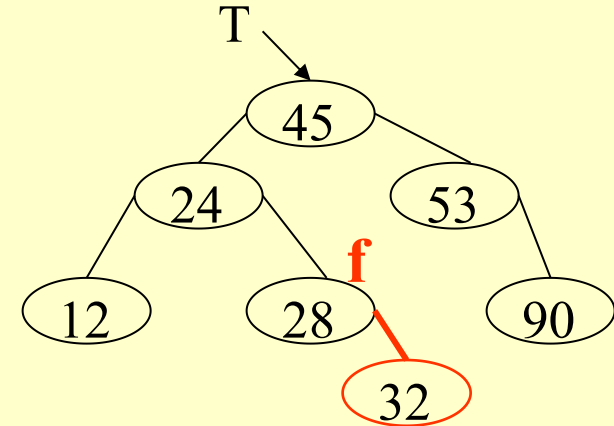
查找成功

key=32

查找失败

1)查找（只查找，不插入）- 递归

```
bool SearchBST(BiTree T, KeyType key, BiTree f, BiTree &p)
//f指向当前结点的双亲,初始调用值为NULL。
//查找成功, p指向该结点, 并返回TRUE;
//查找失败, p指向查找路径上的最后一个结点, 并返回FALSE
{
    if(!T)
    {
        p=f;
        return FALSE;
    }
    else if EQ(key, T->data.key)
    {
        p=T;
        return TRUE;
    }
    else if LT(key, T->data.key)
        return SearchBST(T->lchild, key, T, p);
    else
        return SearchBST(T->rchild, key, T, p);
} //SearchBST    P228-9.5(b)
```

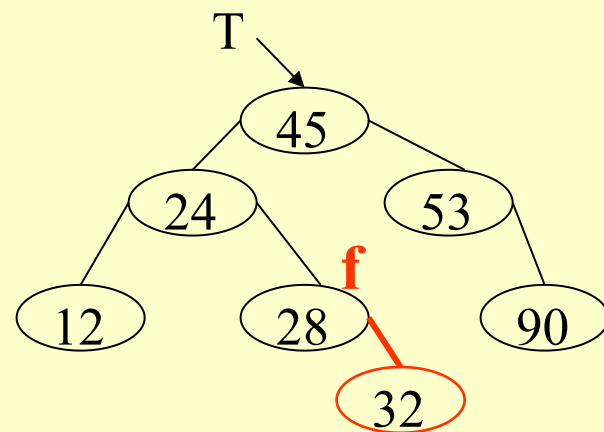


1)查找（只查找，不插入）- 非递归

```
bool SearchBST(BiTree T, KeyType key, BiTree &p)
//查找成功，p指向该结点，并返回TRUE；
//查找失败，p指向查找路径上的最后一个结点，并返回FALSE
{
    BiTree p = T;  //p作为工作指针
    BiTree q = NULL; //q用于跟踪p的双亲，根的双亲为NULL
    while(p)
    {
        if(EQ(key, p->data.key))
            return TRUE;

        q = p;
        if(LT(key, p->data.key))
            p = p->lchild;
        else
            p = p->rchild;
    }
    //此时q是空指针p的双亲结点，当需要插入key时即插入在q的下面；

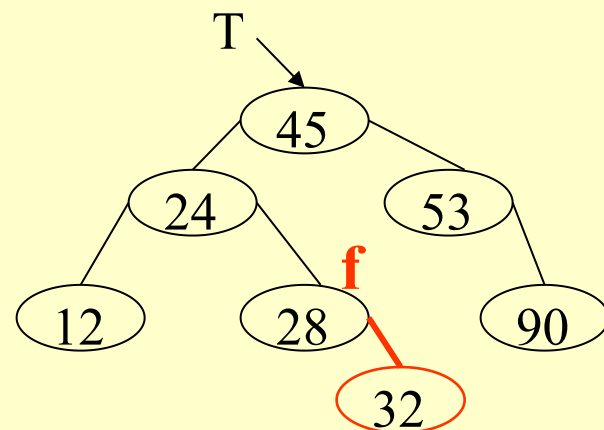
    p = q; //将p置为q的值，返回
    return FALSE;
}
```



2) 插入-递归

```
bool InsertBST (BiTree &T, ElemType e)
{
    if(!SearchBST(T, e.key, NULL, P))
    {
        s=(BiTree)malloc(sizeof(BiTNode));
        s->data=e;
        s->lchild=s->rchild=NULL;

        if(!T)
            T=s;    //空二叉排序树
        else if LT(e.key, p->data.key)
            p->lchild = s; //小则插到p的左孩子
        else
            p->rchild = s; //大则插到p的右孩子
        return TRUE;
    }
    else
        return FALSE
} //InsertBST    P228-9.6
```



3)查找并插入(递归)

```
Status SearchBST(BiTree &T, KeyType key, BiTree &p)
```

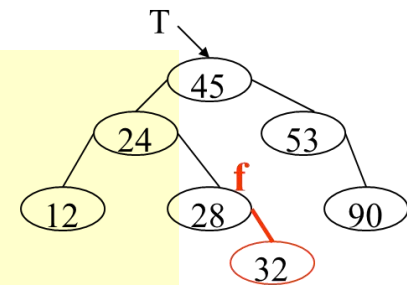
```
{  
    if(T) return SertchInsertBST(T, NULL, key, p);  
    //空树，将第一个结点插入  
    T=(BiTree)malloc(sizeof(BiTNode));  
    if(!T) return OVERSTACK;  
    T->data=key; T->lchild=T->rchild=NULL; p = T;  
    return OK;  
}
```

```
Status SearchInsertBST(BiTree T, BiTree f, KeyType key, BiTree &p)
```

```
//f指向当前结点的双亲,初始调用值为NULL。
```

```
//查找key成功, p指向该结点; 查找key失败, 插入keyy, p指向新结点, 返回OK
```

```
{  
    if(!T) {  
        if(!(p = (BiTree)malloc(sizeof(BiTNode)))) return OVERSTACK;  
        p->data=key; p->lchild=s->rchild=NULL;  
        if LT(key, f->data.key) // 这里为什么不担心f为NULL?  
            f->lchild = p; //小则成为f的左孩子  
        else  
            f->rchild = p; //大则成为f的右孩子  
        return OK;  
    }  
    else if EQ(key, T->data.key){  
        p=T; return OK;  
    }  
    else if LT(key, T->data.key)  
        return SearchInsertBST(T->lchild, T, key, p);  
    else  
        return SearchInsertBST(T->rchild, T, key, p);  
}
```



key=28

查找成功

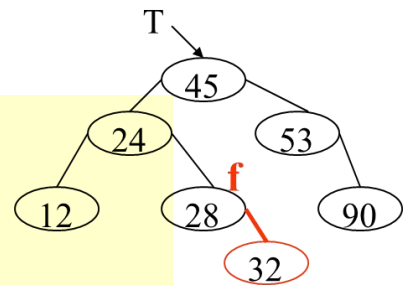
key=32

查找失败

插入32

3)查找并插入(非递归)

```
Status SearchBST(BiTree &T, KeyType key, BiTree &p)
{
    p = T;  //p用于返回key所在结点
    Bitree q=NULL; // q用于跟踪p的双亲，根的双亲为NULL
    while(p)
    {
        if(EQ(key, p->data.key)) { return OK; }
        q=p;
        if(LT(key, p->data.key)) { p = p->lchild;}
        else {p = p->rchild; }
    }
    //此时q是空指针p的双亲结点，当需要插入key时即在q的下面；
    if(!(p = (BiTree)malloc(sizeof(BiTNode))) return
OVERSTACK;
    p->data=key; p->lchild=p->rchild=NULL;
    if(!T) //空树
        T = p;
    else if(LT(key, q->data))
        q->lchild = p;
    else
        q->rchild = p;
    return OK;
}
```



key=28

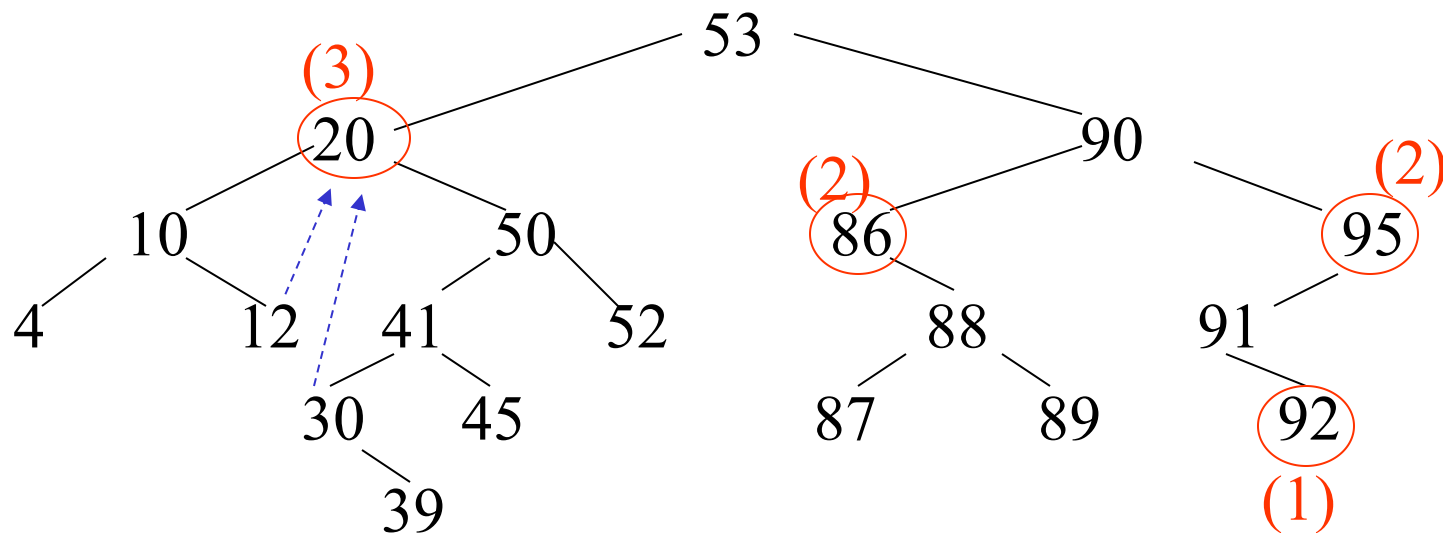
查找成功

key=32

查找失败

插入32

4) 删除

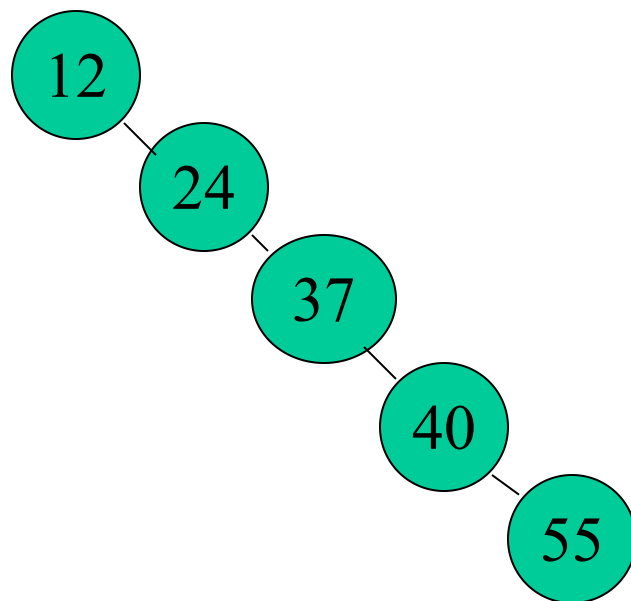
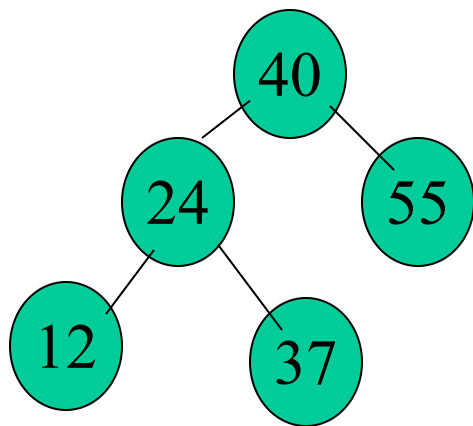


被删除结点为*p的不同情况分析：

- (1) p是叶子结点：修改其双亲指针即可
- (2) p只有左孩子：用p的左子树代替以p为根的子树
- (2) p只有右孩子：用p的右子树代替以p为根的子树
- (3) p有两个孩子：找到p的中序后继(或前趋)结点q；
q的数据复制给p；
递归处理q的删除问题。

4. 性能分析

- 给定树的形态，等概率查找成功时的 $ASL = \sum c_i / n$
 - 最差(单支树): $(n+1)/2$
 - 最好(类似折半查找的判定树): $\log_2(n+1)-1$
 - 随机: $\leq 2(1 + \frac{1}{n})\ln n$ P232 (9-20)
- 关键字有序出现时，构造出“退化”的二叉排序树，树深为 n ，各种操作代价 $O(n)$ 。避免方法：采用平衡二叉树



9.3.2 平衡二叉树(AVL树)

1.平衡二叉树的定义

或者是空树，或者是满足如下性质的**二叉排序树**：

1)它的左、右子树的高度之差的绝对值不超过1；

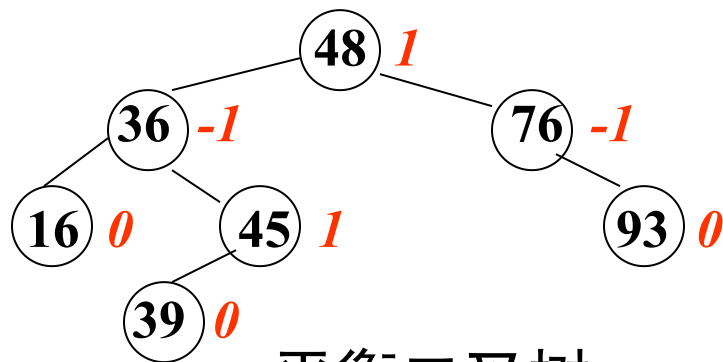
$$|h_L - h_R| \leq 1$$

2)其左右子树本身又各是一棵平衡二叉树。

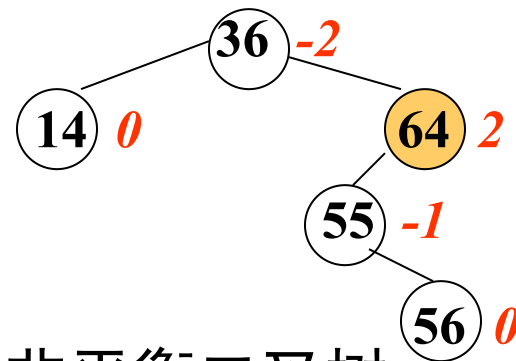
二叉树上结点的**平衡因子**：

该结点的左子树高度减去右子树的高度。

由两位前苏联数学家**A**del'son-**V**el'sii和**L**andis于1962年给出。



平衡二叉树



非平衡二叉树

平衡二叉树不一定是理想的最矮状态。

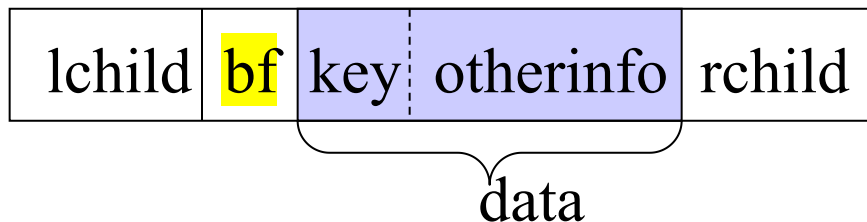
2. 平衡树查找的分析(P238)

一个具有n个结点的平衡二叉树的最大深度h为

$$\log_a(\sqrt{5} * (n+1)) - 2, \text{ 其中 } a = (1 + \sqrt{5}) / 2$$

结论：最坏情况下，AVL树的高度约为 $1.44\log_2 n$ ，而完全平衡的二叉树高度约为 $\log_2 n$ ，因此AVL树是接近最优的，其平均查找长度与 $\log_2 n$ 同数量级。

3. 结点的存储结构



lchild: 左孩子指针

rchild: 右孩子指针

bf: 平衡因子

key: 记录的关键字

```
typedef struct {
    KeyType key;
    .....
}ElemType
typedef struct BSTNode {
    ElemType data;
    int bf;
    struct BSTNode *lchild,
                  *rchild;
}BSTNode, *BSTree;
```

4. 在平衡二叉树上的操作

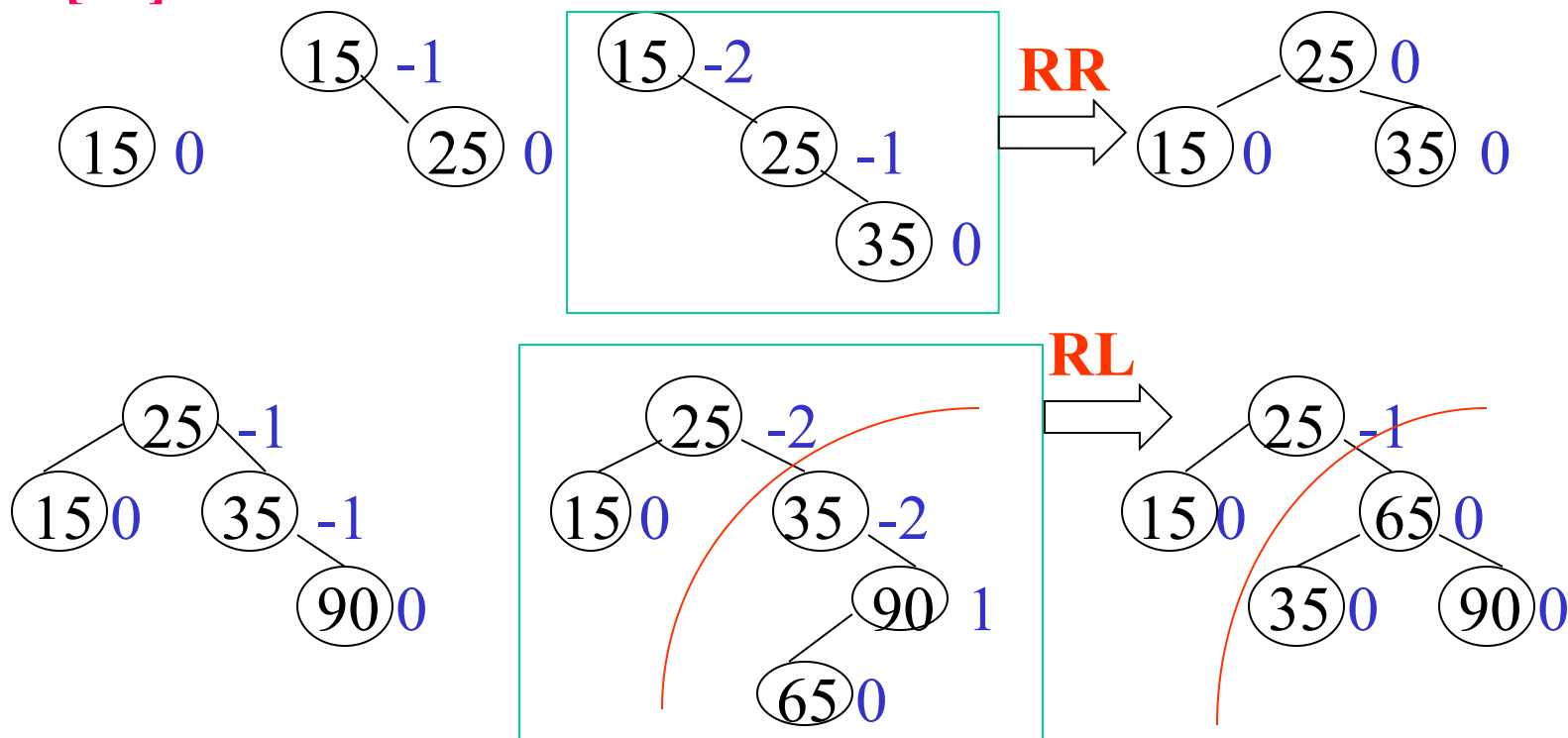
1) 查找

查找方法同二叉排序树。

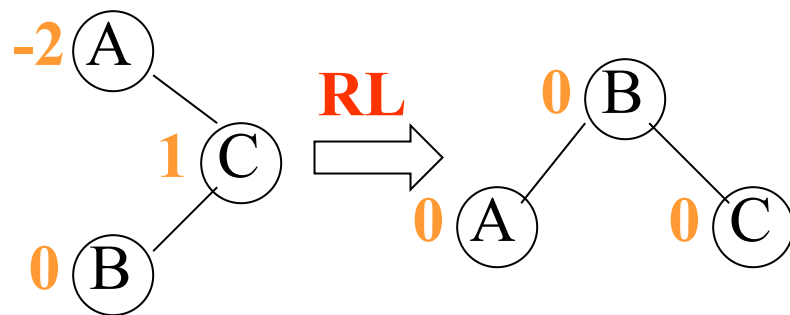
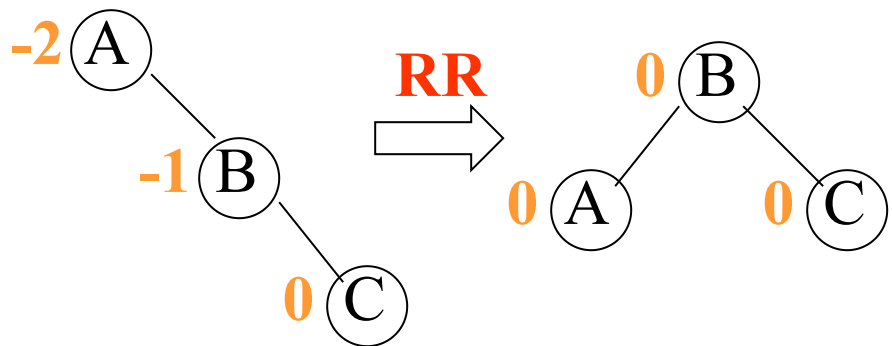
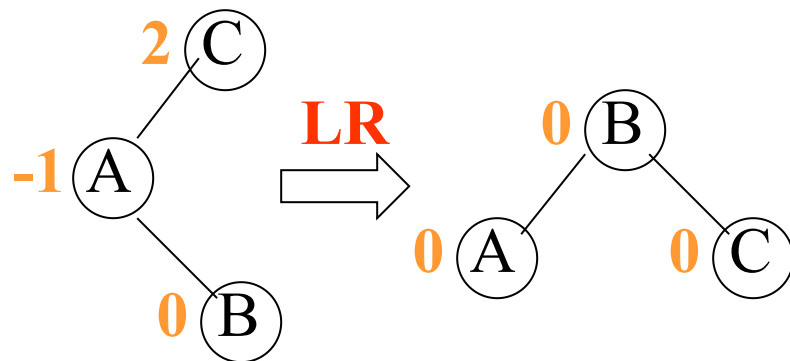
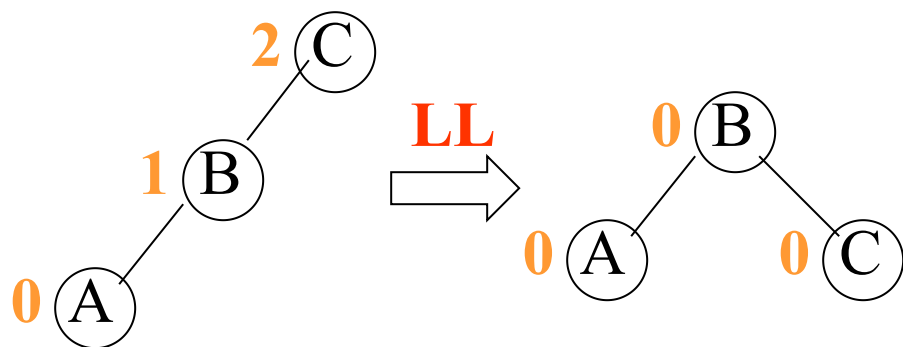
2) 插入

[例] 平衡二叉树的生成过程

在插入过程中采用平衡旋转技术

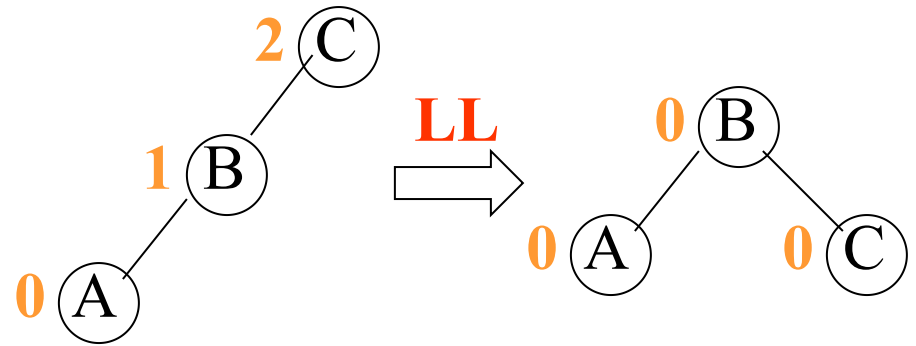


四种平衡旋转---将插入后非平衡的子树转为平衡

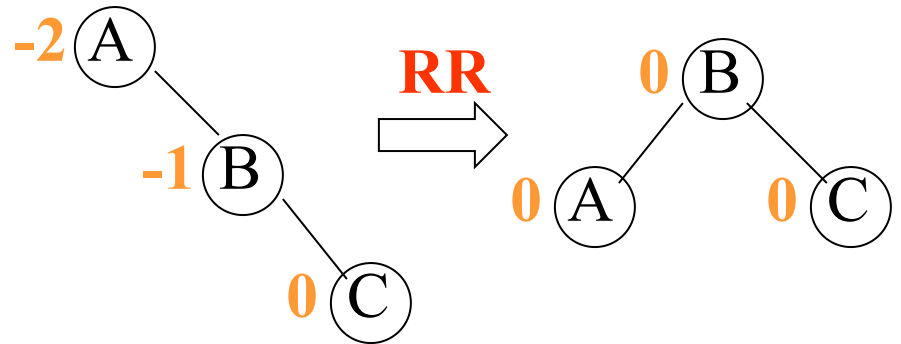


Single & Double Rotations

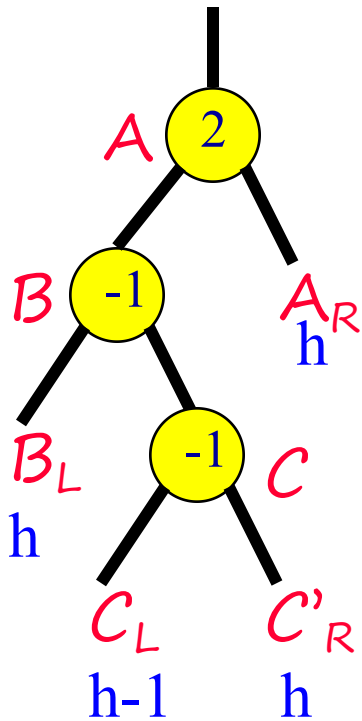
- Single
 - LL and RR



- Double
 - LR and RL
 - LR is RR followed by LL
 - RL is LL followed by RR

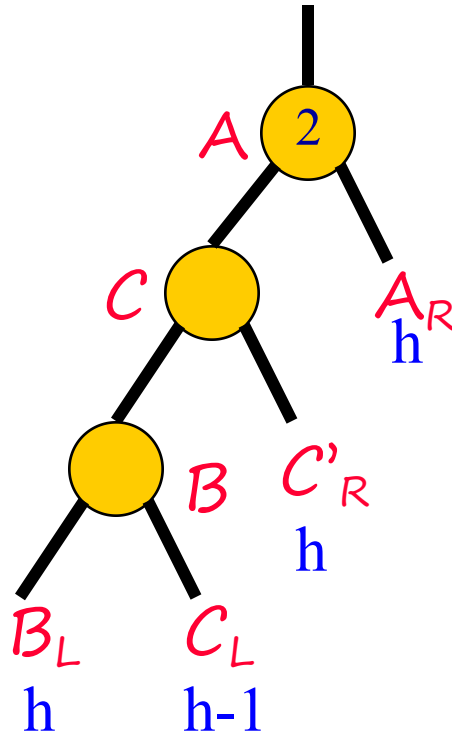


LR Is $RR + LL$



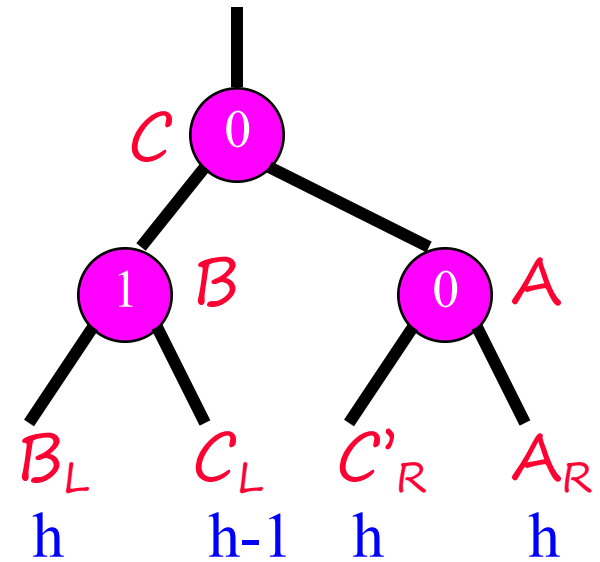
ABC is LR struct

BC is RR struct



After RR rotation.

ACB is LL struct

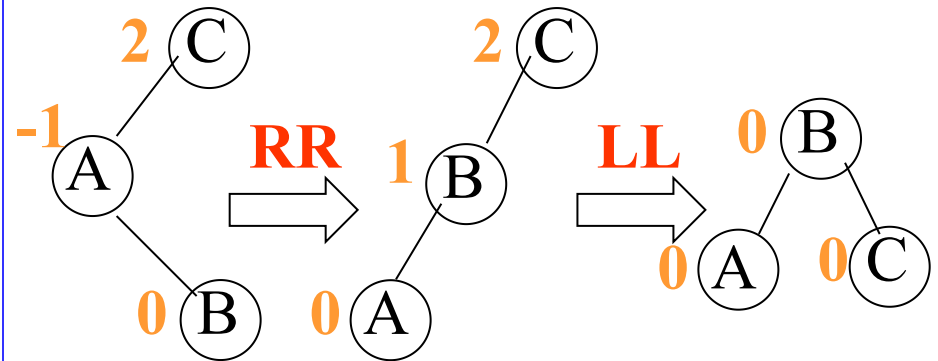


After LL rotation.

Single & Double Rotations

- Single

- LL and RR

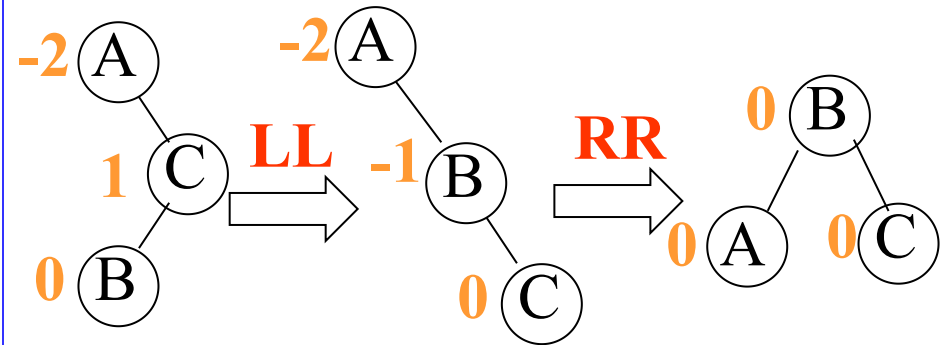


- Double

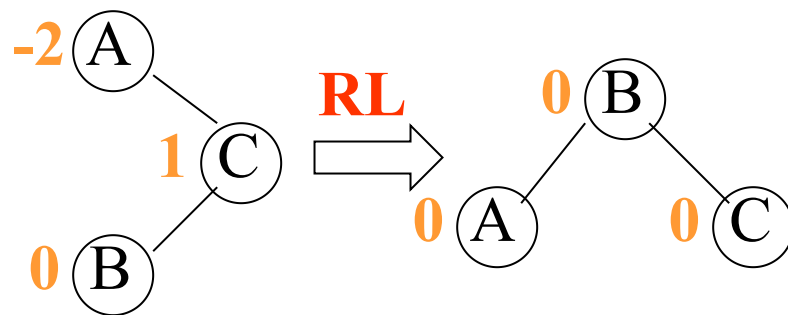
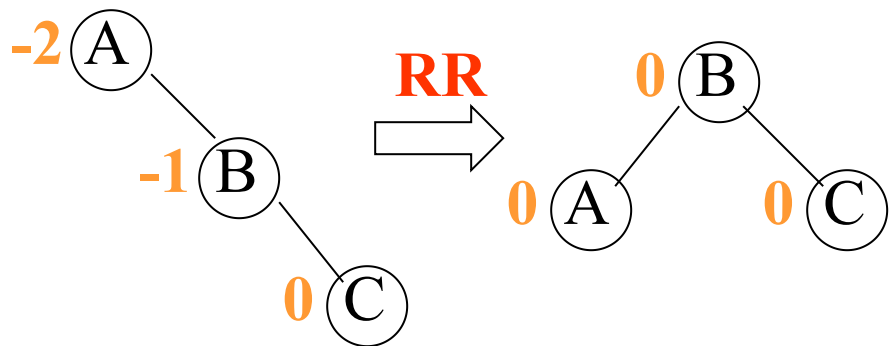
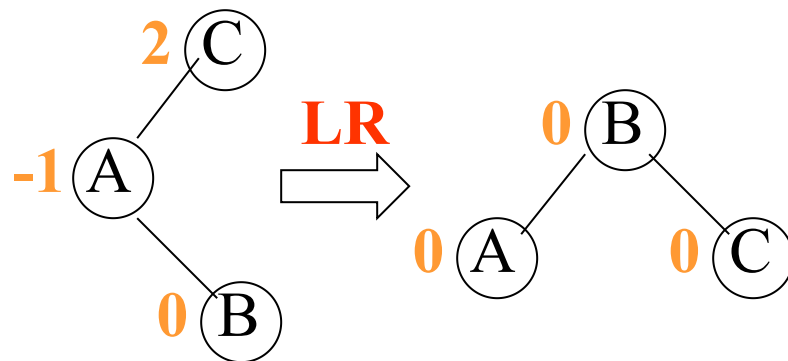
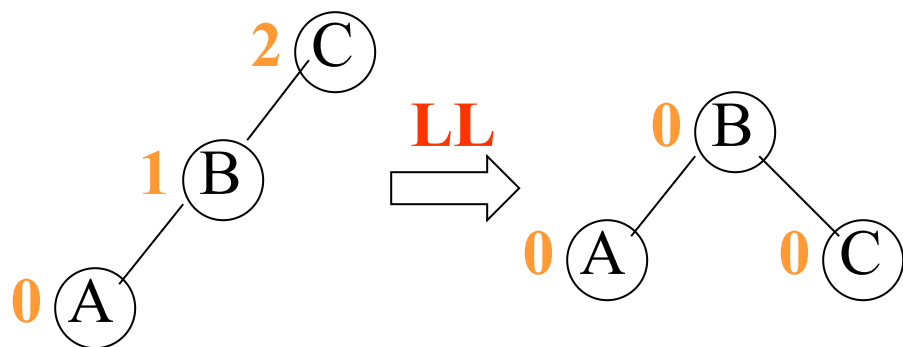
- LR and RL

- LR is RR followed by LL

- RL is LL followed by RR



四种平衡旋转---将插入后非平衡的子树转为平衡

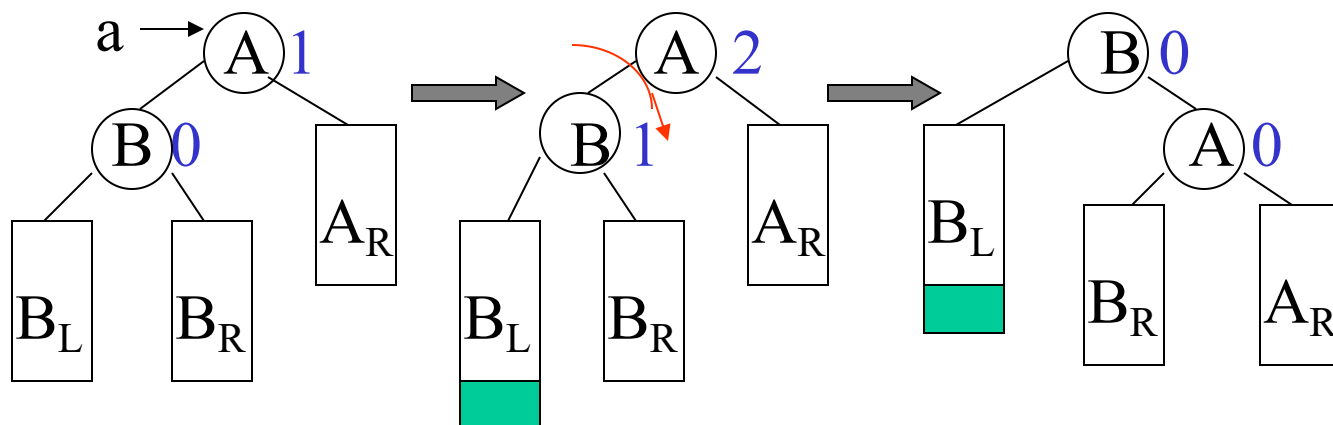


[调整范围的确定]

插入结点后，找到离插入结点最近且平衡因子绝对值超过1的祖先结点，则以该结点为根的子树将是可能不平衡的**最小子树**，可将重新平衡的范围局限于这棵子树。

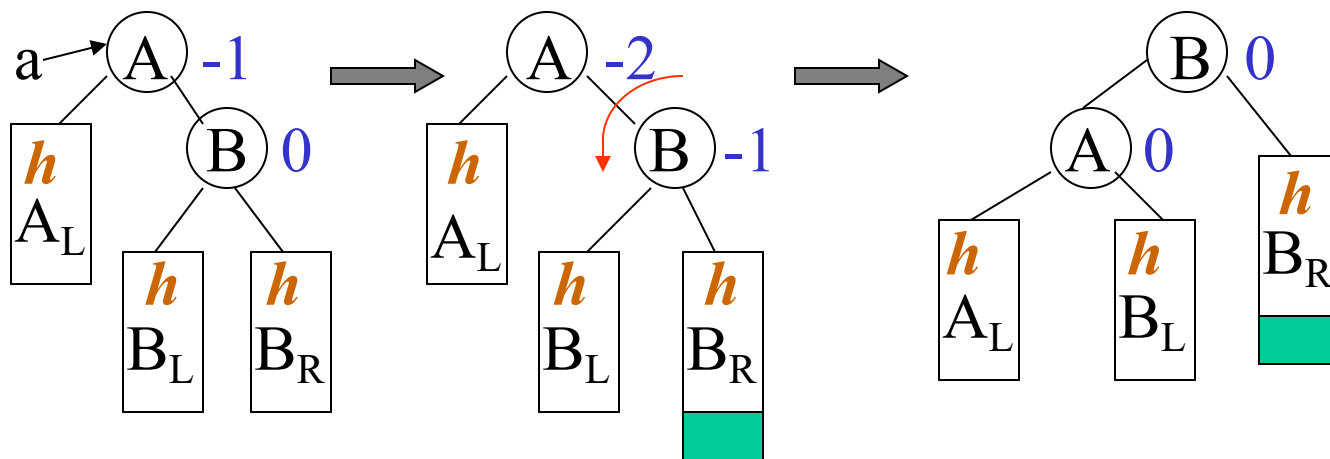
[调整的规律] 设失去平衡的最小子树的根结点指针为a

- **LL型平衡旋转**——单向右旋（一次顺时针旋转）

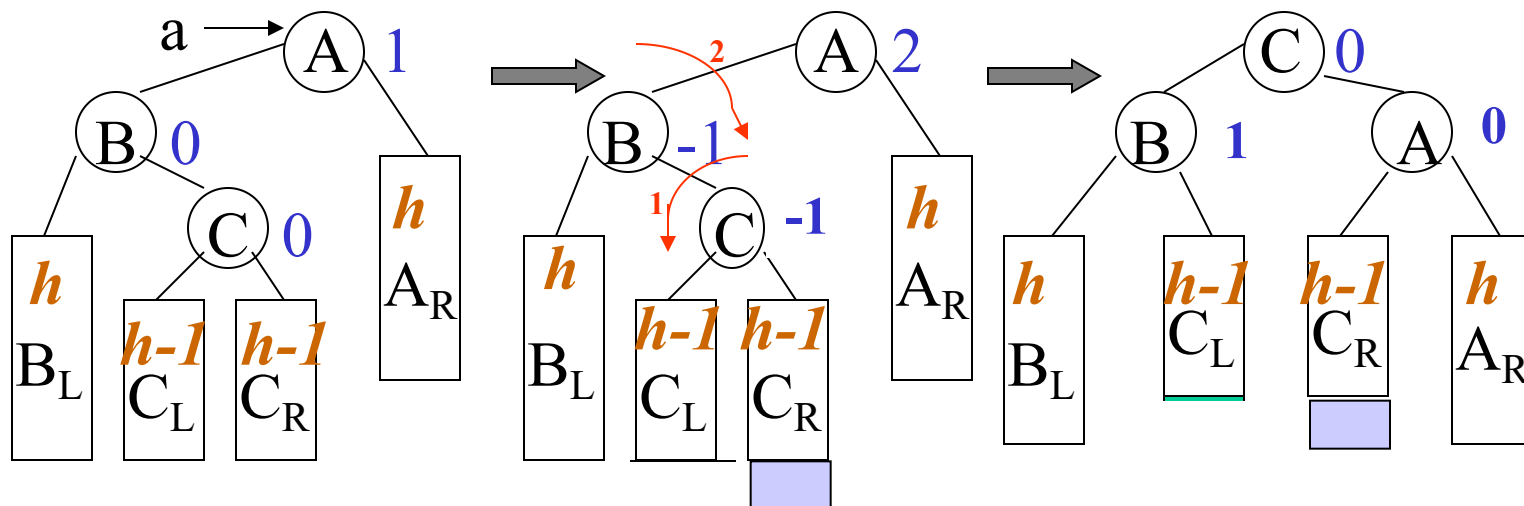


 插入的结点

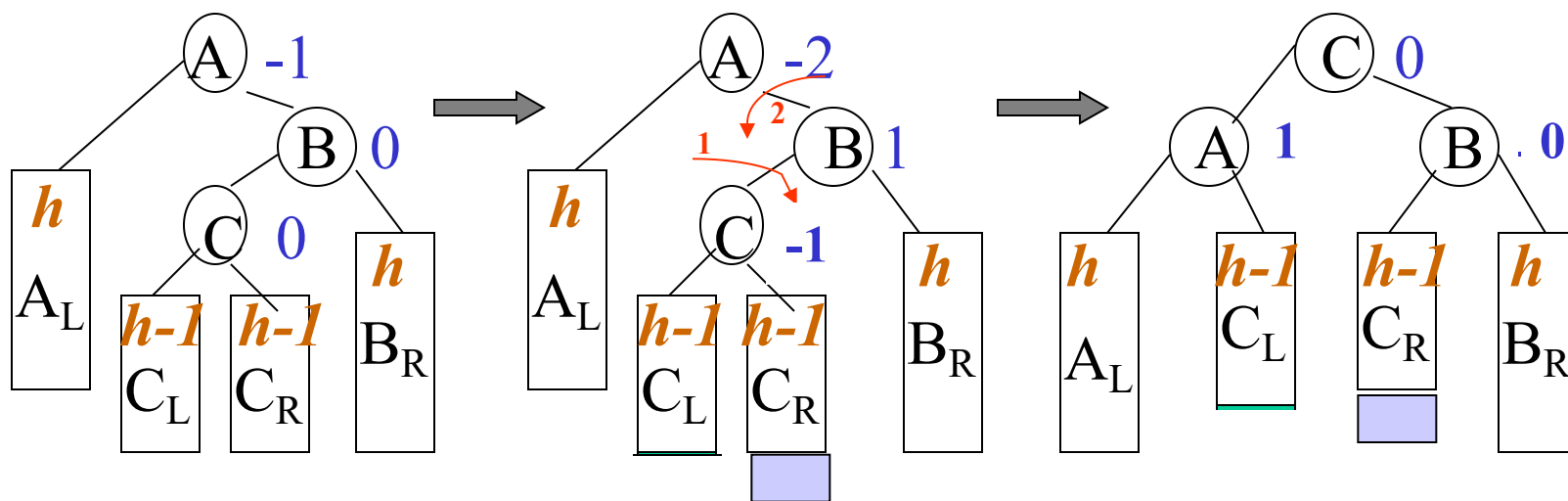
- **RR型平衡旋转**——单向左旋（一次逆时针旋转）



- **LR型平衡旋转**——一次逆时针旋转+一次顺时针旋转



- **RL型平衡旋转**——一次顺时针旋转+一次逆时针旋转



[例]

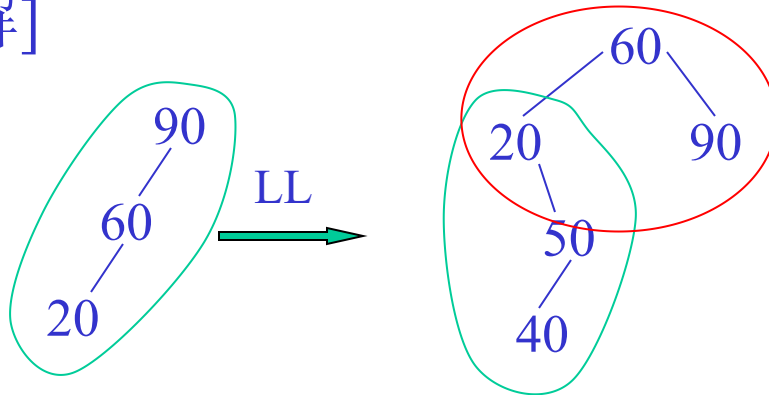
3)删除 （思路同插入）

- 将删除结点 q 转化为 q 最多有一个孩子的情形，即若 q 有两个孩子，则以其中序前驱/后继结点 r 取代它，删除 r ；
- 若树的平衡性被破坏，利用单一/双重旋转恢复。

以下表顺序建立平衡二叉排序树，并求在等概率情况下查找成功的平均查找长度。

(90,60,20,50,40,30,10,110,100,70,120,80)

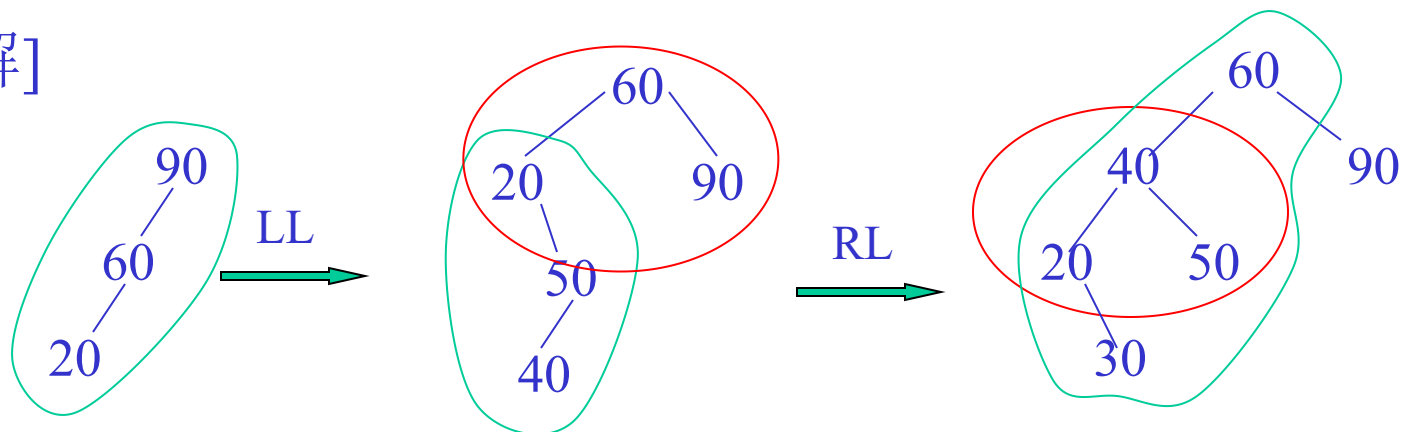
[解]



以下表顺序建立平衡二叉排序树，并求在等概率情况下查找成功的平均查找长度。

(90,60,20,50,40,30,10,110,100,70,120,80)

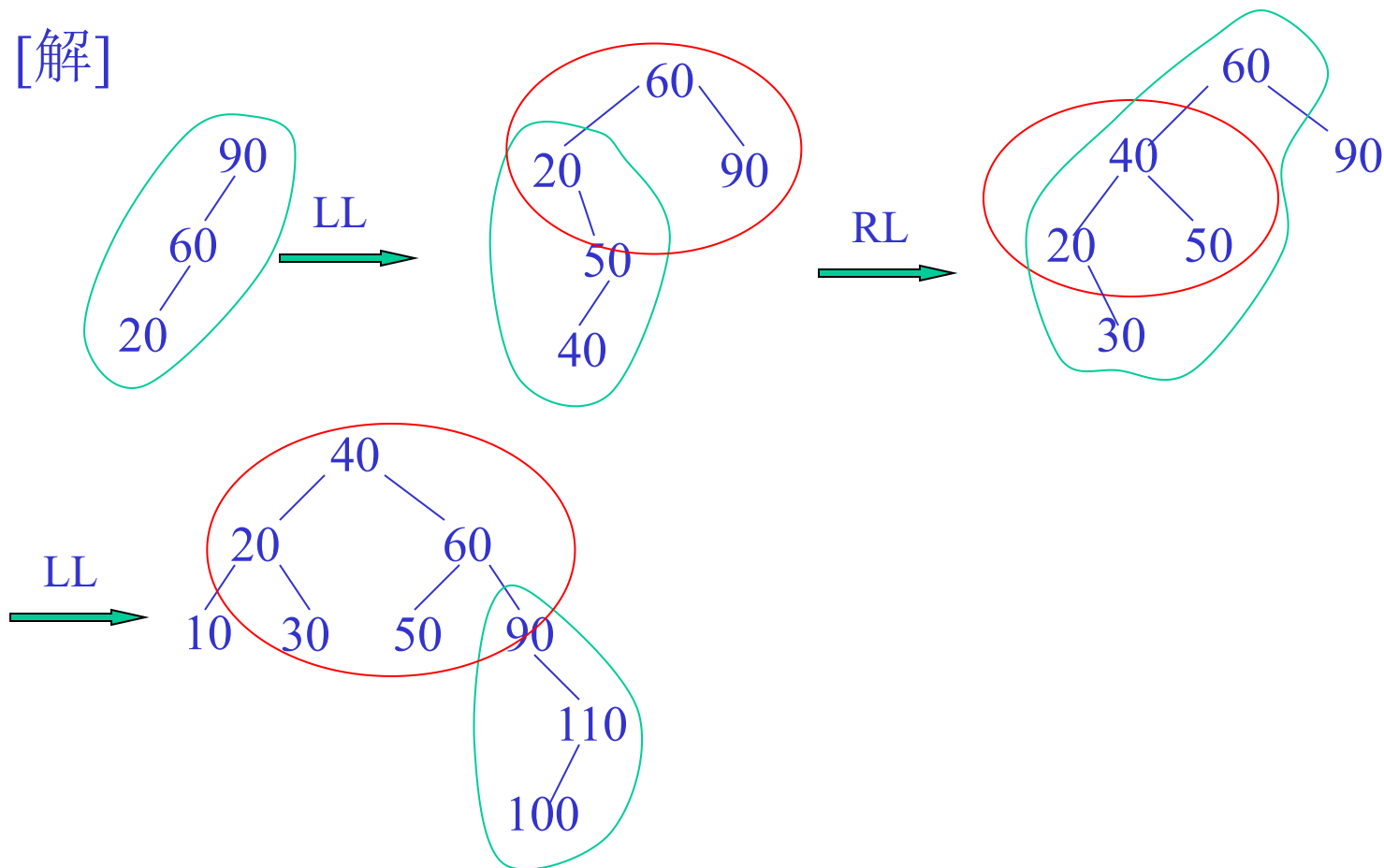
[解]



以下表顺序建立平衡二叉排序树，并求在等概率情况下查找成功的平均查找长度。

(90,60,20,50,40,30,10,110,100,70,120,80)

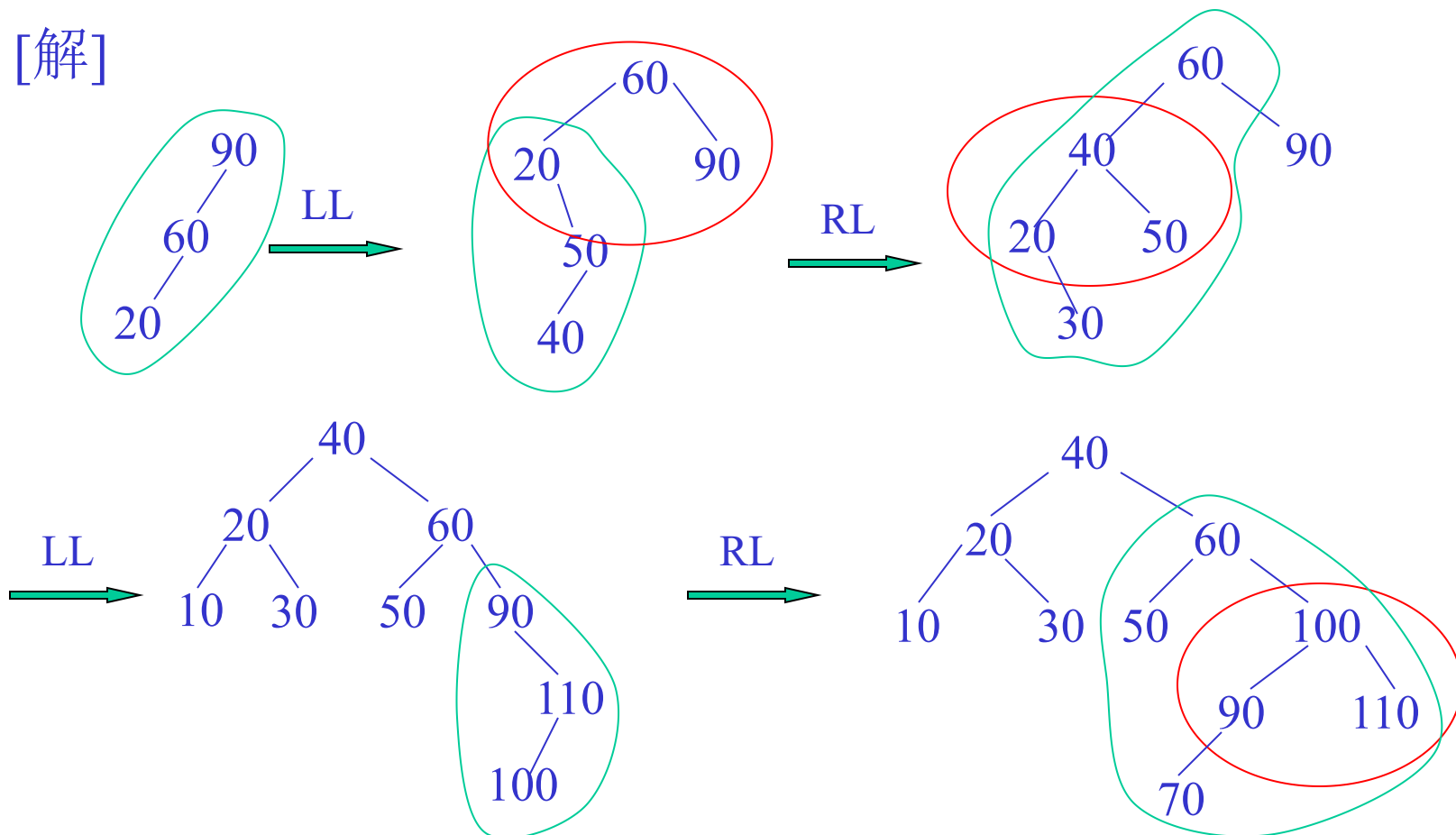
[解]



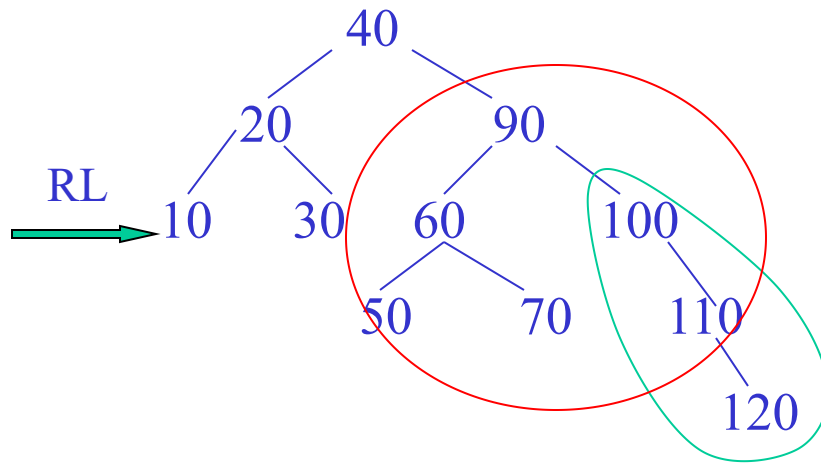
以下表顺序建立平衡二叉排序树，并求在等概率情况下查找成功的平均查找长度。

(90,60,20,50,40,30,10,110,100,70,120,80)

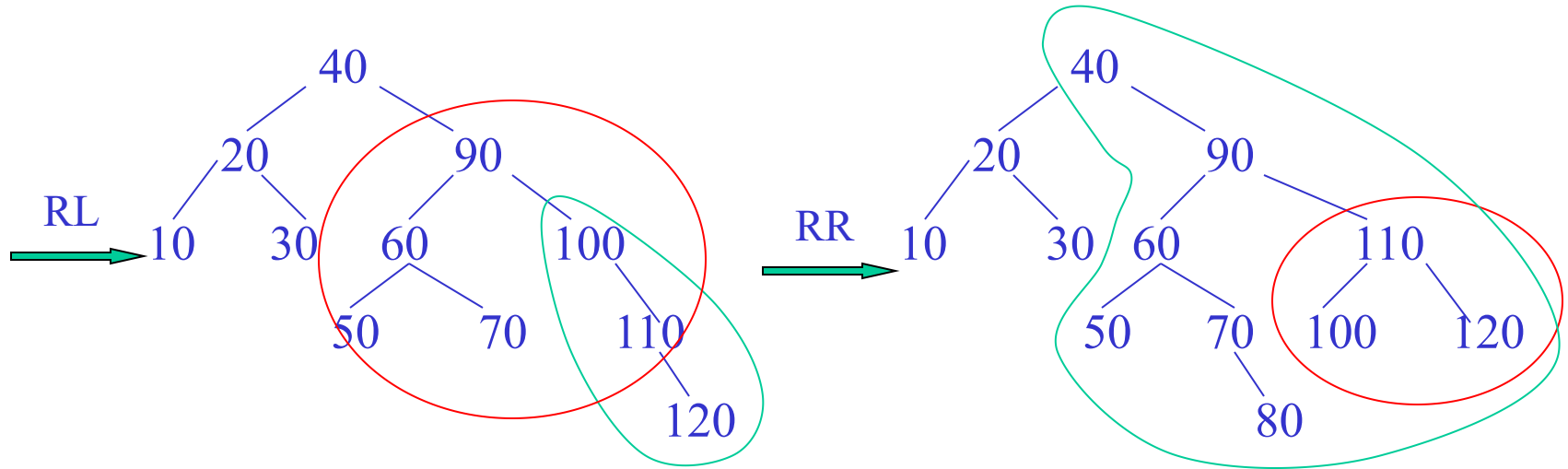
[解]



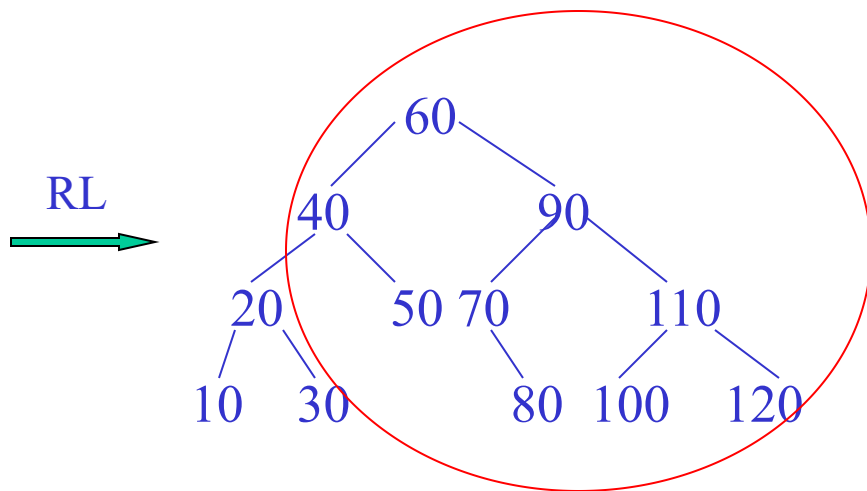
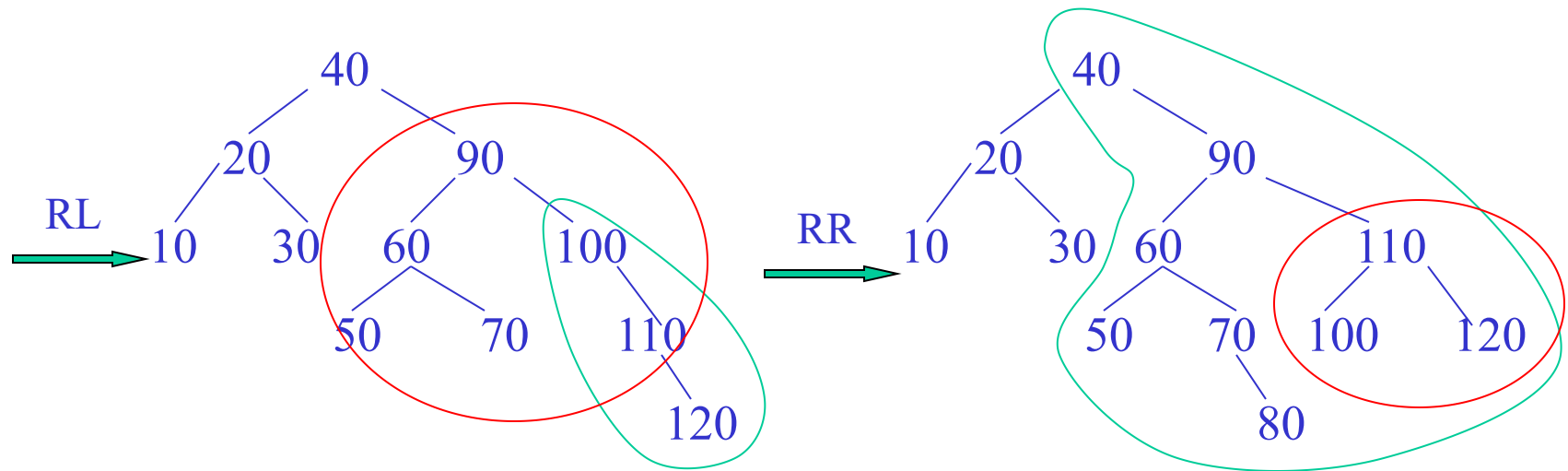
(90,60,20,50,40,30,10,110,100,70,120,80)



(90,60,20,50,40,30,10,110,100,70,120,80)



(90,60,20,50,40,30,10,110,100,70,120,80)



$$ASL = (1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4) / 12$$

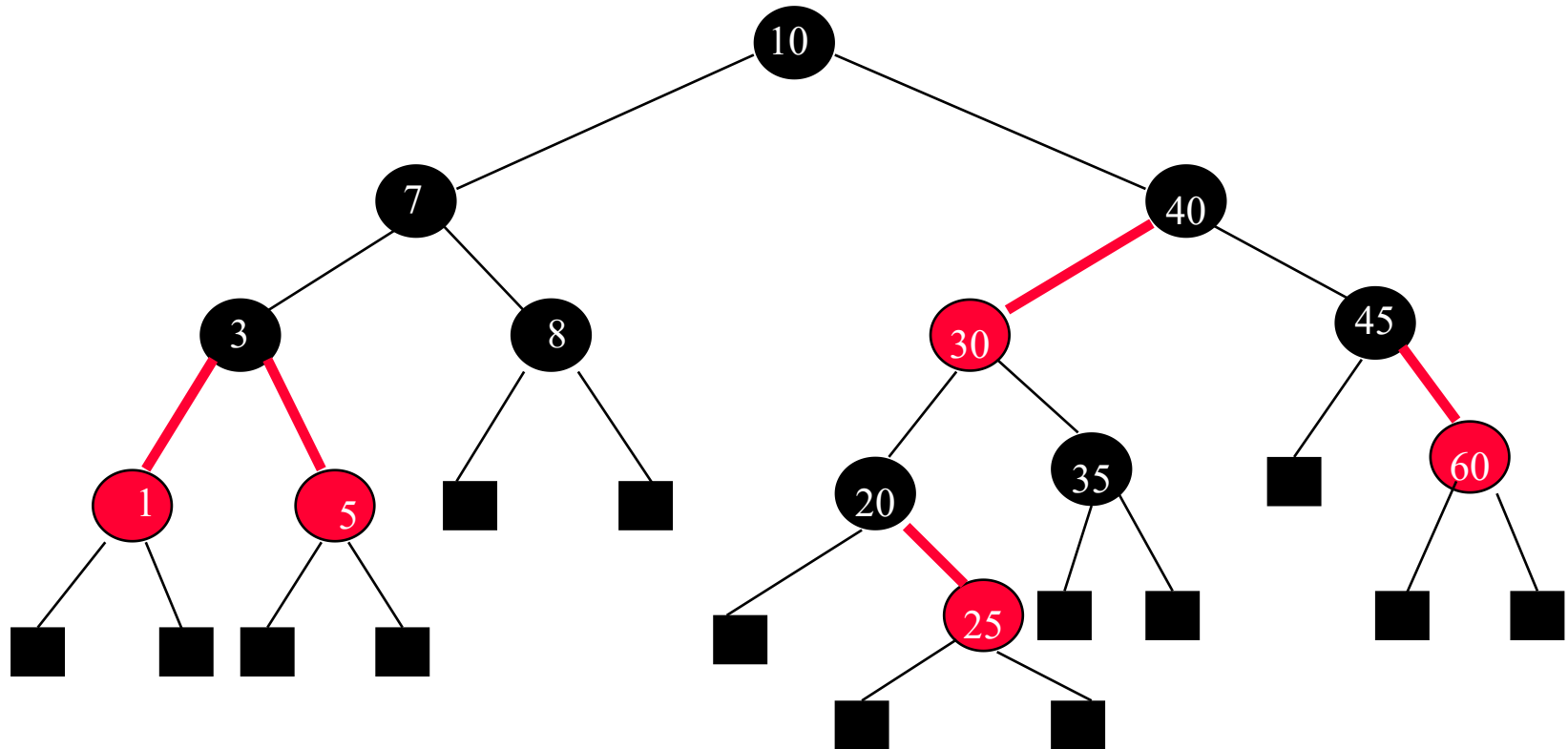
$$= 37 / 12$$

红黑树

- 红黑树（Red Black Tree）是一种自平衡二叉查找树；
- 红黑树是一种特化的AVL树（平衡二叉树），都是在进行插入和删除操作时通过特定操作保持二叉查找树的平衡，从而获得较高的查找性能。
- 它可以在 $O(\log n)$ 时间内做查找，插入和删除。
- 红黑树是一种平衡二叉查找树的变体，它的左右子树高差有可能大于 1，所以红黑树不是严格意义上的平衡二叉树（AVL），但对之进行平衡的代价较低，其平均统计性能要强于 AVL。

红黑树

- 红黑树示例



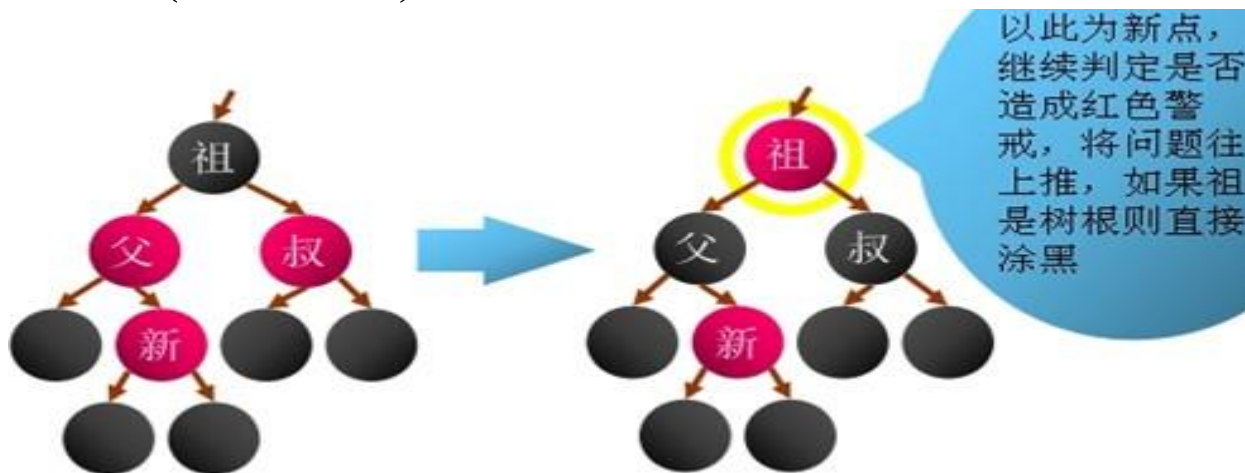
红黑树

- 红黑树的特征：
 - 性质1.红黑树是每个节点都带有颜色属性的二叉查找树，颜色或红色或黑色；
 - 性质2.根节点是黑色。
 - 性质3.所有叶子都是黑色。（叶子是NULL节点，External Node）
 - 性质4.每个红色节点的两个子节点都是黑色。（从每个叶子到根的所有路径上不能有两个连续的红色节点）
 - 性质5.从任一节点到其每个叶子的所有路径都包含相同数目的黑色节点。
 - 从根到叶子的最长的可能路径不多于最短的可能路径的两倍长。

红黑树

• 节点插入算法

- 插入过程首先是根据一般二叉查找树的插入步骤，把新节点 z 插入到某个叶节点的位置上（新结点的两个子节点成为新的叶节点，着黑色）；
- 然后将新节点 z 着为红色
 - 若 z 的父结点为黑色，此时即为红黑树；
 - 若 z 的父结点为红色，对有关节点重点着色(Color flip)和旋转(Rotation)来恢复红黑树的性质



红黑树

- 节点删除算法
 - 与红黑树的插入算法一样，对一个节点的删除算法要花 $O(\log n)$ 时间；
- 恢复红黑属性需要少量($O(\log n)$)的颜色变更（这在实践中是非常快速的）并且不超过三次树旋转（对于插入是两次）。
- 这允许插入和删除的时间复杂度保持为 $O(\log n)$

红黑树

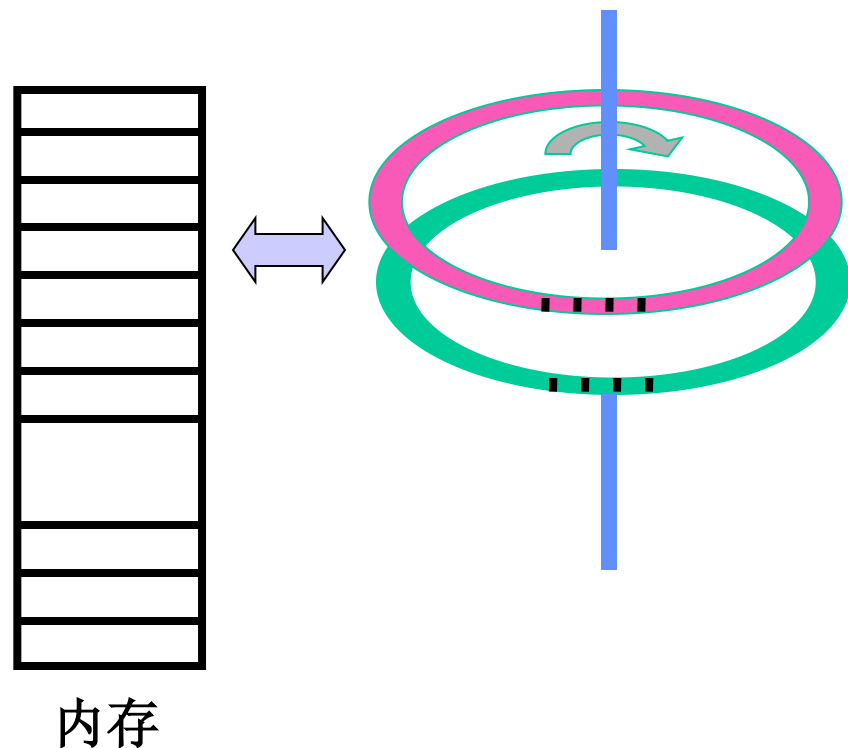
- Red-Black tree, 简称RB-Tree；在1972年由鲁道夫·贝尔发明，他称之为“对称二叉B树”；它现代的名字是在 Leo J. Guibas 和 Robert Sedgewick 于1978年写的一篇论文中获得的。
- **特点**：利用对树中的结点“红黑着色”的要求，降低了平衡性的条件，达到**局部平衡**，有着良好的最坏情况运行时间，它可以在 $O(\log n)$ 时间内做查找，插入和删除，这里的 n 是树中元素的数目。
- C++ STL中的关联式容器：集合set、多重集合multiset、映射map、多重映射multimap
- linux进程调度Completely Fair Scheduler，用红黑树管理进程控制块
- Java hashmap、TreeMap；
- epoll在内核中的实现，用红黑树管理事件块
- Mysql的Innodb数据存储。

9.3.3 B-树 (B-tree, B即balanced)

B-树是一种平衡的多路查找树。可应用于文件系统。

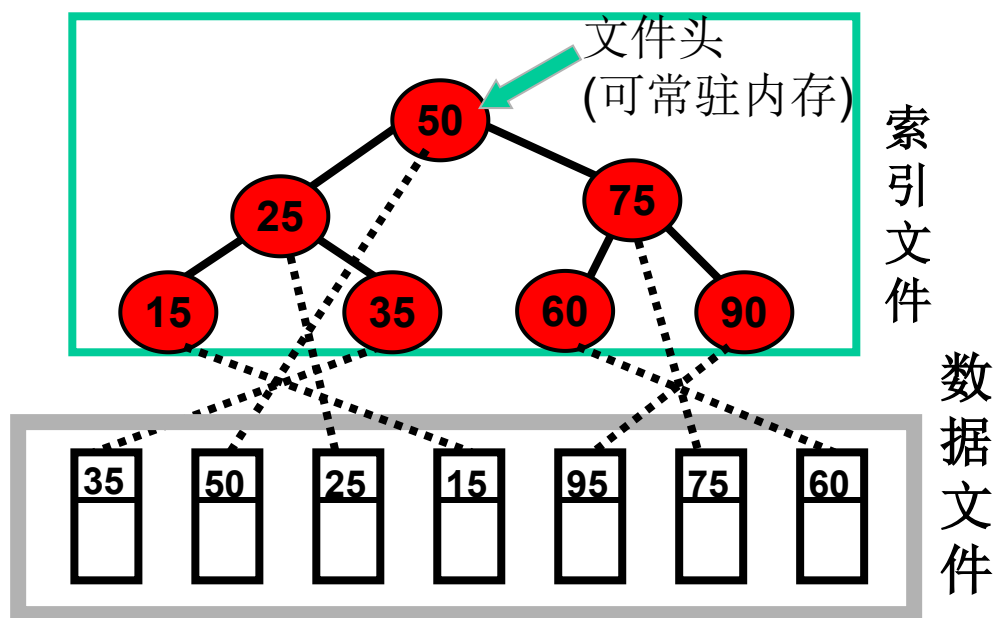
海量数据存放在外存中，不可能一次全部调入内存。因此，要多次访问外存。但硬盘的驱动受机械运动的制约，速度慢。所以，主要矛盾变为减少访外次数。

1970 年由 R bayer 和 E macreight 提出用B-Tree作为索引组织文件。提高访问速度、减少时间开销。



例: 采用平衡二叉树为数据文件的每条记录按**key**值编制一个索引
（索引中保存了记录所在的磁盘文件位置和**key**值）；索引文件
存放在磁盘中，数据文件也存放在磁盘中。

当文件的记录个数为 100,000 时，要找到给定关键字的记录，按
平衡二叉树的时间复杂度计算，平均需访问外存17次（
 $\log_2 100,000$ ）；最坏情况下，会需要24次（最坏情况下，AVL树
的高度约为 $1.44\log_2 n$ ）。



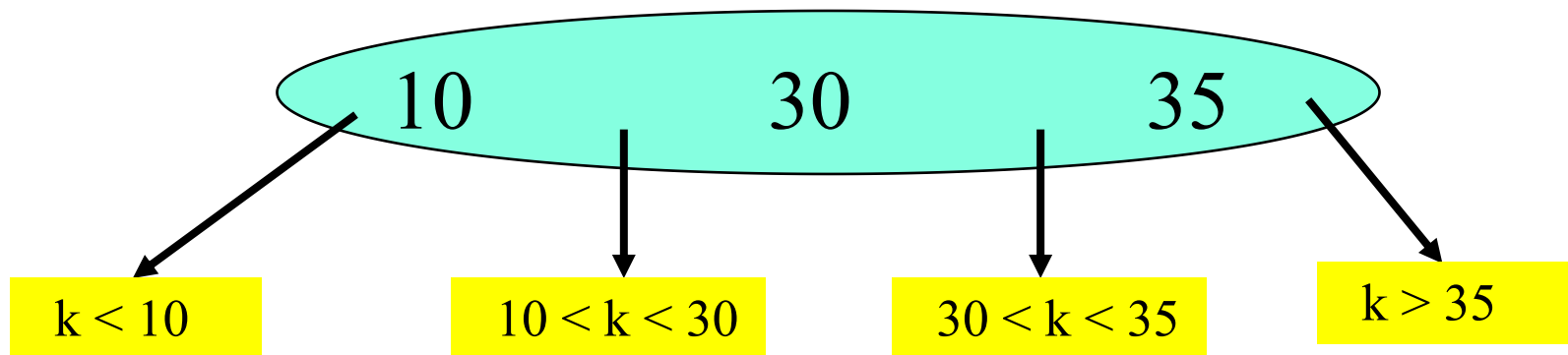
- 考虑非常大的文件时，索引树的结点也非常多
 - $n = 2^{30} = 10^9$ (approx).
- AVL
 - $30 \leq \text{height} \leq 43$.
 - When the AVL tree resides on a disk, up to 43 disk accesses are made for a search.
 - This takes up to (approx) 4 seconds.
- Red Black Tree
 - $30 \leq \text{height} \leq 60$.
 - When the red-black tree resides on a disk, up to 60 disk accesses are made for a search.
 - This takes up to (approx) 6 seconds.
- Not acceptable

优化方向：

降低树的高度，即把二叉树改成多叉树，称为多路(m-way)。

m-way Search Trees

- Each internal node has up to $m - 1$ pairs and m children
 - $m = 2$, 即 binary search tree.
 - $m-1$ pairs 对应 $m-1$ 个 key 值, 可以有 m 个区间, 每个区间对应一个指针 (m 个指针), 每个指针指向一个子树。
- 例: 4-Way Search Tree



Definition Of B-Tree(**B**alanced-**T**ree)

- Definition assumes external nodes (extended **m**-way search tree).
- B-tree of order **m** (**m**阶B树).
 - **m**-way search tree.
 - Not empty $\Rightarrow$ root has at least **2** children.
 - Remaining internal nodes (if any) have at least **ceil(m/2)** children.
 - External (or failure) nodes on same level.

1. 定义 B-树是一种 平衡 的 多路 查找树

一棵 **m 阶B-树**，或为空树，或为满足下列特性的 m 叉树：

- 1) 树中每个结点最多有 **m** 棵子树；
- 2) 若根结点不是叶子结点，则最少有**两**棵子树；
- 3) 除根之外的所有非叶子结点最少有 $\lceil m / 2 \rceil$ 棵子树；
- 4) 所有非叶子结点包含 $(n, A_0, K_1, A_1, K_2, \dots, K_n, A_n)$ 信息数据；其中 n 为结点中关键字个数， A_i 为指向子树的指针， K_i 为关键字。

5) 所有叶子结点在同一层上，且不带信息。

通常给这些叶子结点另外一个名字：**失败结点 (failure)**，简称为**F**。

失败结点实际中并不存在，而是将空值保存在 $h-1$ 层结点的指针域中（和二叉树的叶子结点的左右孩子指针一样）。引入失败结点的目的是为了便于分析B-tree的性能。

为什么要求所有叶子结点在同一层？
叶子结点在同一层带来了哪些其他含义？

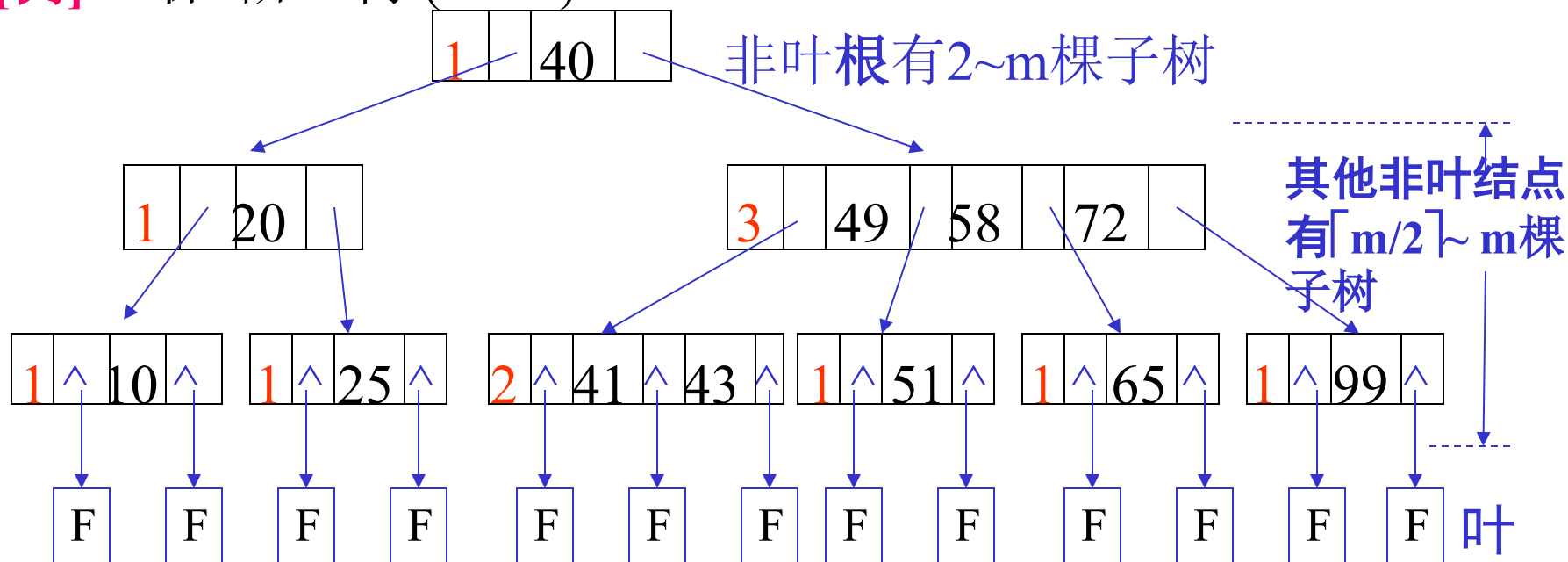
和B-Tree有关的名称：

- B-tree of order 2 is binary search tree.
- 2-3 tree is B-tree of order 3.
- 2-3-4 tree is B-tree of order 4.
- 3-4-5 tree is B-tree of order 5.

非叶结点结构 $(n, A_0, K_1, A_1, K_2, A_2, \dots, K_n, A_n)$

A_i 是子树的根节点指针， K_i 是关键字值， n 个 K_i 关键字将一系列关键字划分为 $n+1$ 段，每段在一个子树中。

[例] 一棵4阶B-树 ($m=4$)



3. B-树的存储定义

B-树节点定义

```
#define m 3 //B-树的阶

typedef struct BTreeNode {
    int keynum;      //关键字个数
    struct BTreeNode *parent;    //双亲指针
    KeyType key[m+1];    //0号未用，关键字数组
    struct BTreeNode *ptr[m+1]; //0号未用，子树指针数组
    Record *recptr[m+1]; //0号未用，磁盘记录地址数组
} BTreeNode, *BTree;
```

查找结果描述

```
typedef struct {
    BTreeNode *pt;
    int i;    //1..m，成功时第i个关键字等于key;
              //失败时，关键字在第i和第i+1的关键字之间。
    int tag; //1成功，0失败
}Result;
```

2. B-树的用途

当内存中不可能容纳所有数据记录时，在磁盘等直接存取设备上组织动态的查找表。

- B-树的根结点可以始终置于内存中；
- 其余非叶结点放置在外存上，每一结点可作为一个读取单位（页/块）
- 选取较大的阶次 m ，降低树的高度，减少外存访问次数
- B-树的信息组织方法
 - 每个记录的其它信息与关键字一起存储。查到关键字即可获取记录的完整信息。
 - 将记录的外存地址(页指针)与关键字一起存储。查到关键字时，还需根据该页指针访问外存。起到索引的作用。

4. B-树上的基本运算

1)查找

[算法思想]

设查找时给定值为K，从根结点开始

(a)在当前结点*p中顺序或者折半查找

若找到某个i，满足 $\text{key}[i]=K$ ，则查找成功，返回p、i、1

否则，确定满足 $\text{key}[i]<K<\text{key}[i+1]$ 的i值

(b)若*p结点中的ptr[i]为空指针，则查找失败，返回p、i、0

(c)从磁盘上读入ptr[i]指示的结点 $\text{DiskRead}(p \rightarrow \text{ptr}[i])$ ，将此结点作为当前结点，转(a)

[例]

K=43（查找成功） K=60（查找失败）

[算法描述]

```
Result SearchBTree(BTree T, KeyType K)
{
    p = T; q = NULL; found = FALSE; i=0; //q指向p的双亲
    while (p && ! found)
    {
        i = Search(p, K); //在p->key[1..keynum]中找i,
                          // 使得key[i]<=K<key[i+1]
        if( i>0 && p->key[i]==K )
            found = TRUE;
        else
        {
            q = p;
            p = p->ptr[i];
        }
    }
    if (found)
        return (p,i,1);
    else
        return (q,i,0);
} //SearchBTree
```

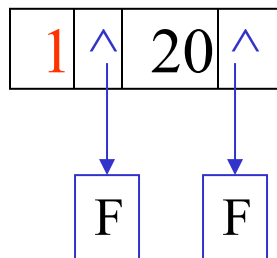
2) 插入

在B-Tree中插入新关键字时，每次关键字插在最靠近叶子的非终端结点中，若该结点插入后关键字个数不超过 $m-1$ ，则插入完毕；

否则需将该结点从中间**分裂**为两个结点，将居中关键字 ($\lceil m/2 \rceil$) 送往上一层（父母结点）中；再对上一层结点做同样处理，直至每个结点的关键字数均不超过 $m-1$ 为止。

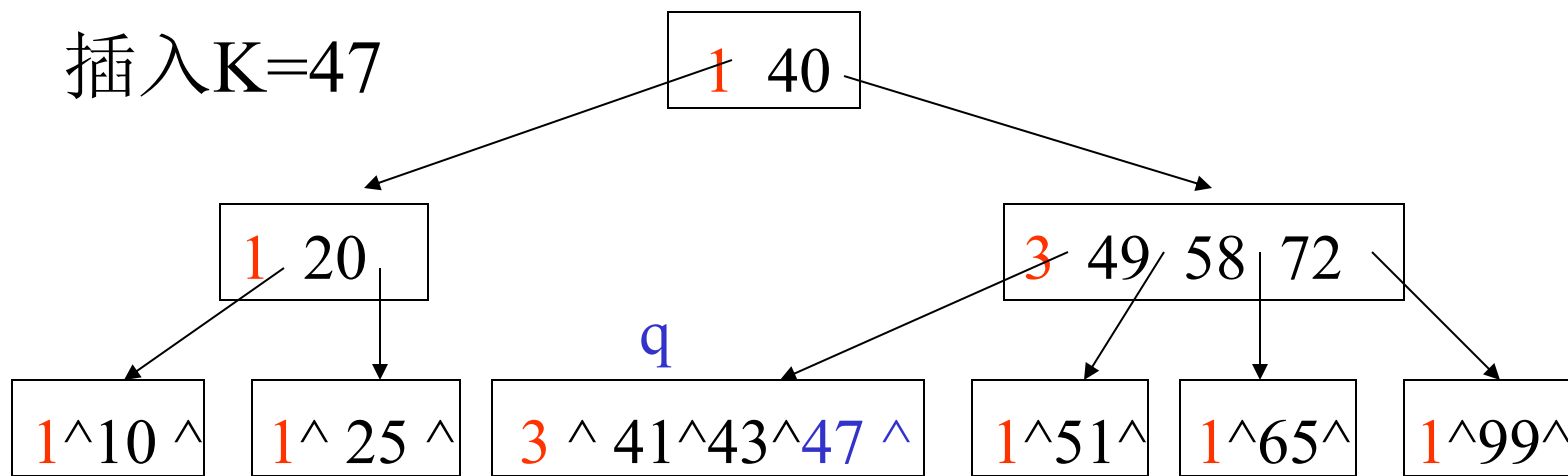
第一个关键字的插入也是从空树开始。

如：空树插入插入20

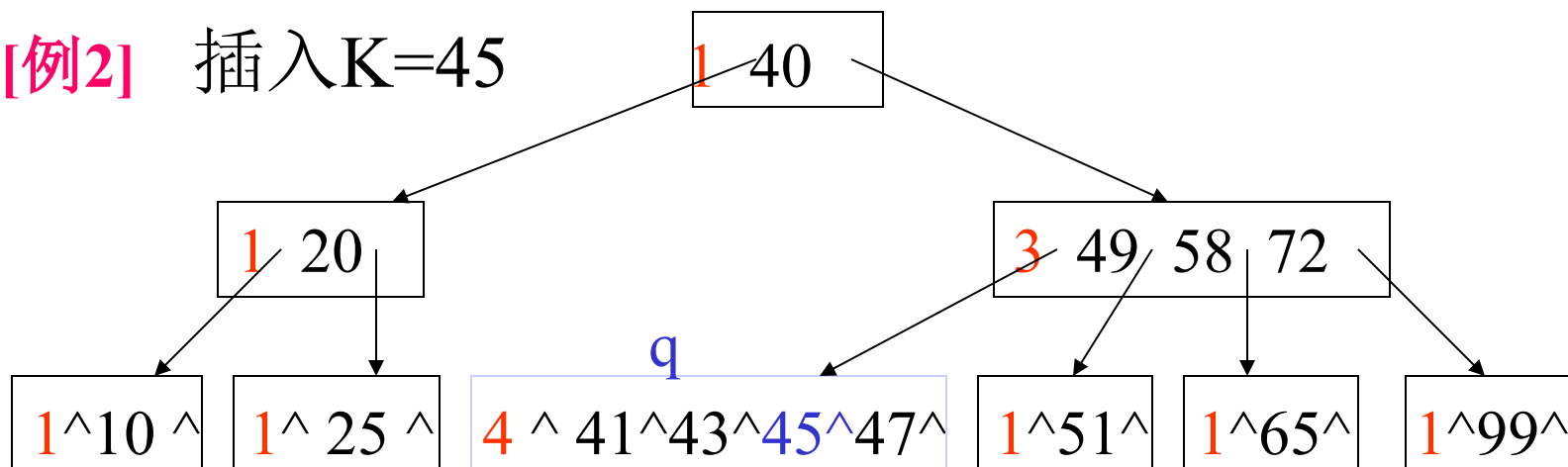


[例1] m=4的B-树

插入K=47

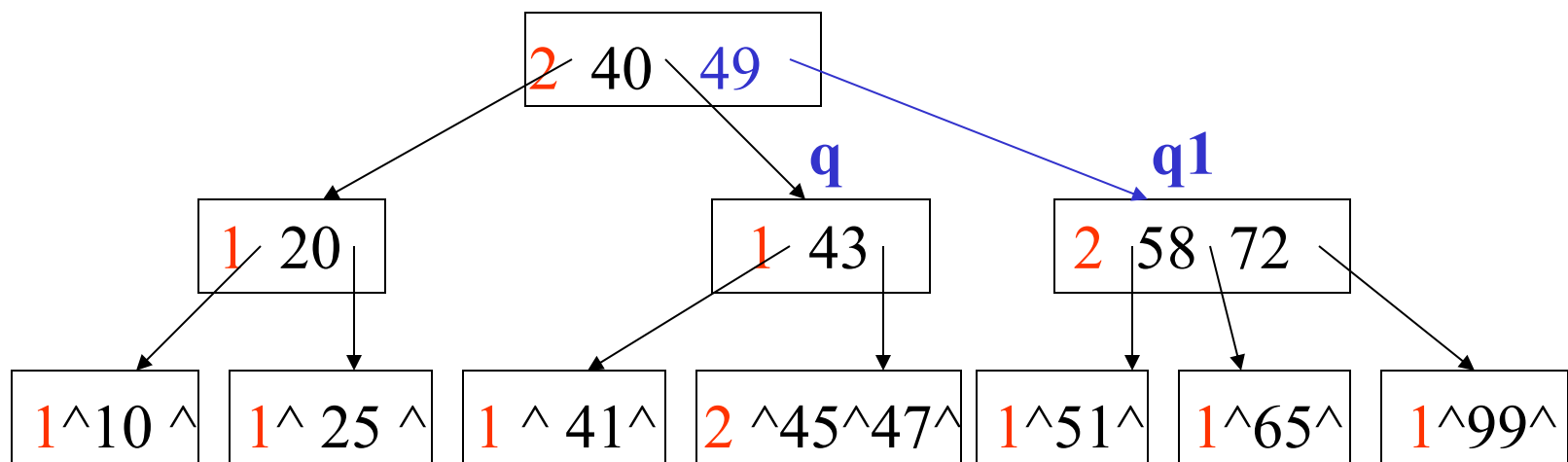
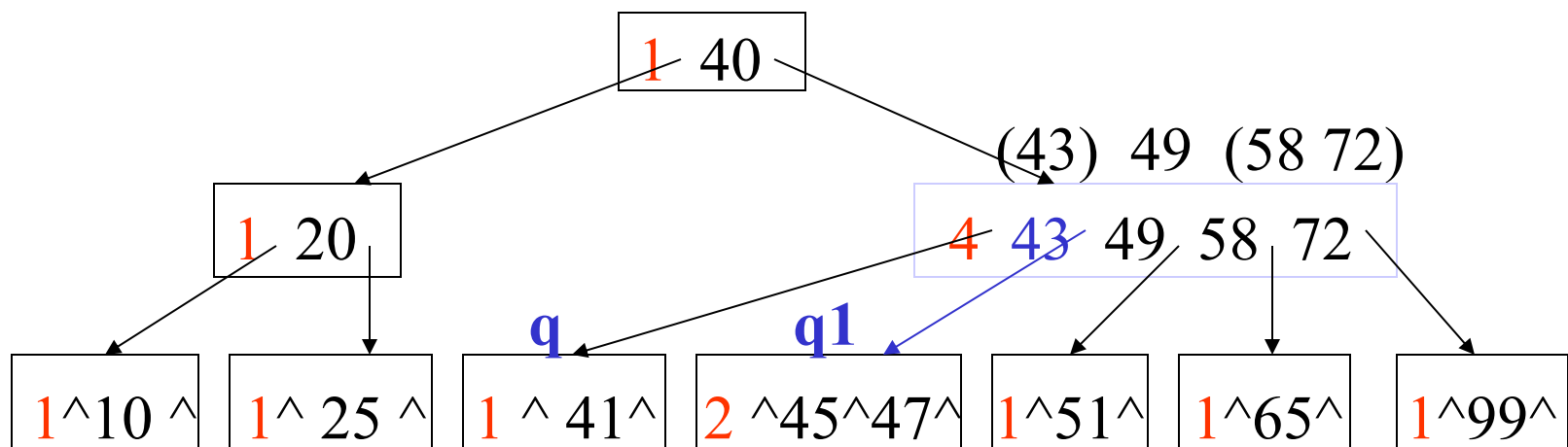


[例2] 插入K=45



(41) 43 (45 47)

数据结┉┉┉第九章 查找 $\lceil m/2 \rceil$



若由底向上至根结点仍需分裂时，需要构造新的根结点，B-树会增加一层。

2) 插入

[算法思想] 设插入的关键字为K

(a)在B-树中查找K，若查找到，则返回

否则查找失败，查找操作定位于某个最低层的非叶结点 (q,i) ，在相应的位置插入偶对 $[K, ^]$

(b)keynum加1，若当前结点未溢出($\text{keynum} < m$)，则写盘
 $\text{DiskWrite}(*q)$ ，返回

(c)否则，以 $\text{key}[\lceil m/2 \rceil]$ 为划分点，分裂当前结点为两个结点：
 $(\lceil m/2 \rceil - 1, \text{ptr}_0, \text{key}_1, \dots, \text{key}_{\lceil m/2 \rceil - 1}, \text{ptr}_{\lceil m/2 \rceil - 1})$ —— q
 $(m - \lceil m/2 \rceil, \text{ptr}_{\lceil m/2 \rceil}, \text{key}_{\lceil m/2 \rceil + 1}, \dots, \text{key}_m, \text{ptr}_m)$ ——新结点 $q1$
写盘 $\text{DiskWrite}(*q)$ ， $\text{DiskWrite}(*q1)$ ，

(d)读当前结点的父结点到内存 $\text{DiskRead}(q \rightarrow \text{parent})$ ，
将偶对 $(\text{key}[\lceil m/2 \rceil], q1)$ 插入该父结点，
以该父结点为当前结点 q ，转(b)

3) 生成

[算法思想]

从空树起，逐个插入关键字得到。

4) 删除

[算法思想]

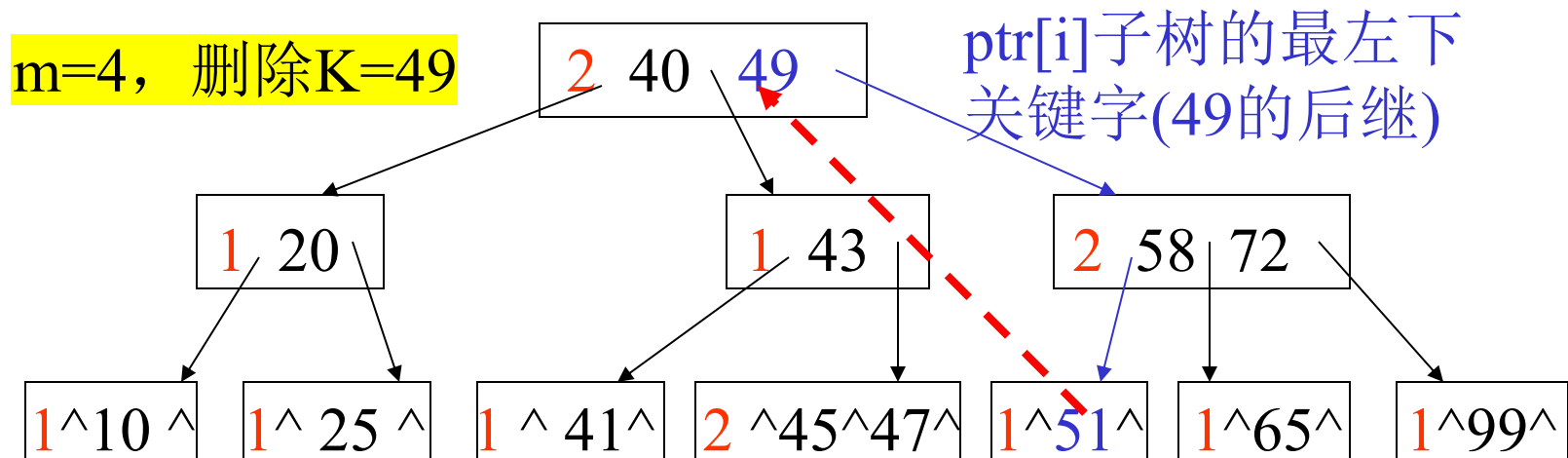
设删除的关键字为K

(a)在B-树中查找K，若未查找到，则返回

否则 查找成功，查找操作定位于某个结点q，

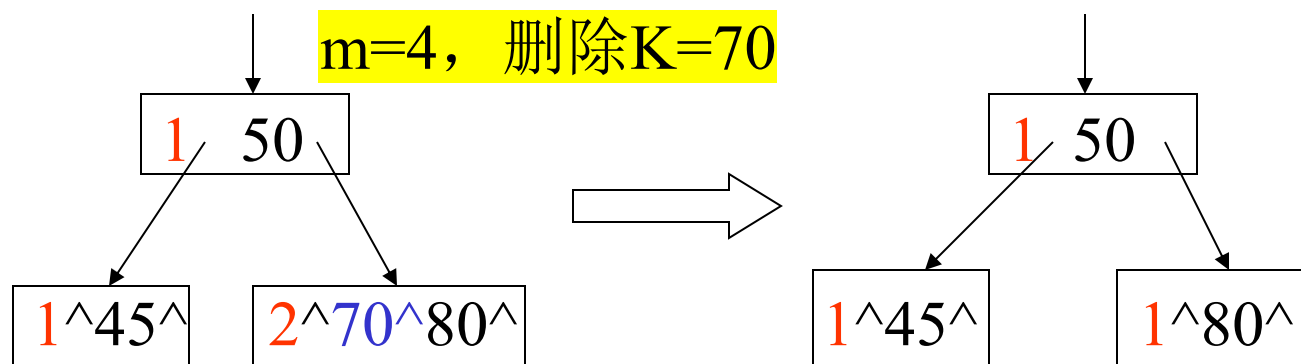
且 $q \rightarrow \text{key}[i]$ 为K

(b)若q结点不是最下层的非叶结点，则查找 $q \rightarrow \text{ptr}[i]$ 所指子树的最小关键字x，用x替换key[i]，使问题转换为删除最下层的非叶结点上的关键字

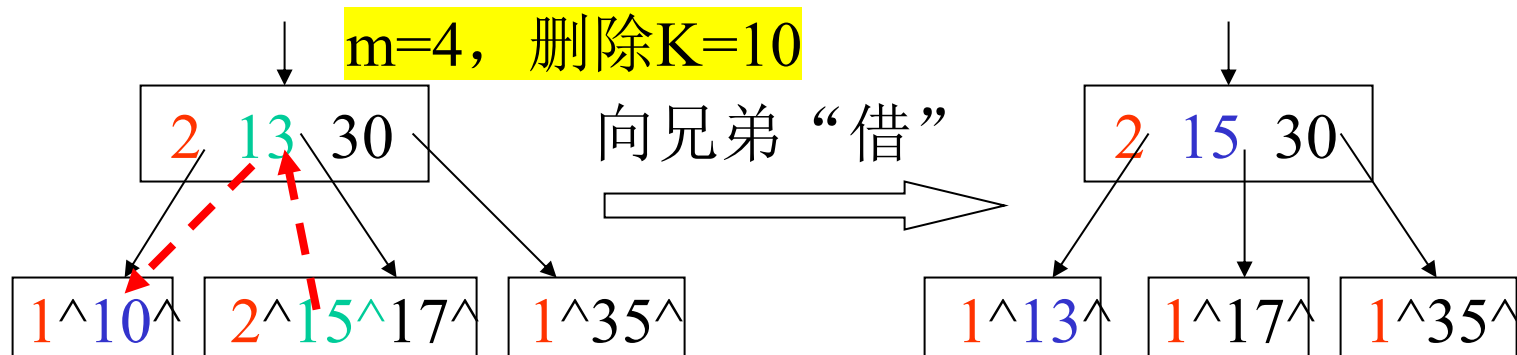


(c) 删除h层（最下层）上的关键字，分三种情况处理：

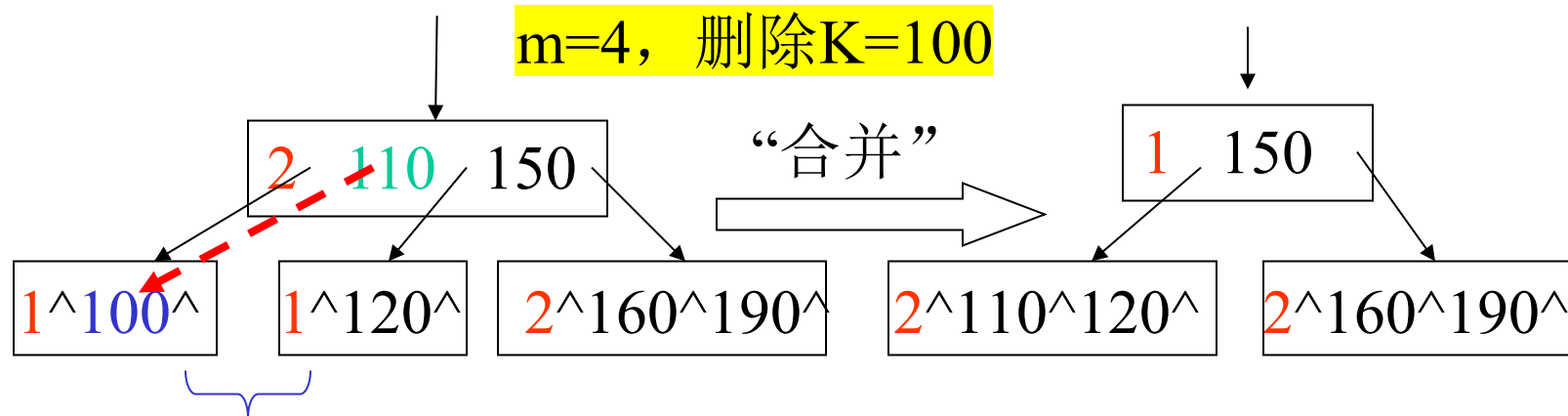
- 1、其所属结点的关键字个数 $> \lceil m/2 \rceil - 1$ ：简单删除



- 2、其所属结点的关键字个数= $\lceil m/2 \rceil - 1$,
其左/右兄弟结点关键字个数 $> \lceil m/2 \rceil - 1$:

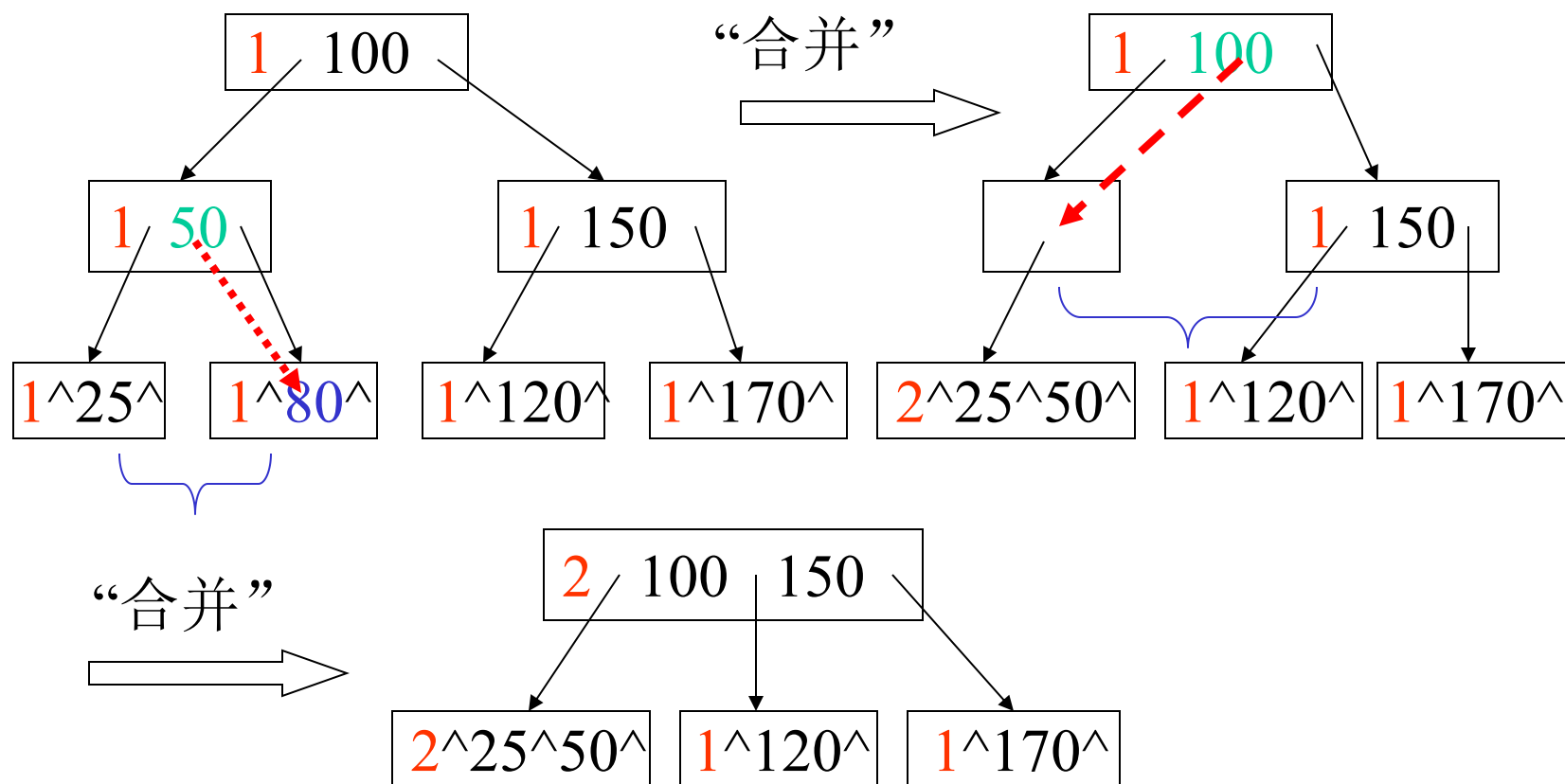


- 3、其所属结点的关键字个数= $\lceil m/2 \rceil - 1$,
其左/右兄弟结点关键字个数= $\lceil m/2 \rceil - 1$



“合并”时出现的较为复杂情况的处理：向上递归

$m=4$, 删除 $K=80$



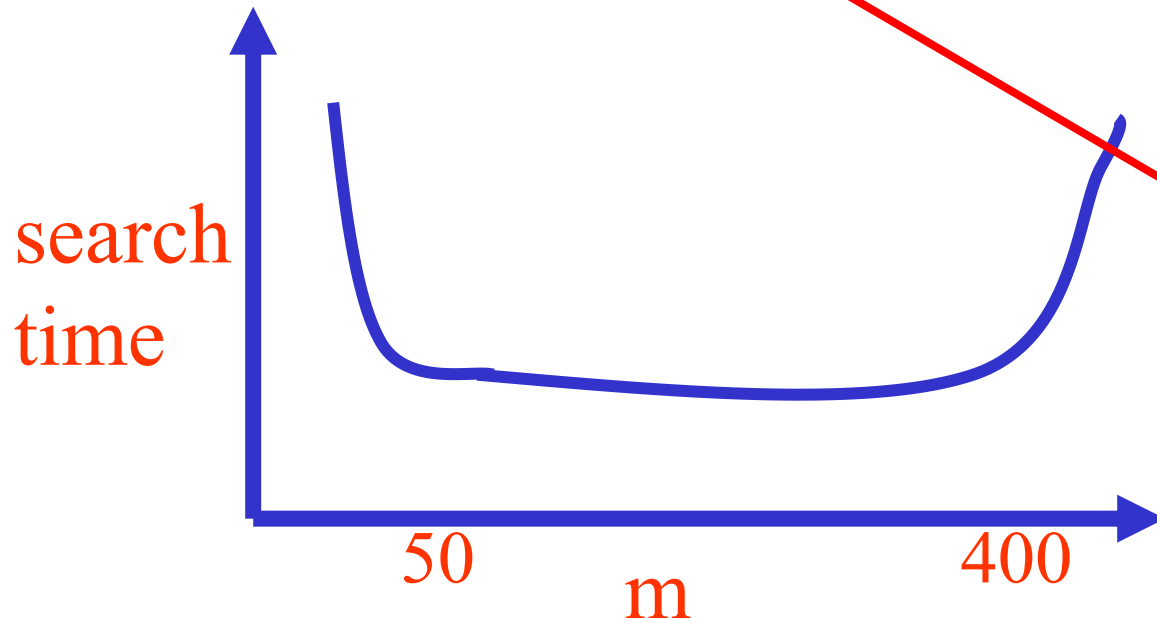
5. B-树的高度及性能分析

- **定理** 对于任一棵具有 $n(n>0)$ 个关键字的 m ($m>2$)阶B-树，其树高 h 至多为 $\log_t((n+1)/2)+1$ 。其中 $t=\lceil m/2 \rceil$
- 决定B-树上操作的时间开销的两个主要因素
 - 外存的访问次数 $O(h)$ —— $O(\log_t n)$
 - CPU的在结点上的比较定位时间 $O(hm)$ 或 $O(h \cdot \log_2 m)$
前者起主导作用
- 作为外存上的B-树， m 越大，则B-树的高度越小，时间性能越好；同时要考虑一个结点的磁盘读取时间，在每一结点作为一个读取单位（页/块）时为好
- 仅在内存中使用的B-树必须取较小的 m ，通常取最小值 $m=3$

外存情况下m应该选择多少？

从最坏情况下的搜索时间算一下

- (从磁盘读取一个结点时间+结点填充到内存时间+结点内查找key的时间) * 树高
 - 即： $(a*k + b*m + c * \log_2 m) * h$ ，a为一个扇区的磁盘读入时间，k为一个结点存储的扇区数量，b为一个key写入内存的单位时间，c为内存查找key的单位时间，h为树高
 - 对某个硬件配置的设备而言，a，b，c是常数



为什么m比较小时
时间长？
为什么m较大时，
时间也长？

9.3.4 B+树

B+树是一种**B-树的变形**树。广泛应用于文件系统中，特别是数据库的VSAM文件系统中（IBM）。

1. m阶B+树与 B-树的不同之处

1)有 n 棵子树的结点中有 n 个关键字；

2)非叶结点可以看成是索引部分 **索引集**

A_i : 第 i 个子结点的指针

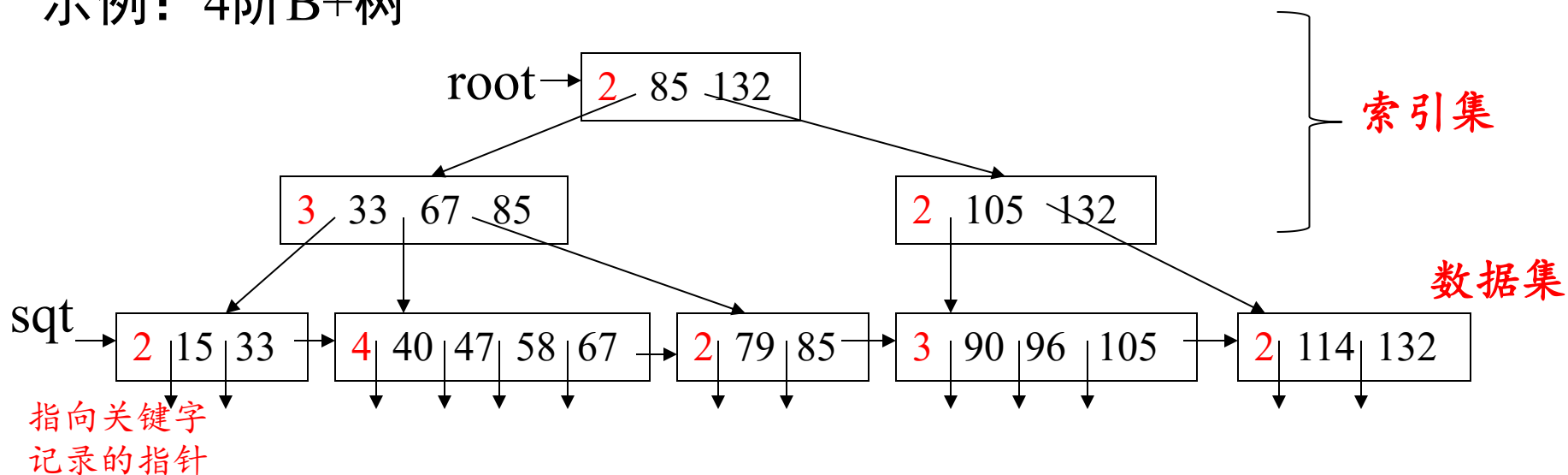
K_i : 第 i 个子结点的最大（或最小）关键字

3)所有叶子结点中包含了全部关键字的信息及指向这些关键字记录的指针，且叶子结点以关键字大小自小至大顺序链接； **数据集**

[结点结构]

- 非叶结点 ($A_1, K_1, \dots, A_i, K_i, \dots, A_n, K_n$) 索引集
 - A_i : 第i个子结点的指针
 - K_i : 第i个子结点的最大（或最小）关键字
- 叶结点
 - 全部关键字及指向关键字记录的指针 数据集

示例：4阶B+树



2. B+树上的基本运算

1)查找

- 方式1：从根结点开始，利用索引集结构，向下查找直到叶子结点
- 方式2：从最小关键字开始，沿叶结点数据集的链结构顺序查找

2)插入

仅在叶子结点上进行，关键字个数大于 m 则分裂

3)删除

也仅在叶子结点上进行，关键字个数小于 $\lceil m/2 \rceil$ 时，需进行合并



9.3.5 键树(数字查找树/字符树)

1. 概念

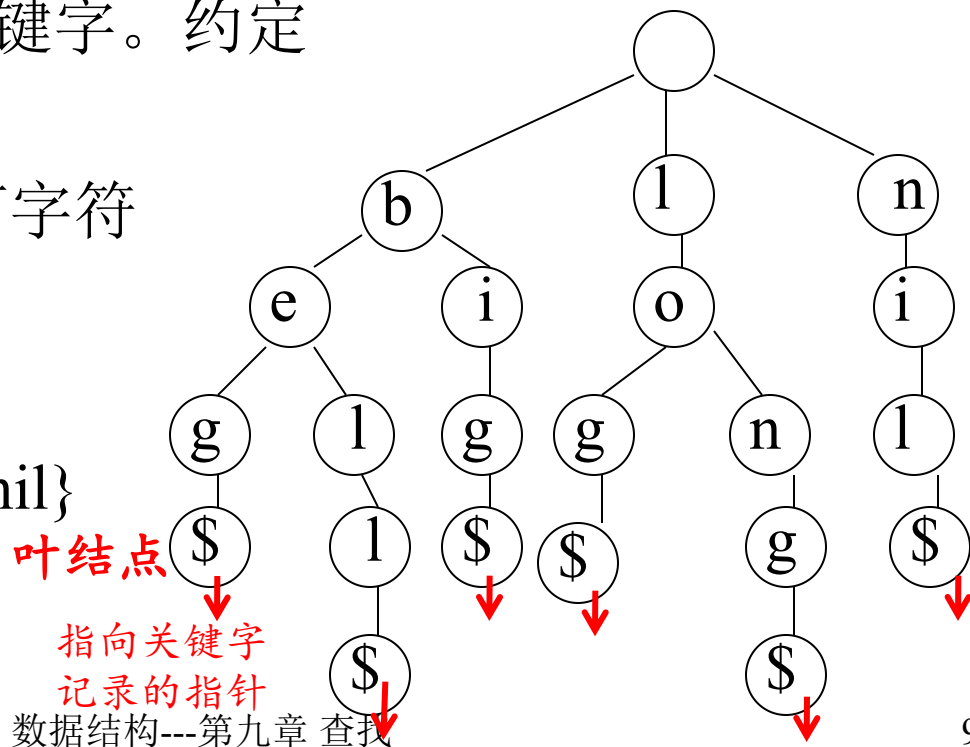
将关键字**分解**为字符的多叉树，多叉树中的每个结点只代表关键字中的一个字符，叶结点用于表示字符串的结束符(如 '\$')，并含有指向该关键字记录的指针。从根到叶子的路径上所有的结点对应的字符连接起来就代表一个关键字。约定

- 键树是有序树
- 结束符小于任何字符

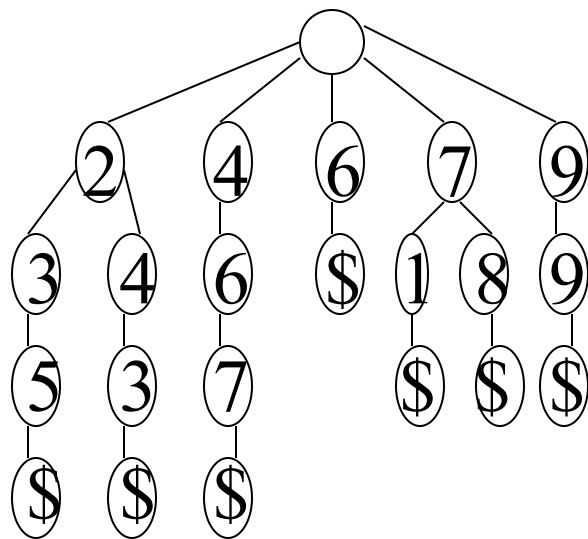
[例1]

关键字集合

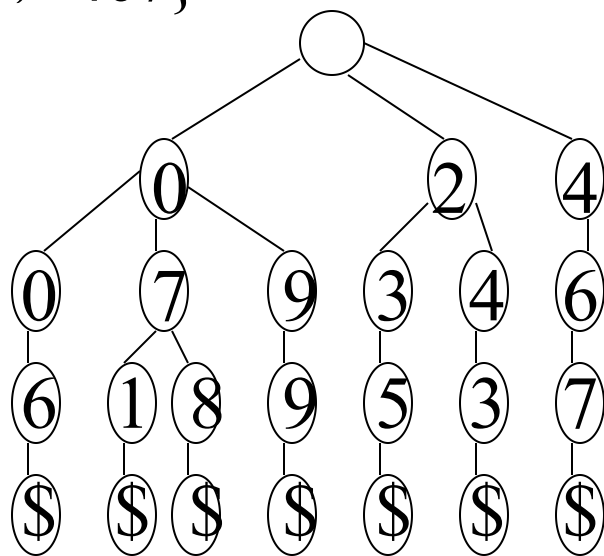
{beg,bell,big,log,long,nil}



[例2] {6, 71, 78, 99, 235, 243, 467}



按纯字符串考虑

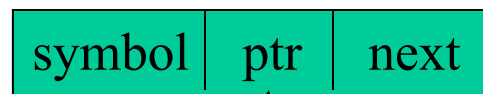


按整形数考虑

2. 键树的存储结构及其操作

1) 双链树

——以树的孩子兄弟链表表示



字符

兄弟指针

{ 结点为leaf型: 指向记录的指针
 { 结点为nonleaf型: 指向孩子指针

[查找]

设给定值为K[1..m]

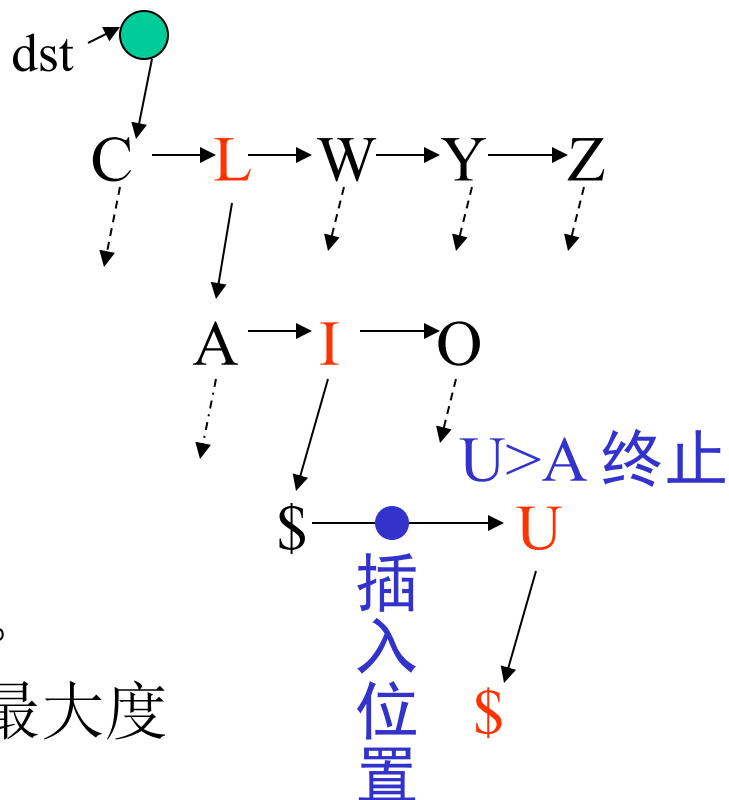
- 成功查找的情况 K='LIU\$'

从p=dst->ptr出发

沿next域顺序查找，直到某个
结点的symbol值等于K[1];
再沿该结点的ptr域进入下一层
结点，查找K[2];

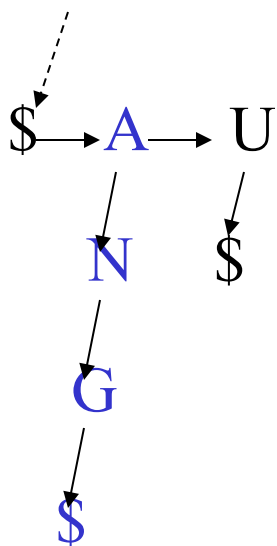
直至K[M]比较在叶结点处终止。

$ASL = h(1+d)/2$ d—键树中结点的最大度

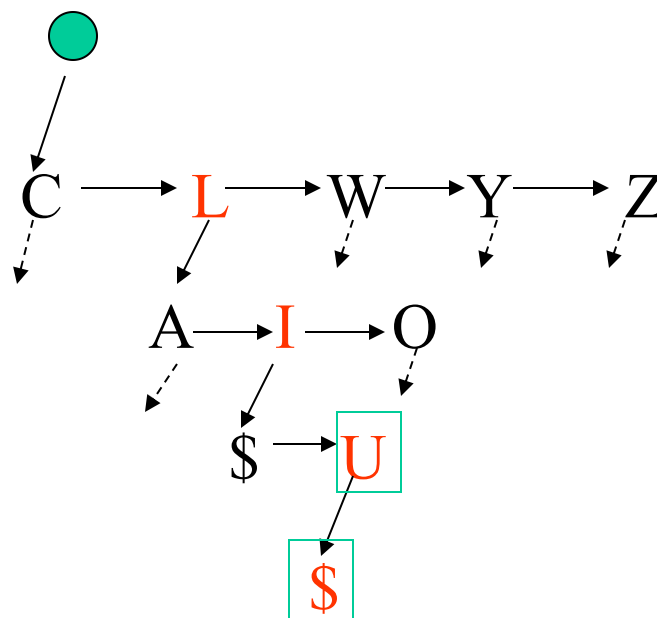


- 不成功查找的情况 K='LIANG'

[插入] 查找不成功时，
插入



[删除] 确认查找成功后，
删除



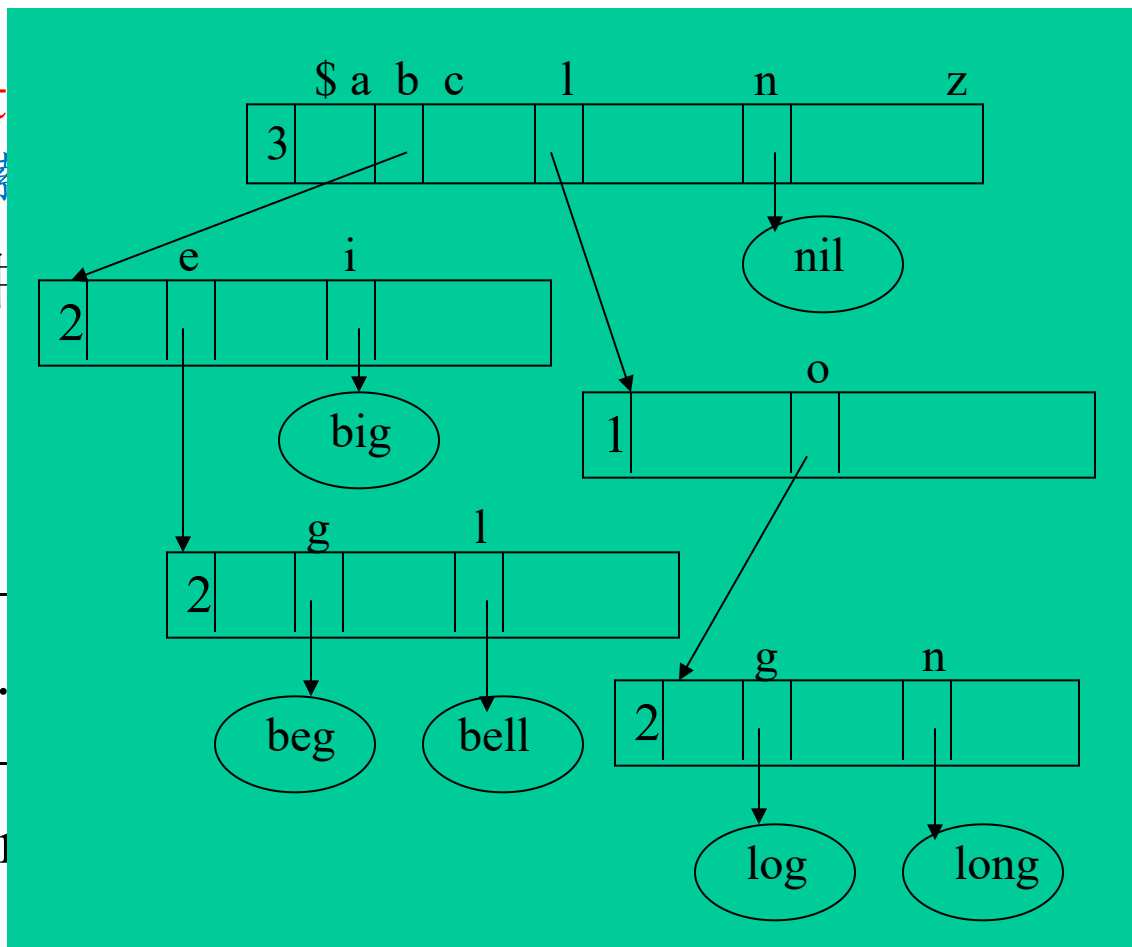
注意：结点只有一个孩子或
是叶结点时才可删除。

2) Trie树(来源于ret)

主要应用：信息检索

若从键树中
每个结点中都只
点压缩成一个“
向记录的指针。

- 分支结点：num-
ptr[0.
- 叶子结点：key-
recptr



[例]

[降低Trie树深度的一种途径]

选择一种合适的分割方式。如前后交叉分割/取样。



[降低Trie树深度的一种途径]

选择一种合适的分割方式。如前后交叉分割/取样。

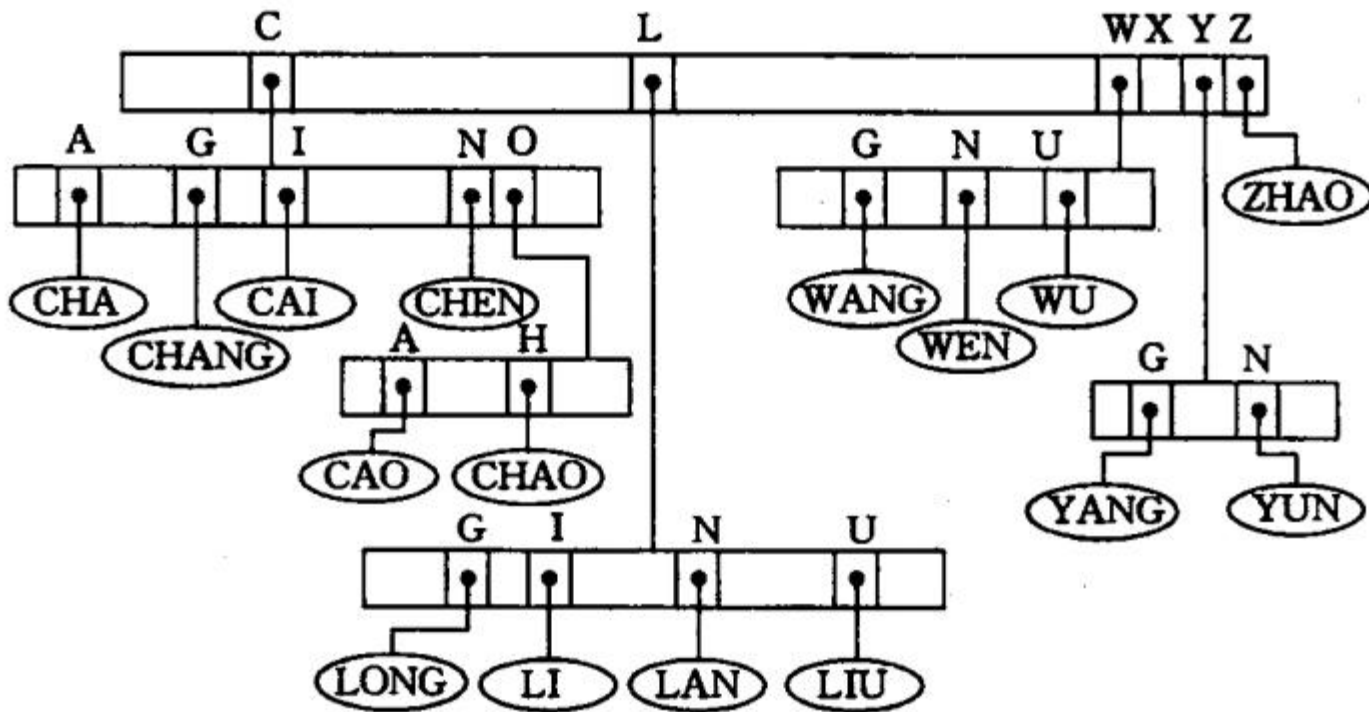


图 9.22 对(9-24)关键字集采用另一种分割法得到的 Trie 树(深度=4)

9.4 哈希表

9.4.1 概述

9.4.2 哈希函数的基本构造方法

9.4.3 处理冲突的常用方法

9.4.4 哈希表的建立、查找及其ASL分析



要建设一个某市的小学生信息处理系统，首先要将学生信息存入，并可以快速查询出来，假设：

- 1、学生人数：每个年级10万人左右，共6个年级；
- 2、可以一次性将数据装载到内存，在内存中查找；
- 3、给每个学生编制了学号：

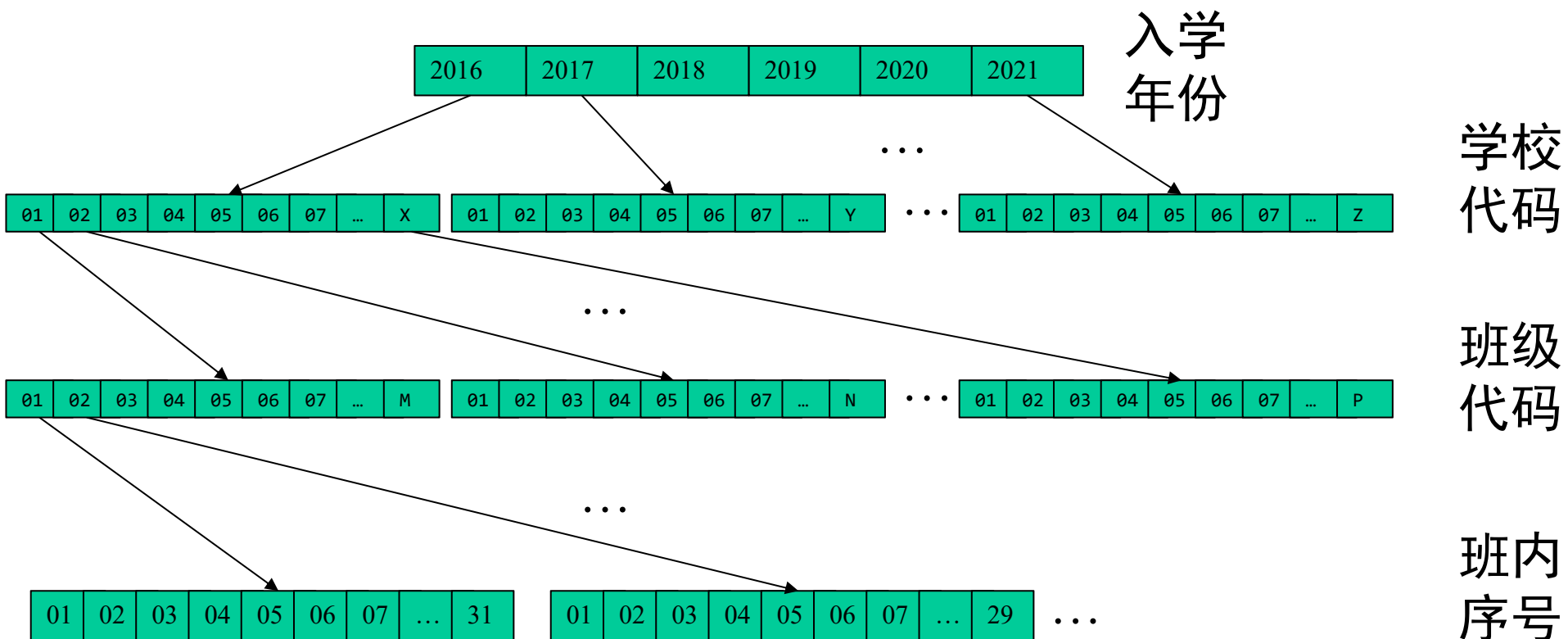
入学年份+学校代码+班级代码+班内序号

问：全部数据用什么方式在内存中组织起来后，查询某个学号的对应的信息时性能好？

- 1、有序的线性表(数组)，二分查找？
- 2、索引顺序表？
- 3、B-树？

4、将多个技术组合起来：

- 1) 按班进行分块，班内数据排序
- 2) 建立分级索引：先查入学年份、再查学校代码、再查班级代码确定分块，再在块里进行二分查找。

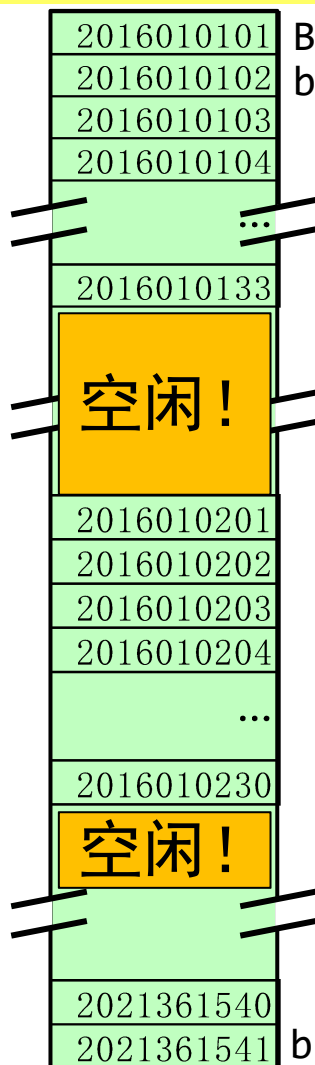


- 1、入学年份，需要几个线性表？用什么方法查某年的数据元素及指向学校代码的指针？
- 2、学校代码，需要几个线性表？用什么方法查某个代码的数据元素及指向班级的指针？
- 3、学校内的班级，需要几个线性表？用什么方法查某个班级的数据元素及指向班内序号的指针？
- 4、在按班分块的分块数据中用什么方法查找？
- 5、按学号查找时的查找成功和查找失败的平均查找次数分别是？

假设内存无限大，无需考虑内存不够的问题；
将学生学号视为一个整数，将学生信息存入：

$[base + \text{学号} - \text{第一个学生的学号}] * S$ 为 地址的内存单元时

- 1) 如何查找？
- 2) 有什么缺点？



Base
 $base+1*S$

当内存不是无限大时
这些空闲区就是很大的
浪费，如何解决？

将学生学号进行如下运算：

$x = \text{学号} - 2016010101;$

$x1 = x / 1000000;$

$x2 = (x \% 1000000) / 10000;$

$x3 = (x \% 10000) / 100;$

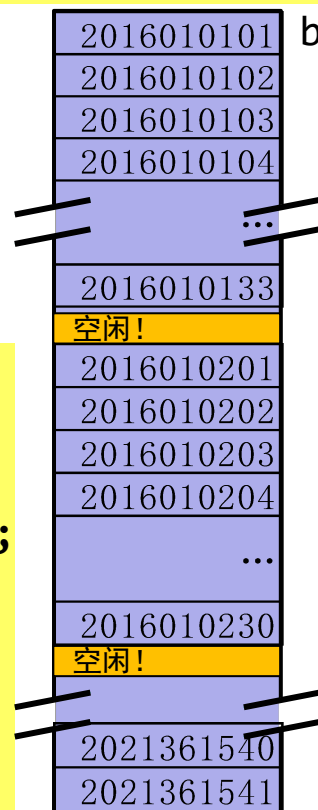
$x4 = x \% 100;$

$y = x1 * \text{学校数} * \text{全部校最大班级数} * \text{全部校最大班人数}$

$+ x2 * \text{全部校最大班级数} * \text{全部校最大班人数}$

$+ x3 * \text{全部校最大班人数}$
 $+ x4;$

$base + (2021361541 - 2016010101) * S$



base

占用“6*学校数*全部校最大班级数*全部校最大班人数”个单元

因：

学校数 < 99

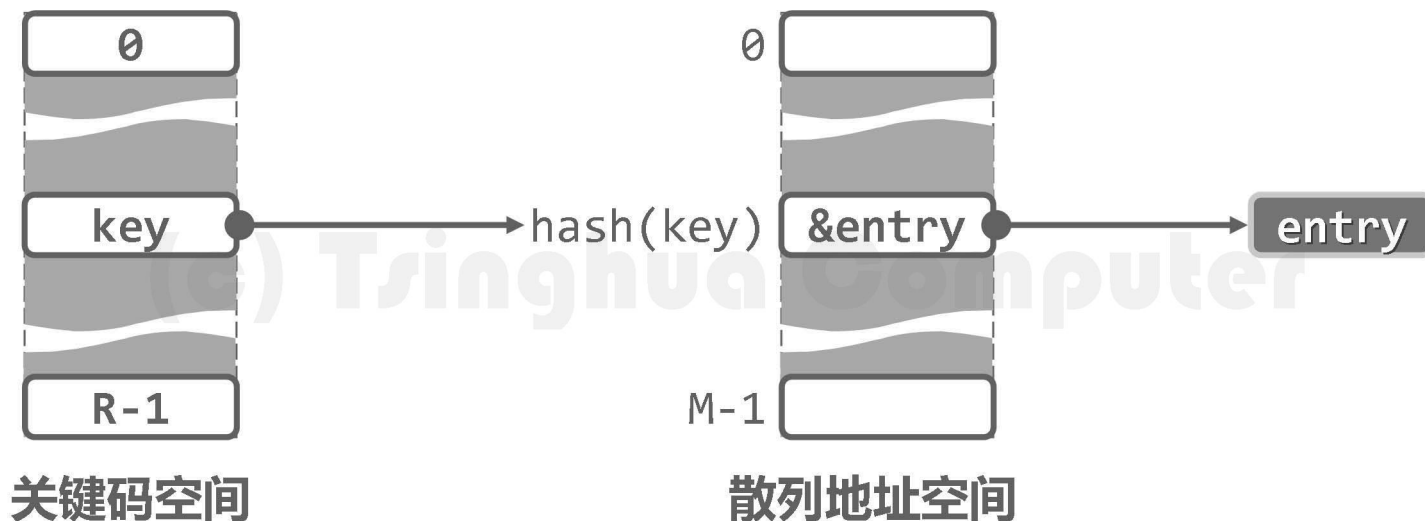
全部校最大班级数 < 99

全部校最大班人数 < 99

空闲空间大大减少

$base + 6 * \text{学校数} * \text{全部校最大班级数} * \text{全部校最大班人数} - 1$

空闲数大大减少，查找依然是O(1)。可以进一步改进运算方法：加快运算速度的同时再减少一些空间占用。



通过 $\text{hash}(\text{key})$ 函数，将 key 在“关键词空间”中的位置变换为“散列地址空间”中的位置，将散列地址空间一一映射到内存地址空间后，就可以将 key 保存在对应的内存地址单元了：

保存时，通过 key 用 $\text{hash}(\text{key})$ 算出其地址存进去；查找时，通过 key 用 $\text{hash}(\text{key})$ 算出地址后将内存地址中的内容和 key 进行比较来确定 key 是否存在。
理想状态下，保存和查找都是 $O(1)$ 时间复杂度。

9.4.1 概述

顺序查找和各类查找表（树表）的结构特点：

- 记录在表中的位置和它的关键字之间不存在一个确定的关系
- 查找的过程为给定值依次和集合中各个关键字进行比较，查找的效率取决于和给定值进行比较的关键字个数。
- 不同的表示方法，其差别仅在于：关键字和给定值进行比较的顺序不同。

换一个思路：让存储位置与关键字之间存在对应关系

$Loc(i) = H(key_i)$ ，其中H为哈希函数。

这样，能由关键字值直接算出存储位置，就更方便了

查找速度极快，理想状态下 $O(1)$ 。

查找效率与元素个数n无关

[哈希表的基本思想]

在记录的存储地址和它的关键字之间建立一个确定的对应关系；这样，**理想状态（无冲突）** 不经过比较，一次存取就能得到所查元素。

[术语]

哈希表(Hash table):

一个有限的连续的地址空间，用以容纳按哈希地址存储的记录。

哈希, Hash, 又称为散列、杂凑。

哈希函数: $Loc(r_i) = H(key_i)$

记录的存储位置与它的关键字之间存在的一种对应关系。

冲突(collision):

对于 $key_i \neq key_j$, 且 $i \neq j$:

- 1) $H(key_i) = H(key_j)$ 的现象;
- 2) 虽然 $H(key_i) \neq H(key_j)$, 但 key_i 在哈希表中占用了 key_j 的存储位置。

同义词(synonym) :

$key_i \neq key_j$ 但 $H(Key_i)$ 和 $H(key_j)$ 相同, 称为对此哈希函数的一组同义词。

有关术语

哈希表(杂凑表): 按上述思想构造的表

数组下标
(地址)

... 9 ... 11 ... 14 ... 23 24 25 ... 39 ...

内容

		9		11		14		23		25		39	
--	--	---	--	----	--	----	--	----	--	----	--	----	--

冲突: 不同的关键字映射到同一个哈希地址

$\text{key1} \neq \text{key2}$, 但 $H(\text{key1}) = H(\text{key2})$

例如: H函数为: $\text{key} \bmod 100$

$H(9) = 9;$

$H(109) = 9;$

同义词: 具有相同函数值的两个关键字

问题:

冲突时, 两个关键字(关键码、关键词)一定是同义词吗?

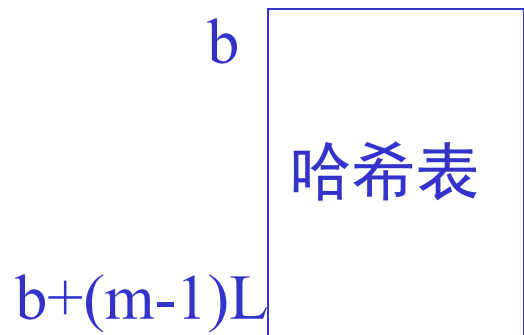
哈希(散列)地址 根据设定的哈希函数 $H(\text{key})$ 和处理冲突的方法确定的记录的存储位置。

装填因子 表中填入的记录数 n 和哈希表表长 m 之比。

$$\alpha = n/m$$

[设计哈希表的过程]

- 1)明确哈希表的地址空间范围：即确定哈希函数的值域。
- 2)选择合理的哈希函数：该函数要保证所有可能的记录的哈希地址均在指定的值域内，并使冲突的可能性尽量小。
- 3)设定处理冲突的方法。



例

数据元素序列(14, 23, 39, 9, 25, 11), 若规定每个元素k的存储地址 $Loc(k) = k \text{ MOD } 100$, 请画出存储结构图。

数组A

下标 (地址) ... 9 ... 11 ... 14 ... 23 24 25 ... 39 ...

内容

		9		11		14		23		25		39	
--	--	---	--	----	--	----	--	----	--	----	--	----	--

如何查找

- 1、根据哈希函数计算被查找key的 $Loc(key)$
- 2、比较 $A[Loc(key)]$ 的值是否为key值:
 - 1) 相同, 则**查找成功**。
 - 2) 不同, 则**处理冲突后继续查找**。
 - 3) $A[Loc(key)]$ 值为空, **查找失败**。

查找key=9, $H(9)=9$, 内容为9, 成功, 返回地址9;

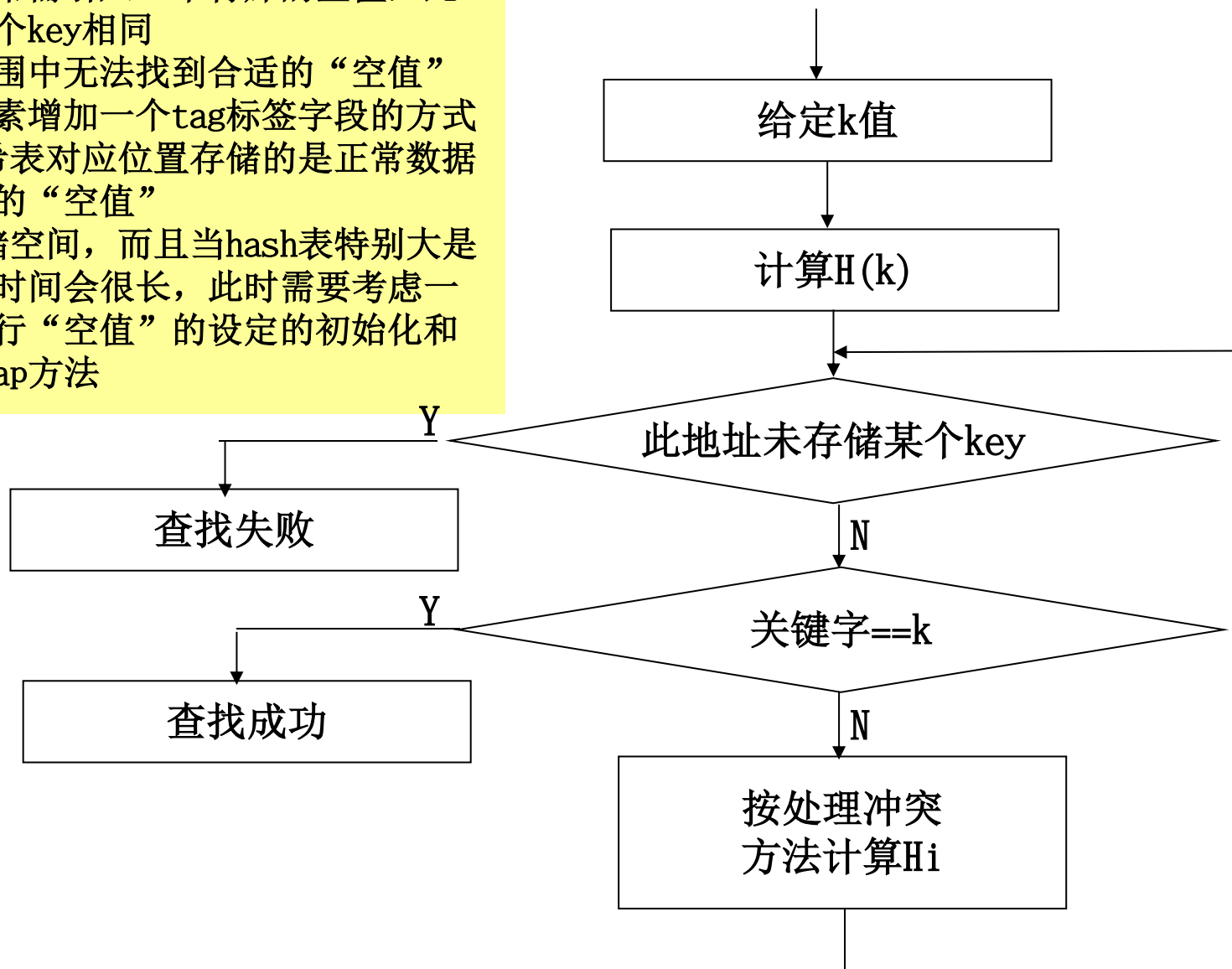
查找key=311, $H(311)=11$, 内容为11, 冲突, 需处理冲突后继续查找;

查找key=24, $H(24)=24$, 内容为空, 查找失败, 返回空地址。

哈希表的查找

哈希计算得到地址后，取地址内的值与关键字比较；冲突后处理冲突得到新的地址，重复上述过程。

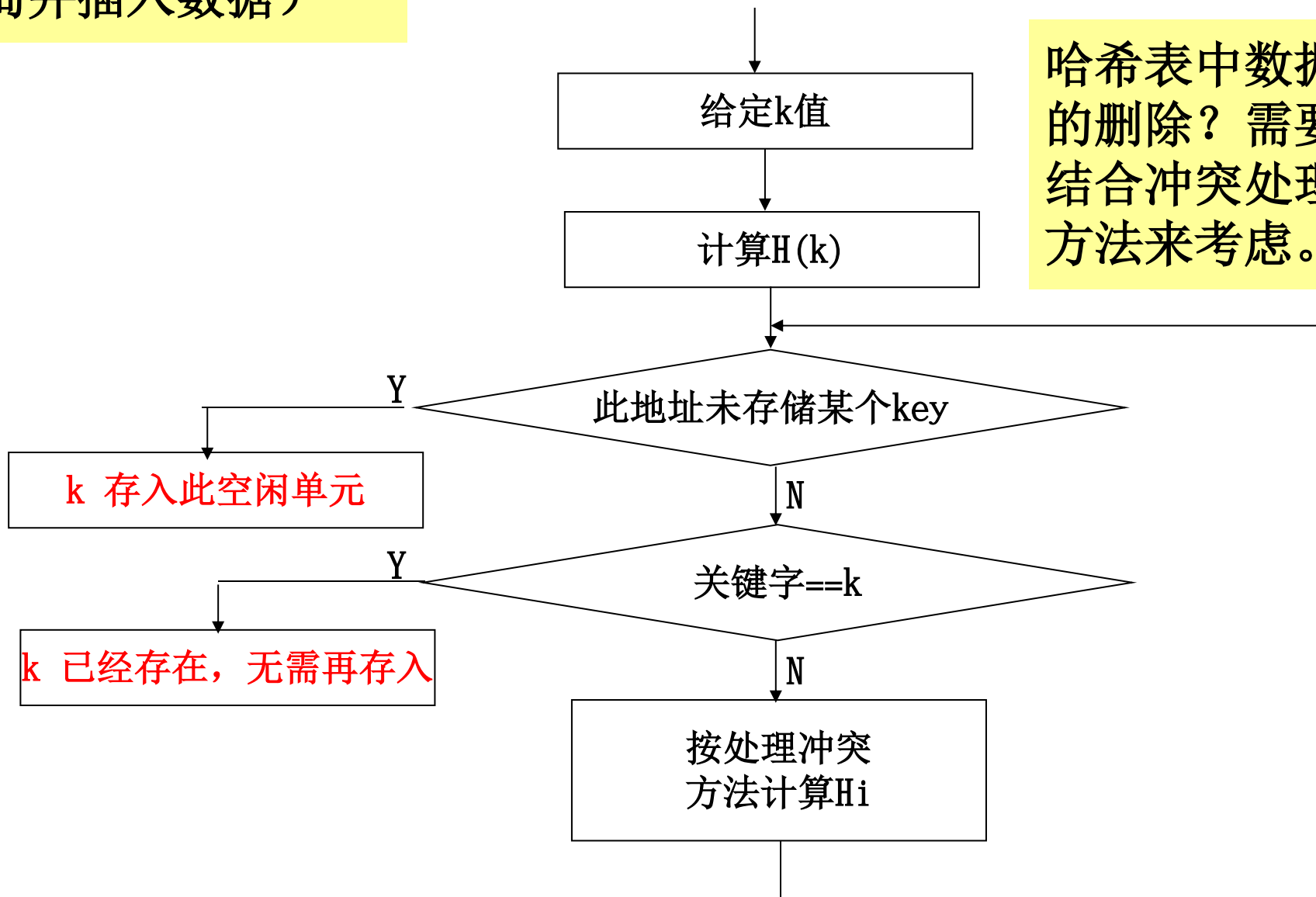
哈希表使用时，通常需引入一个特殊的空值，此空值不能和任何一个key相同
如数据元素的值范围中无法找到合适的“空值”，则采用给数据元素增加一个tag标签字段的方式来在tag中标记哈希表对应位置存储的是正常数据元素值，还是特殊的“空值”
tag字段会占用存储空间，而且当hash表特别大是，初始化hash表的时间会很长，此时需要考虑一个方法特殊对其进行“空值”的设定的初始化和空值判定，如bitmap方法



哈希表的建立（查询并插入数据）

动态表，怎样查找，怎样建立

哈希表中数据的删除？需要结合冲突处理方法来考虑。



哈希函数的构造方法

根据元素集合的特性构造

1、算法简单，运算量小；
值域地址空间尽量小；
2、计算结果尽量均匀；
冲突尽量少。



1. 直接定址法
2. 数字分析法
3. 平方取中法
4. 折叠法
5. 除留余数法
6. 乘余取整法

[直接定址法] $H(key) = a * key + b$, a 、 b 为常数

特点：计算简单，冲突最少。

此法仅适合于：地址集合的大小 = 关键字集合的大小

[数字分析法] 取关键字的若干数位作为哈希地址。

假设关键字集合中的每个关键字都是由 s 位数字组成 ($u_1, u_2, \dots, u_s$)，分析关键字集中的全体，并从中提取分布均匀的若干位或它们的组合作为地址。

此方法仅适合于：能预先估计出全体关键字的每一位上各种数字出现的频度。

[平方取中法] 取关键字平方后的中间几位作为哈希地址

求“关键字的平方值” 的目的是“扩大差别” ，同时平方值的中间各位又能受到整个关键字中各位的影响。

此方法适合于：关键字中的每一位都有某些数字重复出现频度很高的现象。

[折叠法]将关键字分割成位数相同的几部分(最后一部分的位数可以不同)，然后取这几个部分的**叠加和**(舍去进位)作为哈希地址。

此方法适合于：关键字的数字位数特别多。

例：key = 110108380428895

895		895	
428	移位	824	间界
380	叠加	380	叠加
108		801	
+)110		+)110	
1921		3010	

[除留余数法] $H(key) = key \text{ MOD } p \quad (p \leq m)$

m: 哈希表的表长;

p: 最好为素数, 更有利于“散列”

为什么要对 p 加限制?

给定一组关键字为: 12, 39, 18, 24, 33, 21,

若取 $p=9$, 则对应的哈希函数值将为:

3, 3, 0, 6, 6, 3

可见, 若 p 中含质因子 3, 则所有含质因子 3 的关键字均映射到“3 的倍数”的地址上, 从而增加了“冲突”的可能。

[(伪)随机数法] $H(key) = random(key)$

选取一个用关键字作为种子的伪随机数发生器生成散列地址
此方法适合于: 关键字长度不等时的情况。

为产生冲突的地址寻找下一个
哈希地址

9.4.3 处理冲突的常用方法

1. 开放定址法 (空缺编址法)

$$H_i = (H(\text{key}) + d_i) \text{ MOD } m$$

$$i=1, 2, \dots, k \quad (k \leq m-1)$$

m : 哈希表的表长; d_i : 增量序列

0 $H(\text{key})$ $m-1$



1) 线性探测再散列 $d_i = 1, 2, \dots, m-1$

缺陷: 有聚集(堆积)现象—非同义词地址冲突。

2) 二次探测再散列 $d_i = 1^2, -1^2, 2^2, -2^2, 3^2, \dots, \pm k^2 \quad (k \leq m/2)$

缺陷: 不易探查整个散列空间。

3) 伪随机探测再散列 $d_i =$ 伪随机数序列

开放定址法不能直接执行删除操作。Why?

在使用了tag来区分“空值”还是“正常值”时, 可对tag增加一个意义: 表示占用过但被删除了, 这样就可以使用**懒惰删除 “lazy deletion”**的方式标记数据元素已经被删除掉, 而又无需调整HASH表的内容。为了节约内存, tag通常使用某种bit级别的数据结构方法(如bitmap方法)来减少内存的使用。

例如：关键字集合 { 19, 01, 23, 14, 55, 68, 11, 82, 36 }

设定哈希函数 $H(key) = key \text{ MOD } 11$ (表长=11), 计算结果为:

key	19	1	23	14	55	68	11	82	36
MOD 11	8	1	1	3	0	2	0	5	3

若采用线性探测再散列处理冲突, $d_i = 1, 2, \dots, m-1$

0	1	2	3	4	5	6	7	8	9	10
55	01	23	14	68	11	82	36	19		
1	1	2	1	3	6	2	5	1		

若采用二次探测再散列处理冲突, $d_i = 1^2, -1^2, 2^2, -2^2, \dots, \pm k^2$

0	1	2	3	4	5	6	7	8	9	10
55	01	23	14	36	82	68		19		11
1	1	2	1	2	1	2		1		3

2. 再哈希法

$$H_i = RH_i(\text{key})$$

$$i=1,2, \dots, k$$

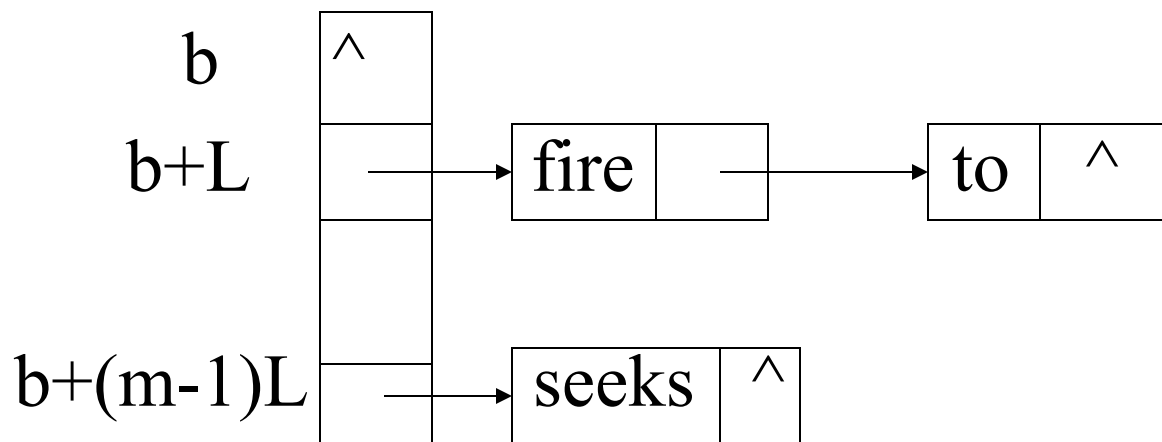
RH_i 为不同的散列函数

准备了一组哈希函数，在冲突发生时，依次使用这一组函数中的下一个来再次计算哈希值，直到冲突不发生为止。

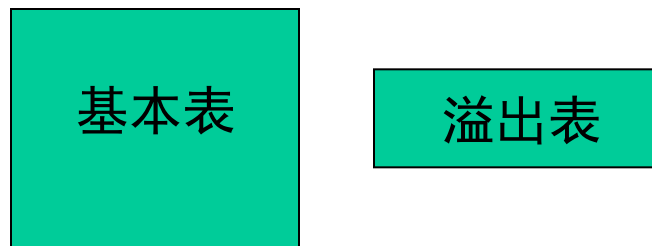
这种方法不易产生“聚集”，但增加了计算的时间。

3. 链地址法

为每个哈希地址建立一个单链表，存储所有具有同义词的记录。



4. 建立公共溢出区法



9.4.4 哈希表的建立、查找及其ASL分析

[例] 有一组关键字序列(38、59、125、168、216、719、455、20)，用函数 $H(\text{key}) = \text{key} \text{ MOD } 10$ 将其按顺序散列到哈希表 $HT(0:9)$ 中，分别用三种方法解决冲突：线性再探测法、链地址法、公共溢出区法，画出这三种方法建立的散列表，并分别求在等概率情况下查找成功和不成功的平均查找长度。

$$ASL = \sum_{i=1}^n p_i C_i$$

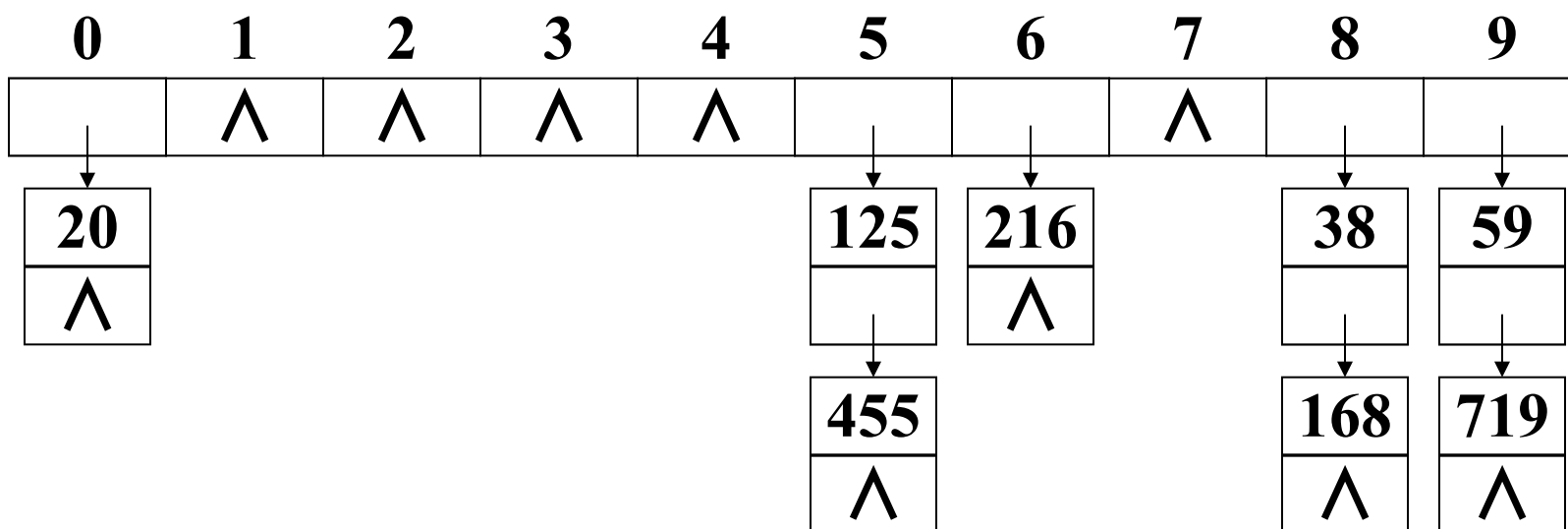
1) 线性再探测法

0	1	2	3	4	5	6	7	8	9
168	719	20			125	216	455	38	59
719	20				455	455		168	168

$$ASL_{\text{成功}} = \frac{1+1+1+3+1+3+3+3}{8} = \frac{16}{8} = 2$$

$$ASL_{\text{不成功}} = \frac{4+3+2+1+1+9+8+7+6+5}{10} = \frac{46}{10} = 4.6$$

2) 链地址法



$$ASL_{\text{成功}} = \frac{1+1+2+1+1+2+1+2}{8} = \frac{11}{8} = 1.375$$

$$ASL_{\text{不成功}} = \frac{1+2+1+2+2+10}{10} = \frac{18}{10} = 1.8$$

3) 公共溢出区法

0	1	2	3	4	5	6	7	8	9
20					125	216		38	59
					455				
							168 719		

公共溢出区:

0	1	2	3	4	5
168	719	455			
719	455				

$$ASL_{\text{成功}} = \frac{1+1+1+1+1+2+3+4}{8} = \frac{14}{8}$$

$$ASL_{\text{不成功}} = \frac{5+1+1+1+1+5+5+1+5+5}{10} = \frac{30}{10} = 3$$

[平均查找长度与哈希表的装填因子的关系]

$$\alpha = \frac{\text{表中填入的记录数}}{\text{哈希表的长度}}$$

在一般情况下， $\alpha \downarrow$ ，冲突的可能性 $\downarrow$ ，ASL $\downarrow$ ，但空间的浪费 $\uparrow$ 。

	成功ASL	失败ASL
线性探测再散列	$\frac{1}{2} (1 + \frac{1}{1 - \alpha})$	$\frac{1}{2} (1 + \frac{1}{(1 - \alpha)^2})$
伪随机探测再散列	$-\frac{1}{\alpha} \ln(1 - \alpha)$	$\frac{1}{1 - \alpha}$
链地址法	$1 + \frac{\alpha}{2}$	$\alpha + e^{-\alpha}$



哈希表的缺点

- 冲突的问题，当 α 接近1时，会产生大量冲突，导致性能下降
- 哈希表是非排序表，无法通过某种方式从表中查出关键词在所有数据元素中的顺序

[CRC]循环冗余校验(Cyclic Redundancy Check, CRC)是一种根据网络数据包或电脑文件等数据产生简短固定位数校验码的一种散列函数

名称	多项式	应用举例
CRC-8	X^8+X^2+X+1	
CRC-12	$X^{12}+X^{11}+X^3+X^2+X+1$	telecom systems
CRC-16	$X^{16}+X^{15}+X^2+1$	Bisync, Modbus, USB, ANSI X3.28, SIA DC-07, many others; also known as CRC-16 and CRC-16-ANSI
CRC-CCITT	$X^{16}+X^{12}+X^5+1$	ISO HDLC, ITU X.25, V.34/V.41/V.42, PPP-FCS
CRC-32	$X^{32}+X^{26}+X^{23}+X^{22}+X^{16}+X^{12}+X^{11}+X^{10}+X^8+X^7+X^5+X^4+X^2+X+1$	ZIP, RAR, IEEE 802 LAN/FDDI, IEEE 1394, PPP-FCS
CRC-32C	$X^{32}+X^{28}+X^{27}+X^{26}+X^{25}+X^{23}+X^{22}+X^{20}+X^{19}+X^{18}+X^{14}+X^{13}+X^{11}+X^{10}+X^9+X^8+X^6+1$	iSC

软件实现，用查表法；如计算CRC16校验值

软件实现，用查表法计算CRC16校验值

```
16 // CRC16码表
17 //
18 ///////////////////////////////////////////////////////////////////
19 static uint16_t const CRC16Table[256] = {
20     0x0000, 0xC0C1, 0xC181, 0x0140, 0xC301, 0x03C0, 0x0280, 0xC241,
21     0xC601, 0x06C0, 0x0780, 0xC741, 0x0500, 0xC5C1, 0xC481, 0x0440,
22     0xCC01, 0x0CC0,
23     0x0A00, 0xCAC1,     for(int i = 0; i < length; i++)
24     0xD801, 0x18C0,     {
25     0x1E00, 0xDEC1,     tableNo = ((result & 0xff) ^ (dataIn[i] & 0xff));
26     0x1400, 0xD4C1,     result = ((result >> 8) & 0xff) ^ CRC16Table[tableNo];
27     0xD201, 0x12C0,
28     0xF001, 0x30C0,
29     0x3600, 0xF6C1,     }
30     0x3C00, 0xFCC1,
31     0xFA01, 0x3AC0, 0x3B80, 0xFB41, 0x3900, 0xF9C1, 0xF881, 0x3840,
32     0x2800, 0xE8C1, 0xE981, 0x2940, 0xEB01, 0x2BC0, 0x2A80, 0xEA41,
33     0xEE01, 0x2EC0, 0x2F80, 0xEF41, 0x2D00, 0xEDC1, 0xEC81, 0x2C40,
34     0xE401, 0x24C0, 0x2580, 0xE541, 0x2700, 0xE7C1, 0xE681, 0x2640,
35     0x2200, 0xE2C1, 0xE381, 0x2340, 0xE101, 0x21C0, 0x2080, 0xE041,
36     0xA001, 0x60C0, 0x6180, 0xA141, 0x6300, 0xA3C1, 0xA281, 0x6240,
```

硬件实现，用寄存器的移位法实现

硬件计算过程

1. 设置CRC寄存器，并给其赋值FFFF(hex)。
2. 将数据的第一个8-bit字符与16位CRC寄存器的低8位进行异或，并把结果存入CRC寄存器。
3. CRC寄存器向右移一位，MSB补零，移出并检查LSB。
4. 如果LSB为0，重复第三步；若LSB为1，CRC寄存器与多项式码相异或。

注意：该步检查LSB应该是右移前的LSB，即第3步前的LSB。

5. 重复第3与第4步直到8次移位全部完成。此时一个8-bit数据处理完毕。
6. 重复第2至第5步直到所有数据全部处理完成。
7. 最终CRC寄存器的内容即为CRC值。

[Jhash] Linux内核里面用散列函数

Linux内核里面用的jhash是一个久经考验，并被实践证明经得起考验的hash函数，可以CPMS(Copy Paste Modify Save)之。Jhash的作者Bob Jenkins在其网站上还公布了诸如针对能预知的数据进行hash的hash函数--完美 (perfect) hash函数等一系列其他hash函数，看客们可以选择之，如果有兴趣继续钻研，也可以踏在他们的肩膀上

[Pearson' s hash, 皮尔逊哈希] 适用于不支持shift和xor操作的情况，也是利用一个码表来计算的。可以将一组字节，计算为2个字节、4字节数（可修改代码调整）。

```
static const unsigned char T[256] = {  
    // 0-255 shuffled in any (random) order suffices  
    98,  6, 85,150, 36, 23,112,164,135,207,169,  5, 26, 64,165,219, // 1  
    61, 20, 68, 89,130, 63,  
    90,168,156,203,177,120, unsigned char h, hh[2];  
    237, 18,253,225,  8,208,  
    123,251, 67,250,161,  0,  
    59,153, 29,  9,213,167, for (j = 0; j < 2; j++) {  
    197, 96,210, 45, 16,227, h = T[(token[0] + j) % 256];  
    39,158,178,187,131,136, for (i = 1; i < len; i++)  
    154, 35, 86,171,105, 34, h = T[h ^ token[i]];  
    133,232,196,144,198,124, hh[j] = h;  
    189,205,199,128,176, 19,  
    183,201, 22, 83, 13,214, }  
    221, 31,209,182,143, 92,  
    3, 14,204, 72, 21, 41,  
    238, 87,240,155,180,170, return hh[0] * 256 + hh[1];  
    43,119,224, 71,122,142,  
};
```

//--- 开放定址哈希表的存储结构 ---

```
int  hashsize[] = { 997, ... };
typedef struct {
    //按hashsize[sizeindex]值申请存储空间
    ElemType  *elem;
    int  count;      // 当前数据元素个数
    int  sizeindex; // hashsize[sizeindex]为当前容量
    float  a;        //填装因子上限
} HashTable;

#define  SUCCESS  1
#define  UNSUCCESS  0
#define  DUPLICATE  -1
```

```
Status SearchHash (HashTable H, KeyType K, int &p, int &c)
{
```

```
    // 在开放定址哈希表H中查找关键码为K的记录
```

```
    // c, 记录冲突次数
```

```
    p = Hash(K);           // 求得哈希地址
```

```
    while ( !isNull(H.elem[p])  && !EQ(K, H.elem[p].key) )
        collision(p, ++c);    // 求得下一探查地址 p
```

```
    if (EQ(K, H.elem[p].key))
```

```
        return SUCCESS;    //查找成功，待查数据元素位置 由参数p返回
```

```
    else
```

```
        return UNSUCCESS; //查找不成功，参数p返回一个空值占用位置
```

```
} // SearchHash
```

//按某个冲突解决方法计算下一个位置，c可以使用也可以不使用

```
void collision(int &p, int c);
```

```

Status InsertHash (HashTable &H, Elemtype e){
{
    if((float(H.count+1)/float(hashsize[H.sizeindex]))>H.a)
        return UNSUCCESS;
    int p = 0; //哈希表插入位置
    int c = 0; //冲突次数
    if( HashSearch ( H, e.key, p, c ) == SUCCESS )
        return DUPLICATE;

    if( c<hashsize[H.sizeindex]/2 )//冲突次数判定，自行选择标准
    {
        H.elem[p] = e;
        ++H.count;
    }
    else
        RecreateHashTable(H, e); // 重建哈希表
    return OK
}
}

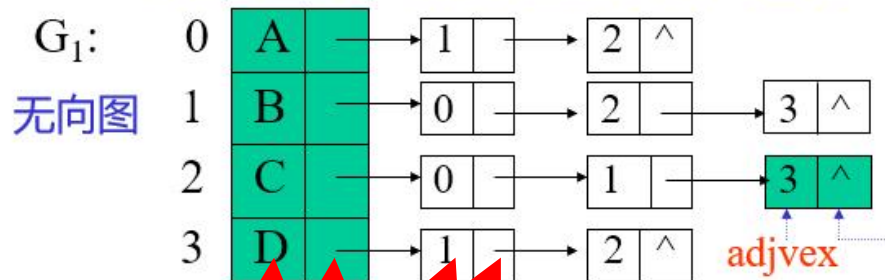
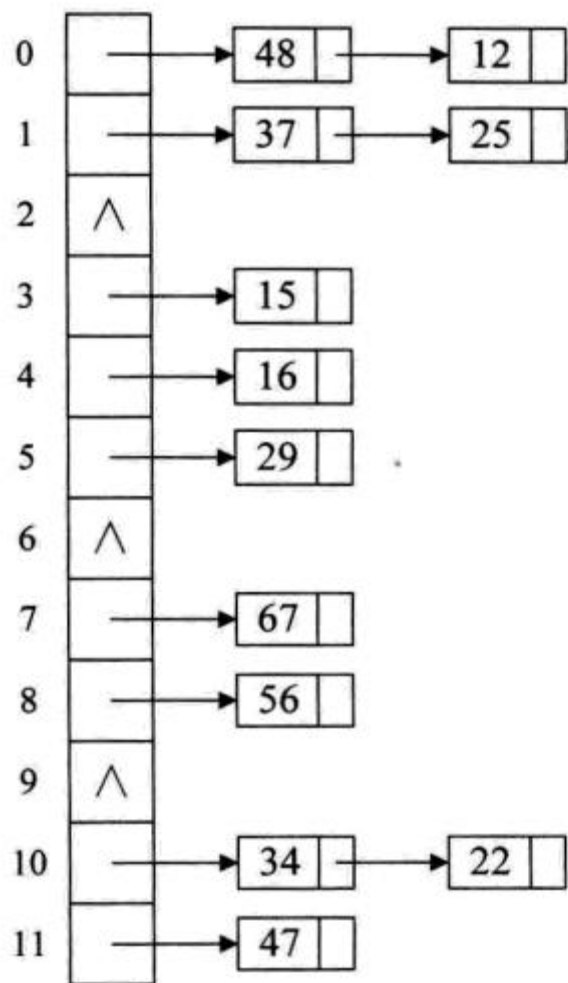
```

```

typedef struct ElemType{
    int key;
    char tag; //0 == NULL
              //1 == normal
              //2 == deleted
}

```


//--- 链地址法的存储结构 ---



```
typedef struct ArcNode {
    int          adjvex;      // 该弧所指向的顶点的位置
    struct ArcNode * nextarc; // 指向下一条弧的指针
    int          weight;     // 权重
    InfoType     info;      // 该弧的其它相关信息的指针
}ArcNode;

typedef struct VNode {
    VertexType   data;      // 顶点信息
    ArcNode      * firstarc; // 指向第一条依附该顶点的弧的指针
}VNode, AdjList[MAX_VERTEX_NUM];

typedef struct {
    AdjList      vertices; // 顶点向量
    int          vexnum, arcnum; // 图的当前顶点数和弧数
    GraphKind    kind;      // 图的种类标志
}ALGraph;
```

```
#define MAX_HASH_VALUE_NUM    2000    //哈希函数结果集值数量
typedef struct  HashNode  {
    ElemType  data;                //存入hash表的data
    struct HashNode * next ; //指向下一个结点的指针
} HashNode, *PHashNode;
```

```
PHashNode pHashTable[MAX_HASH_VALUE_NUM]; //数组变量方式
HashNode dHashTable[MAX_HASH_VALUE_NUM];  //数组变量方式
```

```
typedef struct  PHashTable  {
    PHashNode HashTable[MAX_HASH_VALUE_NUM];
} PHashTableA;
```

```
typedef struct  PHashTableB  {
    PHashNode * HashTable;
    int nHashTableSize;
} PHashTableB;
```

```
typedef struct  DHashTable  {
    HashNode HashTable[MAX_HASH_VALUE_NUM];
} DHashTable;
```

```
typedef struct  DHashTable  {
    HashNode * HashTable;
    int nHashTableSize;
} DHashTable;
```

哈希方法的其他用途

- 用哈希方法对原始数据做数字摘要，用于发现接收到的数据是否和原始数据一致。如，报文通信中的报文校验字段，下载文件后进行MD5校验等。
- 在分布式系统中，将同源、同目的的数据快速用哈希算法分配到同一个设备进行处置，避免数据混淆；
- 海量数据时，用哈希做快速数据分类，快速筛选符合一定条件的数据进行进一步处理

本章学习要点

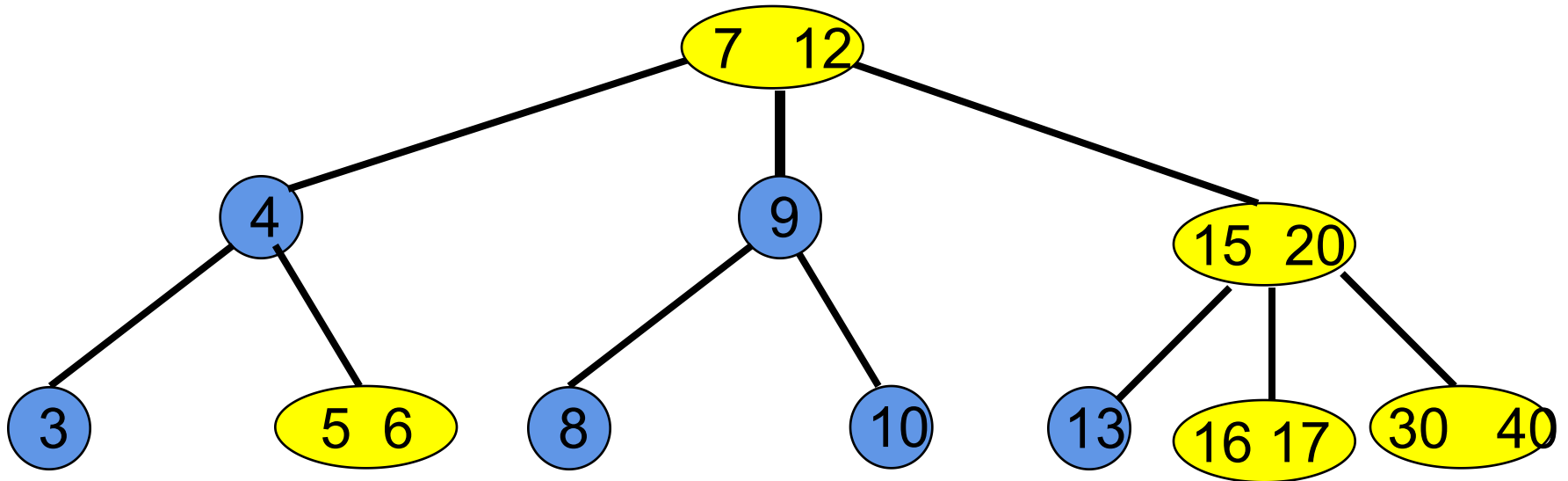
1. 顺序表和有序表的查找方法及其平均查找长度的计算方法。
2. 了解静态查找树的构造方法和查找方法及其和有序表的差别。
3. 熟练掌握二叉排序树以及平衡二叉树的构造和查找方法。
4. 理解B-树和B+树的结构特点以及它们各自查找过程的不同之处。

本章学习要点

5. 熟练掌握哈希表的构造方法，深刻理解哈希表与其它结构的表的实质性的差别。
6. 掌握按定义计算各种查找方法在等概率情况下查找成功时的平均查找长度。

B-Tree删除练习

B-Tree – Delete

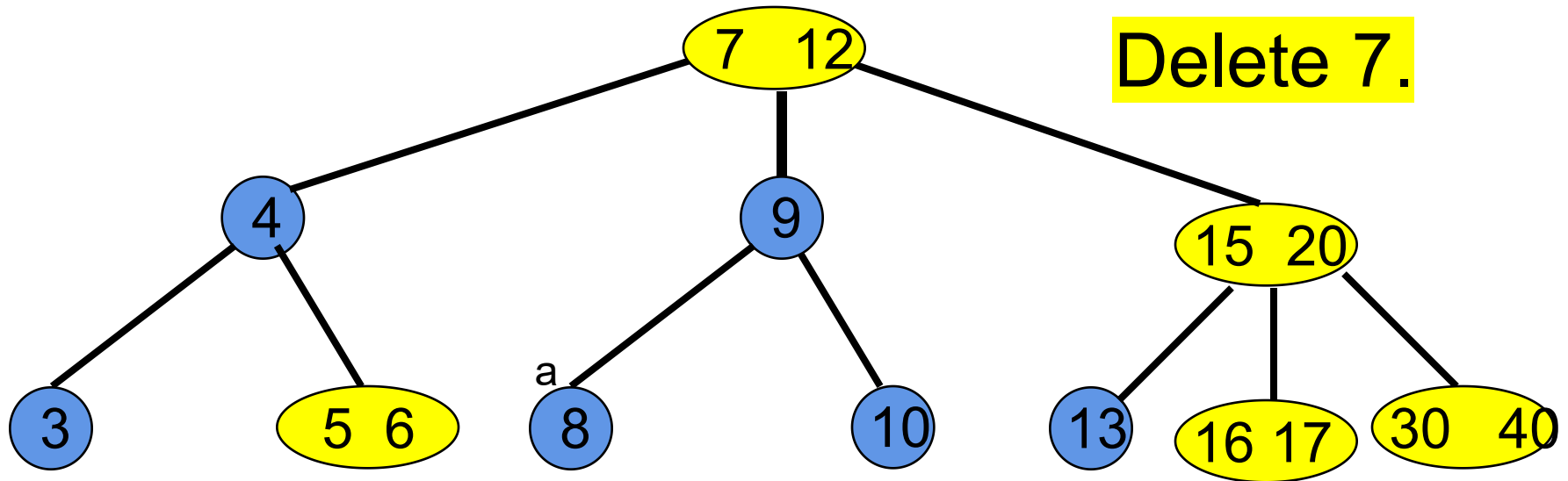


Delete 7.

Delete 3.

Delete 8.

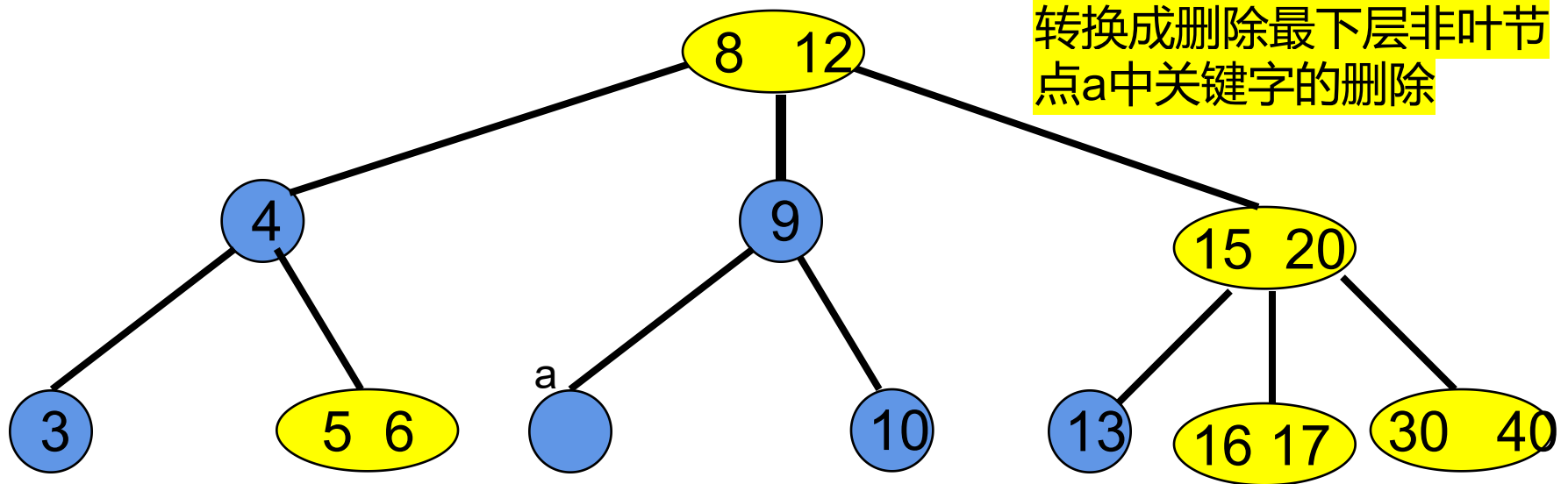
B-Tree – Delete例子：3阶B-Tree



反之,若在 B-树上删除一个关键字,则首先应找到该关键字所在结点,并从中删除之,若该结点为最下层的非终端结点,且其中的关键字数目不少于 $\lceil m/2 \rceil$,则删除完成,否则要进行“合并”结点的操作。假若所删关键字为非终端结点中的 K_i ,则可以指针 A_i 所指子树中的最小关键字 Y 替代 K_i ,然后在相应的结点中删去 Y 。例如,在图 9.16(a)的

1) 7所在结点不是最下层非叶结点, 用后继8填上, 变成删除8所在的最下层非叶结点a中的关键字8

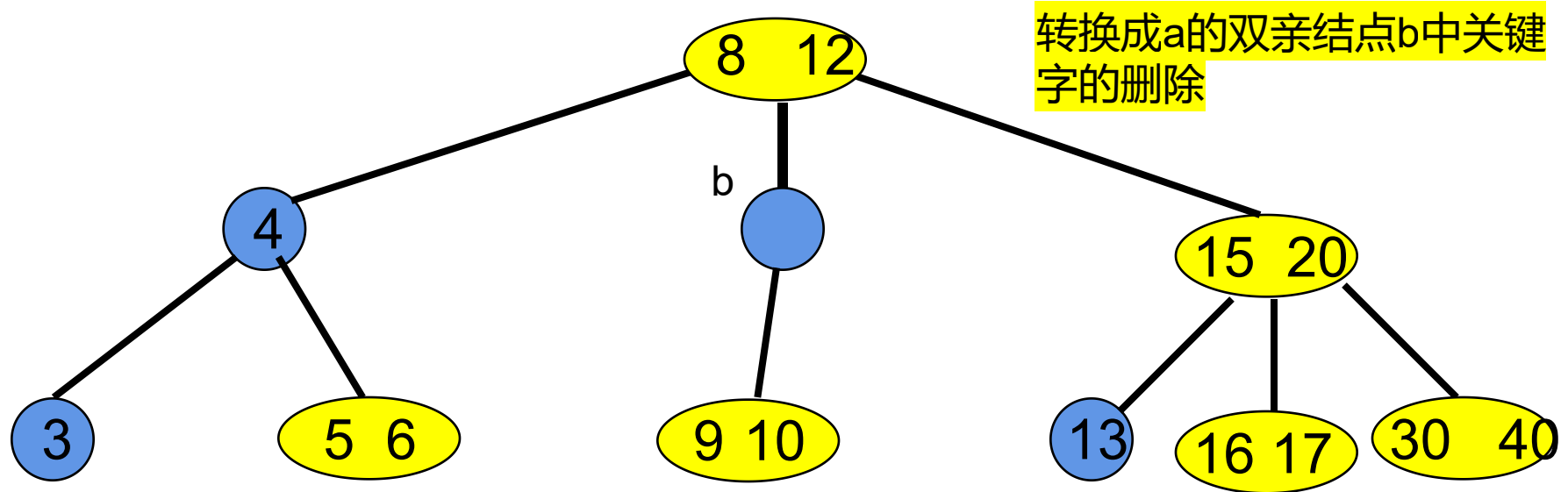
B-Tree – Delete例子：3阶B-Tree



(3) 被删关键字所在结点和其相邻的兄弟结点中的关键字数目均等于 $\lceil m/2 \rceil - 1$ 。假设该结点有右兄弟,且其右兄弟结点地址由双亲结点中的指针 A_i 所指,则在删去关键字之后,它所在结点中剩余的关键字和指针,加上双亲结点中的关键字 K_i 一起,合并到 A_i 所指兄弟结点中(若没有右兄弟,则合并至左兄弟结点中)。例如,从图 9.17(b)所示

2) 将a结点合并到其右兄弟:

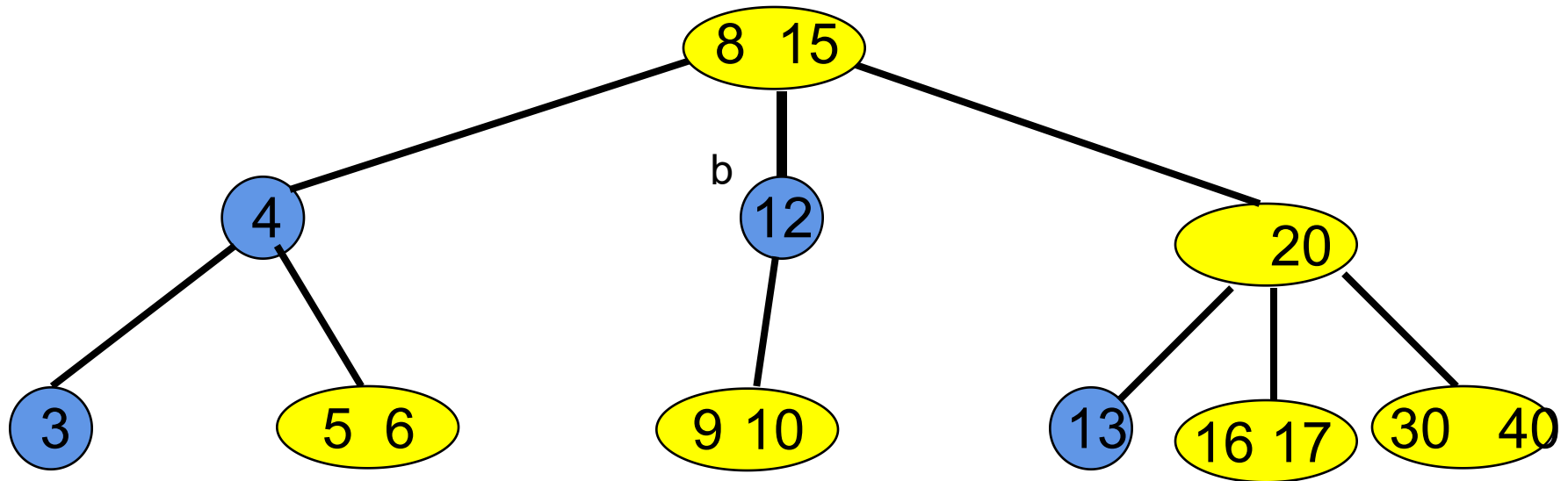
B-Tree – Delete例子：3阶B-Tree



(2) 被删关键字所在结点中的关键字数目等于 $\lceil m/2 \rceil - 1$, 而与该结点相邻的右兄弟(或左兄弟)结点中的关键字数目大于 $\lceil m/2 \rceil - 1$, 则需将其兄弟结点中的最小(或最大)的关键字上移至双亲结点中, 而将双亲结点中小于(或大于)且紧靠该上移关键字的关键字下移至被删关键字所在结点中。例如, 从图 9.17(a)中删去 50, 需将其右兄弟结点中

3) b结点无关键字了, 需要从其右兄弟借入关键字: b的右兄弟的关键字最小是15, b双亲的关键字最大是12, 12下来, 15上去。

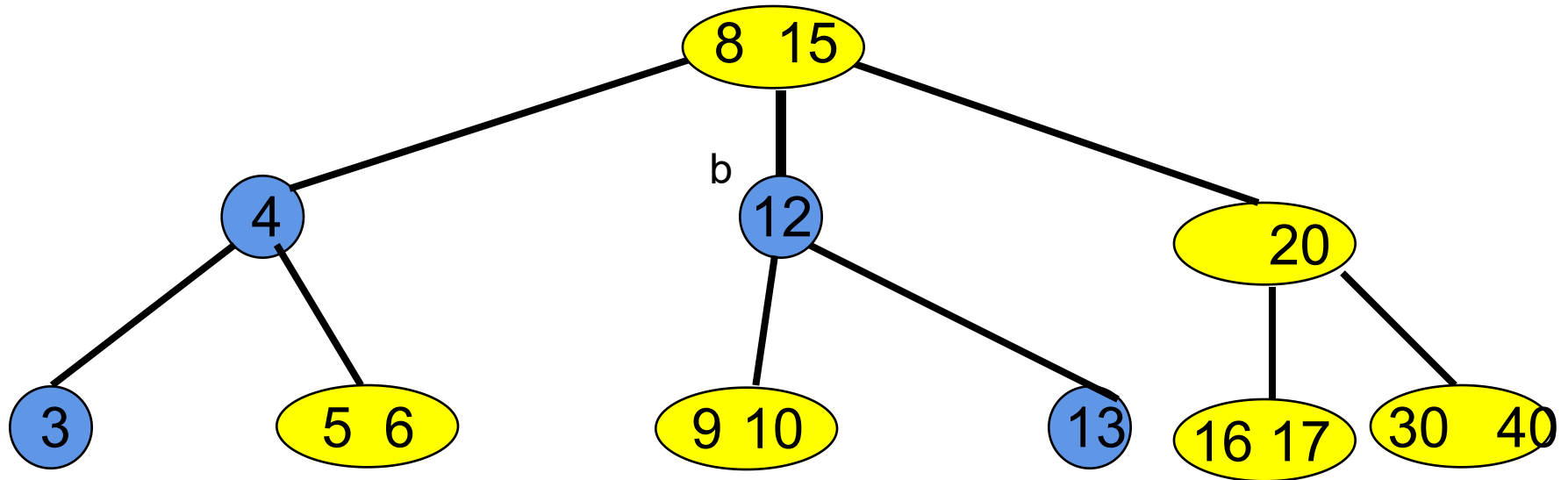
B-Tree – Delete例子：3阶B-Tree



4) 15的所在结点的 A_0 子树 (13所在结点的子树) 怎么办?

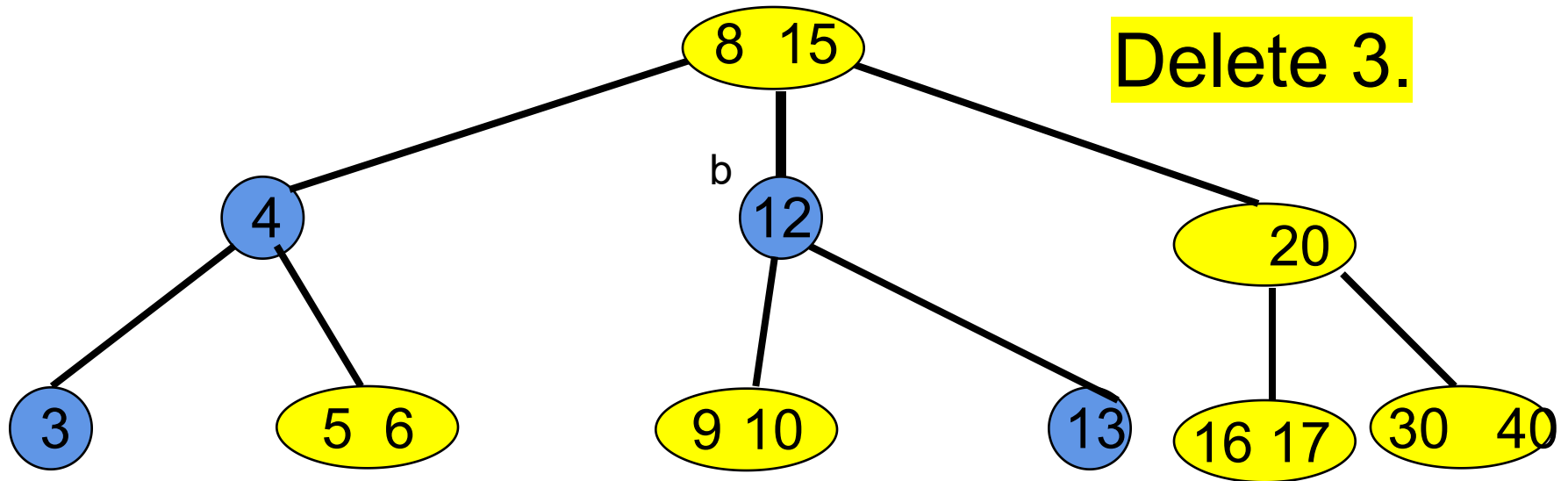
在借入右兄弟时, 将子树一起带过来。是b结点借入右兄弟的关键字的, 就带到b结点去。

B-Tree – Delete例子：3阶B-Tree



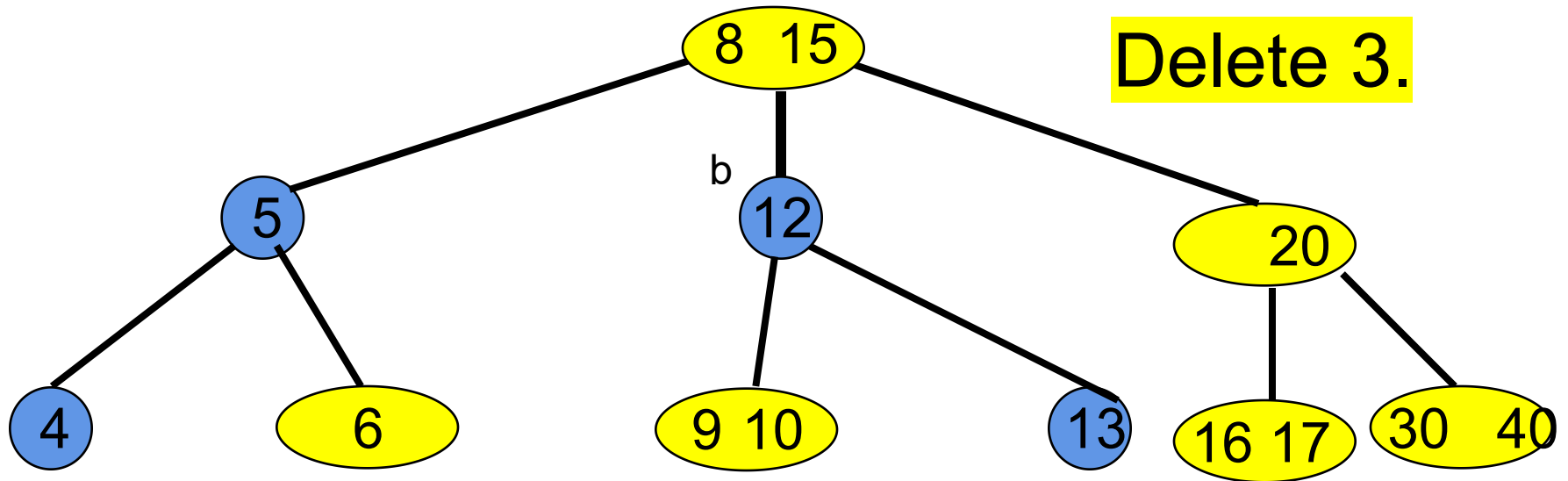
完成删除关键字7。

B-Tree – Delete例子：3阶B-Tree



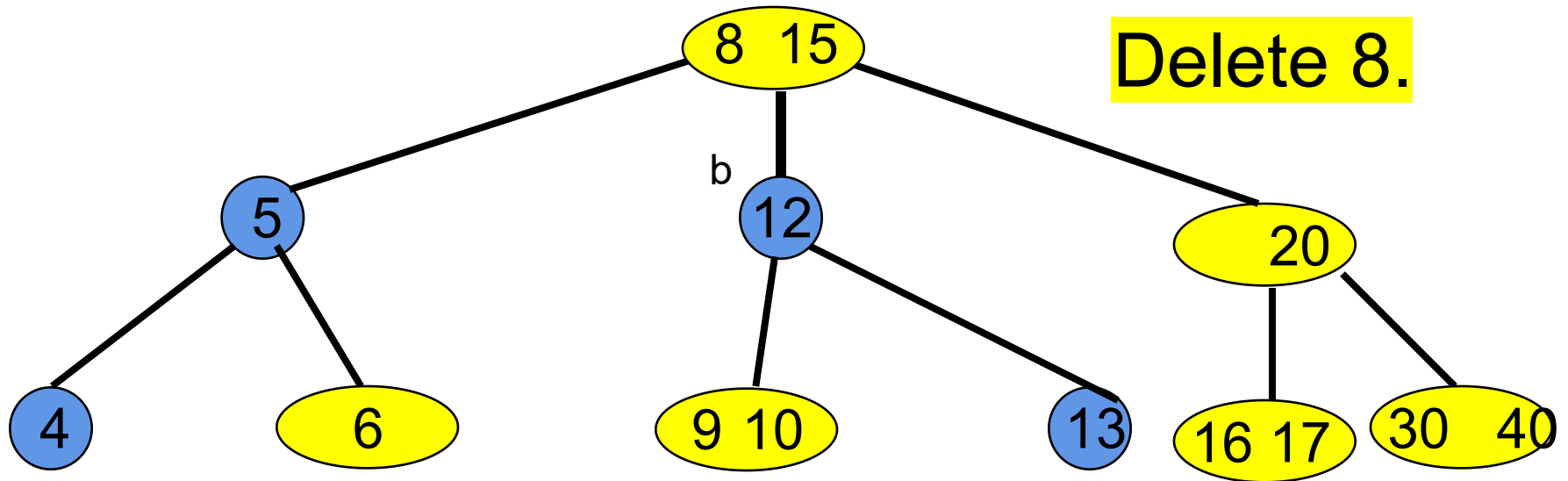
1) 3在最下层非叶子结点。结点内关键字个数为1个，等于 $\lceil 3/2 \rceil - 1$ ，从右兄弟结点借。

B-Tree – Delete例子：3阶B-Tree



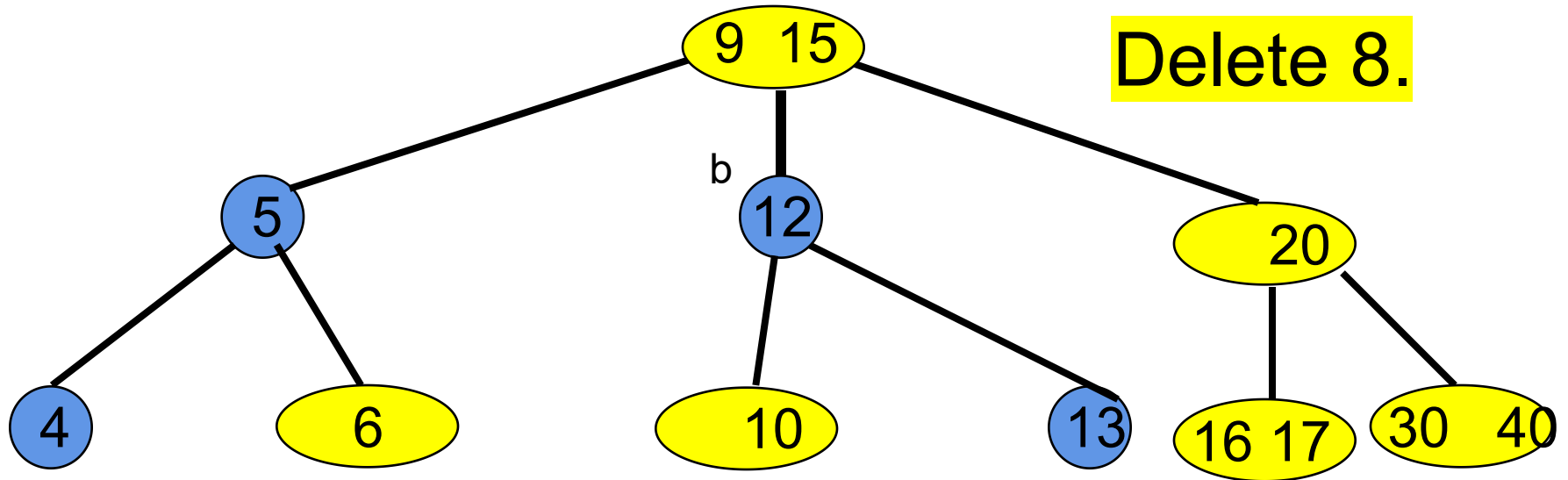
2) 右兄弟中关键字个数是2个，大于 $\lceil 3/2 \rceil - 1$ ，借出一个关键字及子树后不需要调整了。完成。

B-Tree – Delete例子：3阶B-Tree



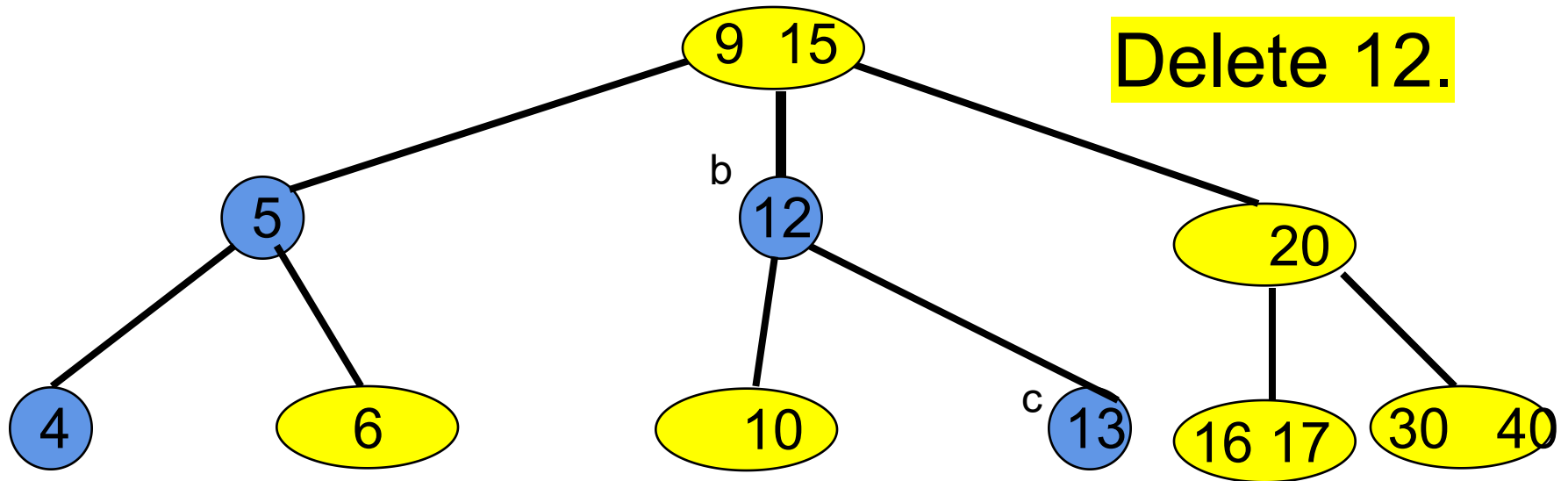
1) 不是最下层非叶子结点，找后继替代：9。9所在结点的关键字个数大于 $\lceil 3/2 \rceil - 1$ ，替换后直接删除。

B-Tree – Delete例子：3阶B-Tree



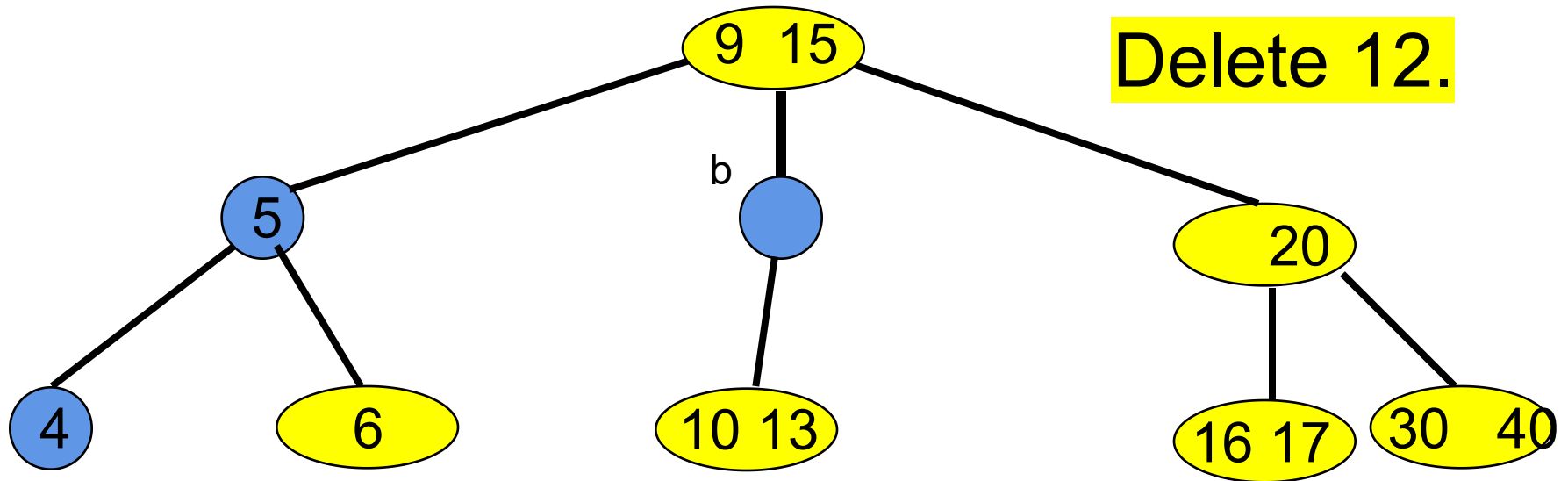
2) 完成

B-Tree – Delete例子：3阶B-Tree



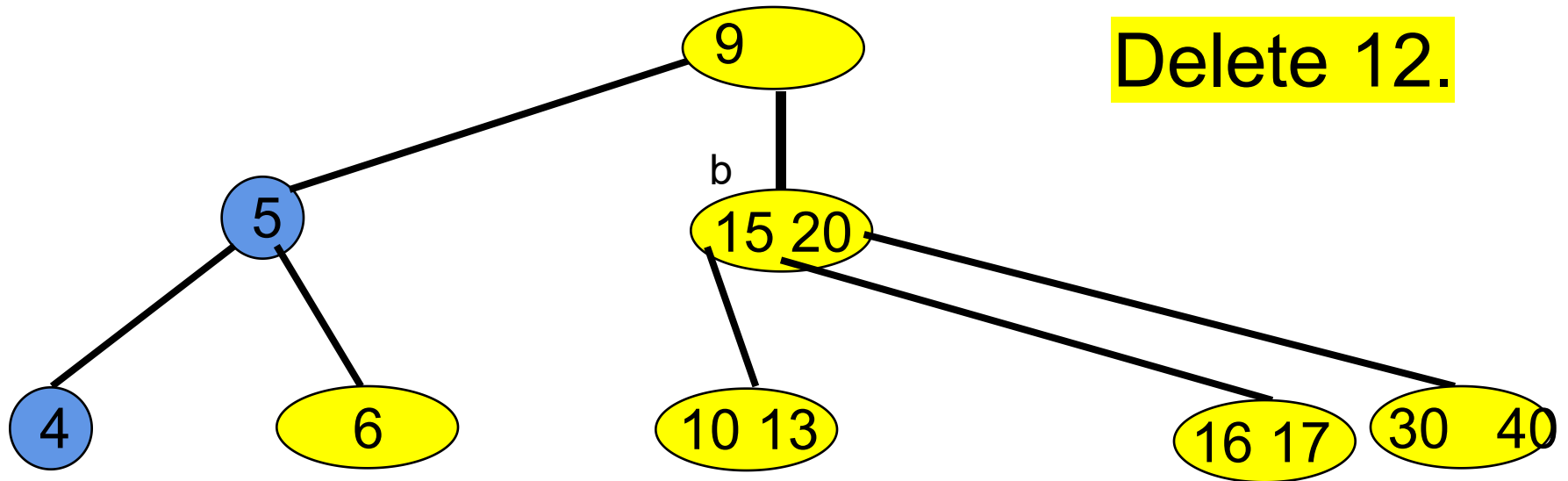
1) 12的后继13替换，再删除12。13所在c结点，关键字数量1个，右兄弟不存在，左兄弟结点关键字个数1个，需要合并。

B-Tree – Delete例子：3阶B-Tree



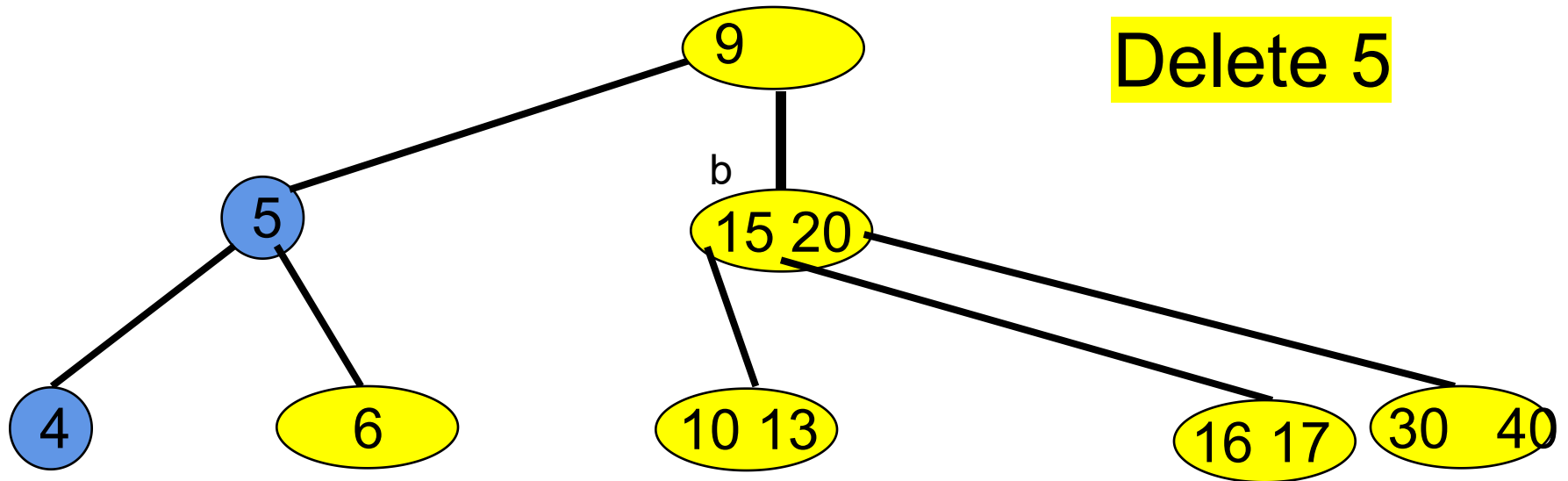
2) b所在结点关键字数量小于 $\lceil 3/2 \rceil - 1$, 需借入右兄弟关键字, 右兄弟关键字数量1个, 需要借入后合并

B-Tree – Delete例子：3阶B-Tree



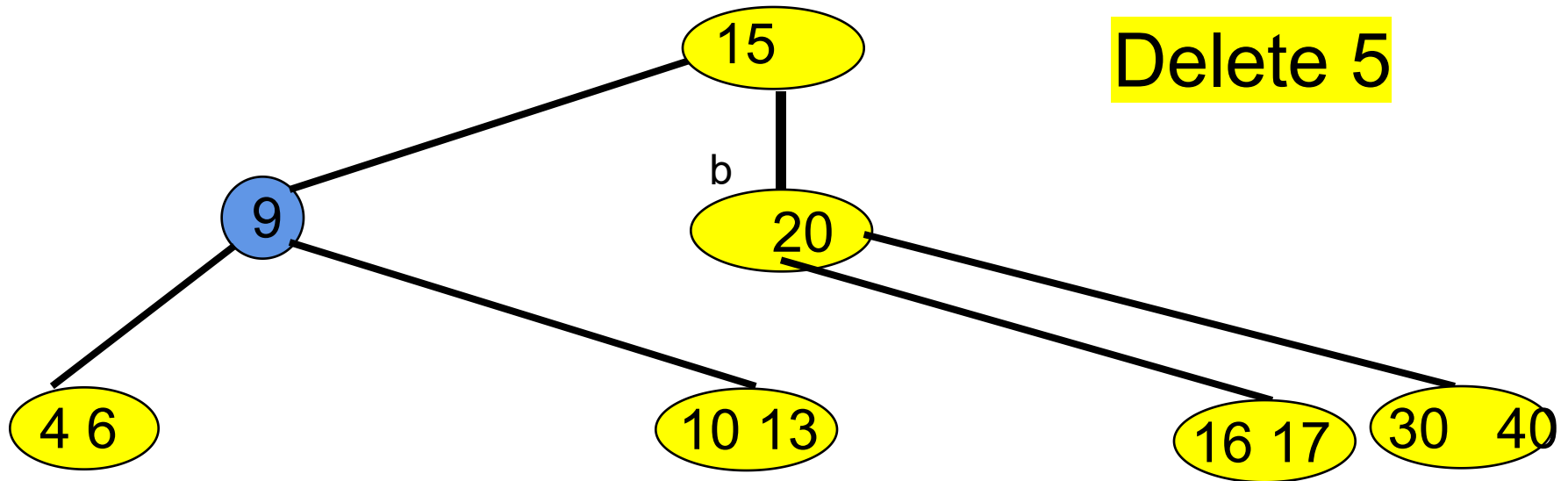
3) b的右兄弟关键字有1个，不够用，需要借入后合并，带着借入关键字的子树一起过来。

B-Tree – Delete例子：3阶B-Tree



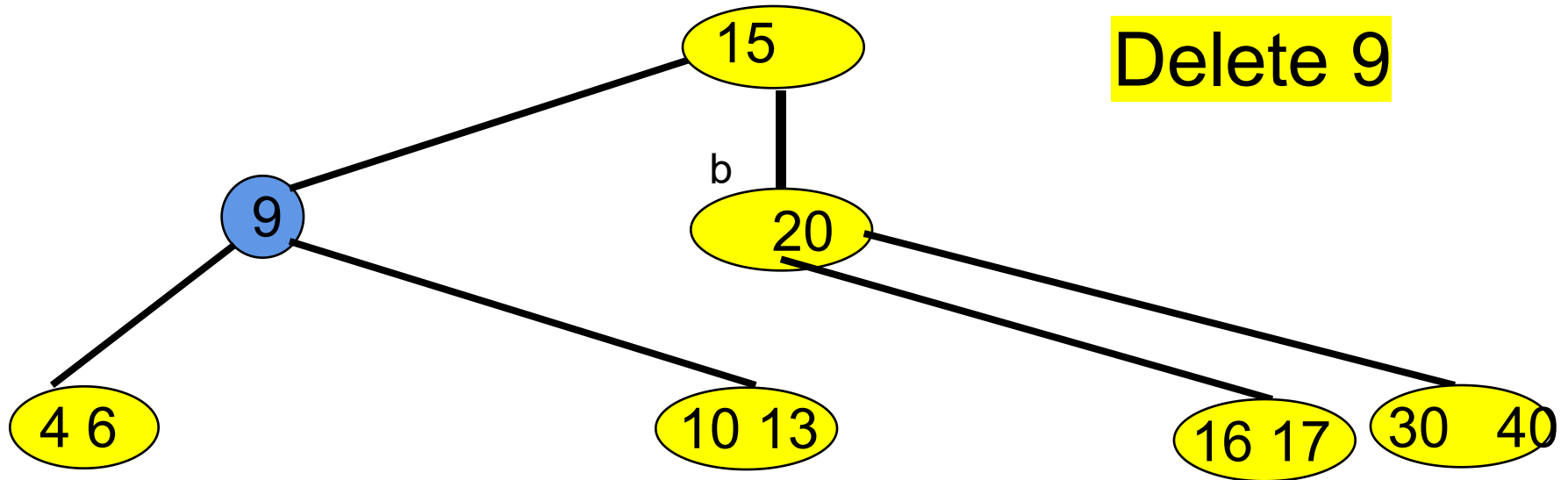
1) 5换成6；删除6；将15带着子树一起借过来。

B-Tree – Delete例子：3阶B-Tree



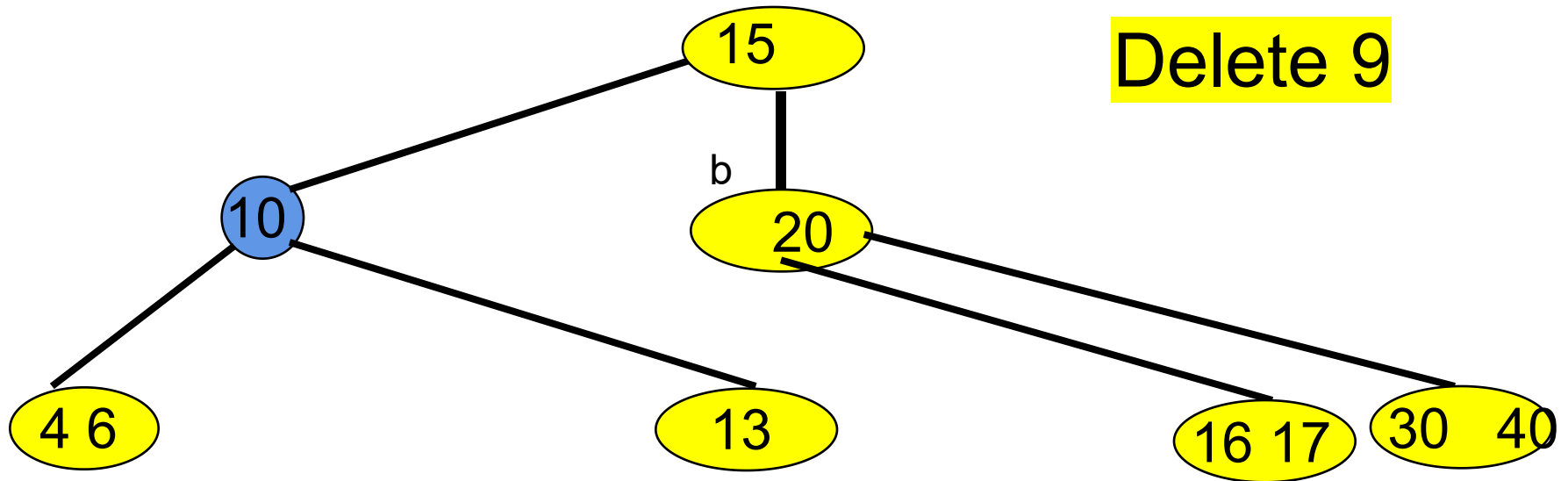
2) 完成

B-Tree – Delete例子：3阶B-Tree



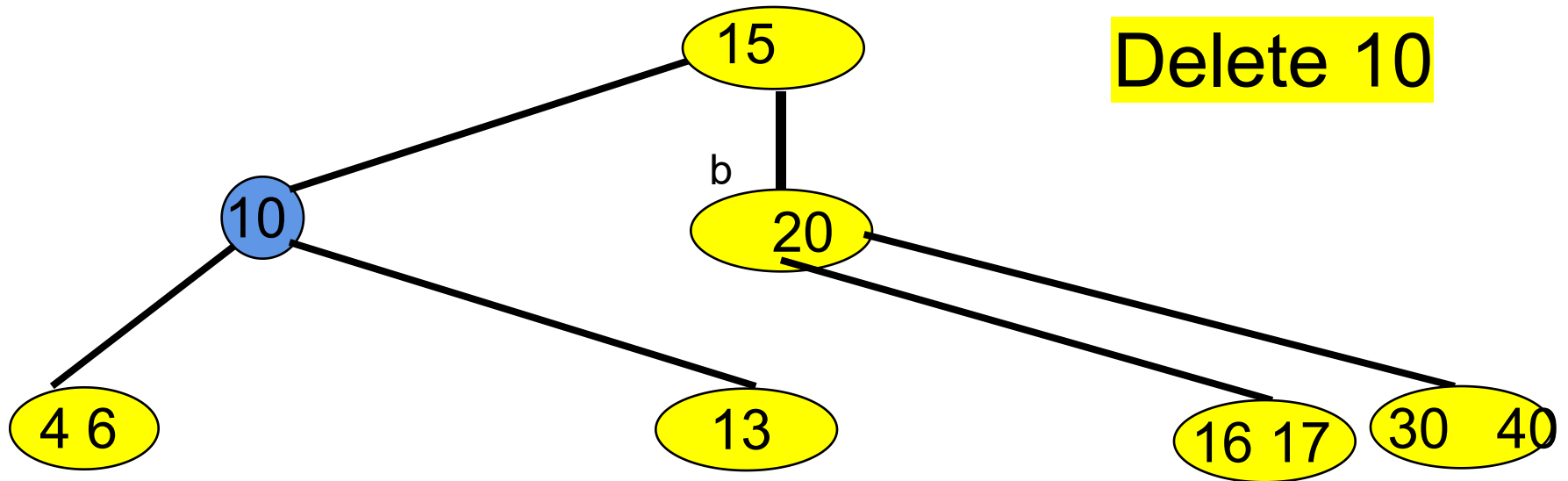
1) 9用10换，删掉9

B-Tree – Delete例子：3阶B-Tree



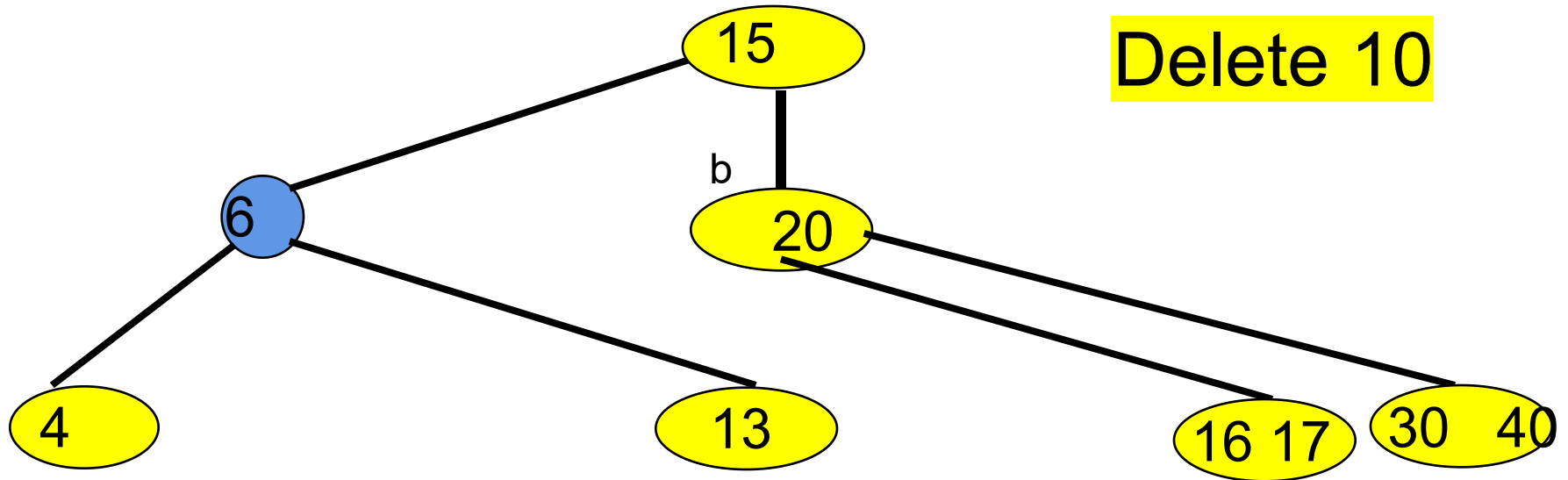
2) 完成

B-Tree – Delete例子：3阶B-Tree



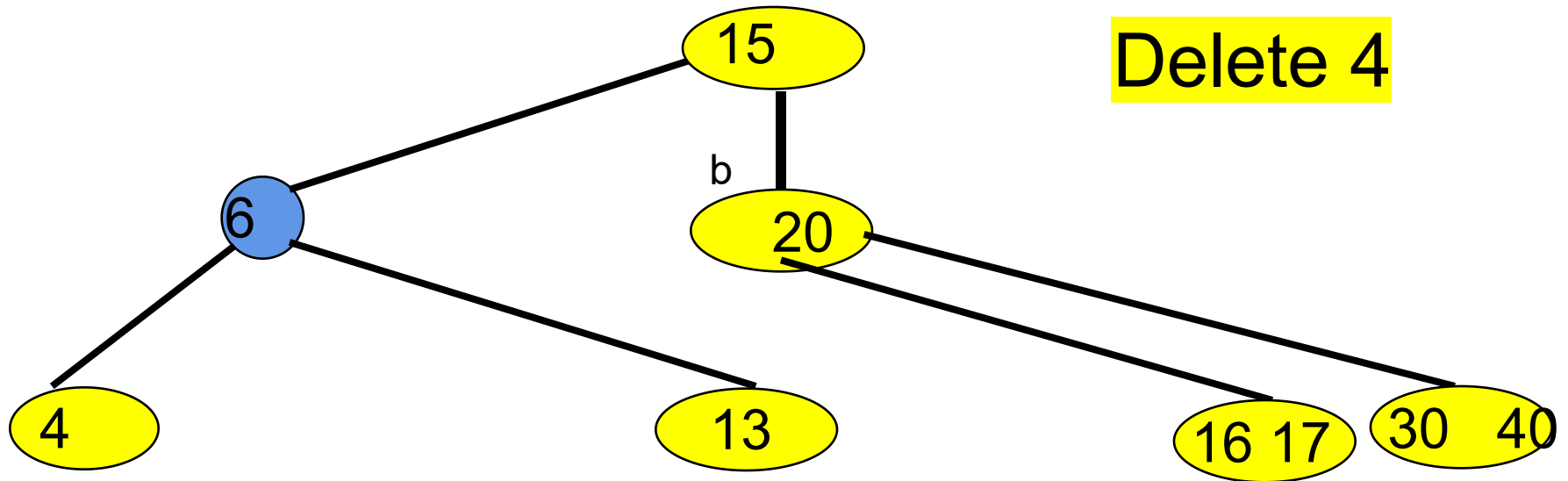
1) 10用13换，删掉10后，从左兄弟借入。

B-Tree – Delete例子：3阶B-Tree



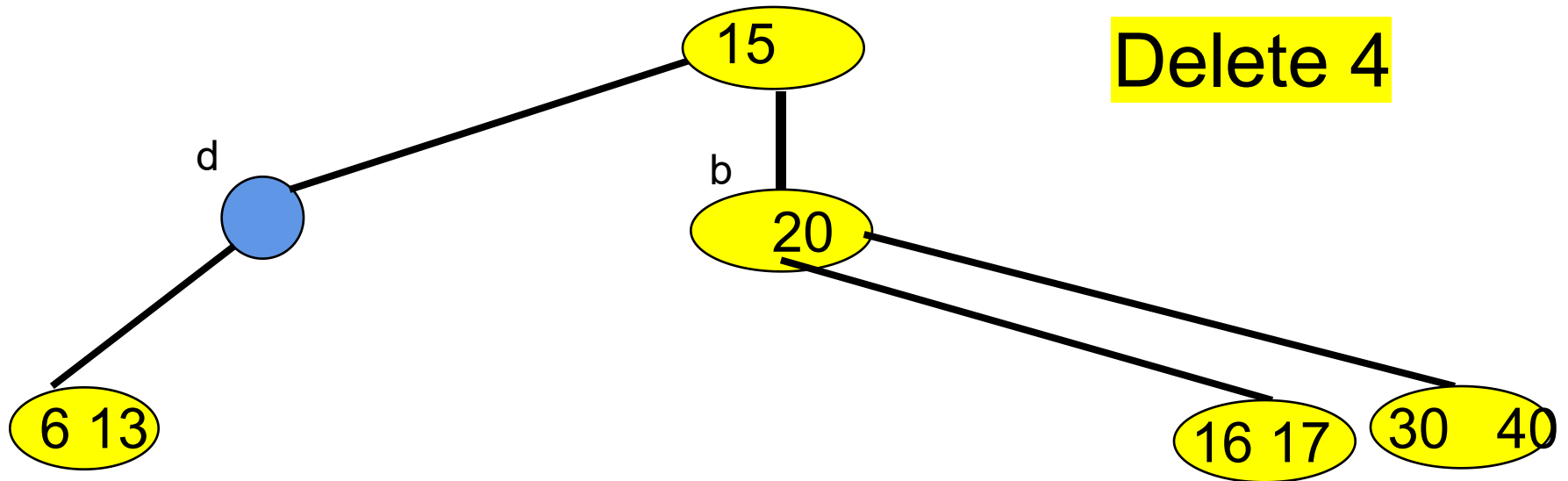
2) 完成

B-Tree – Delete例子：3阶B-Tree



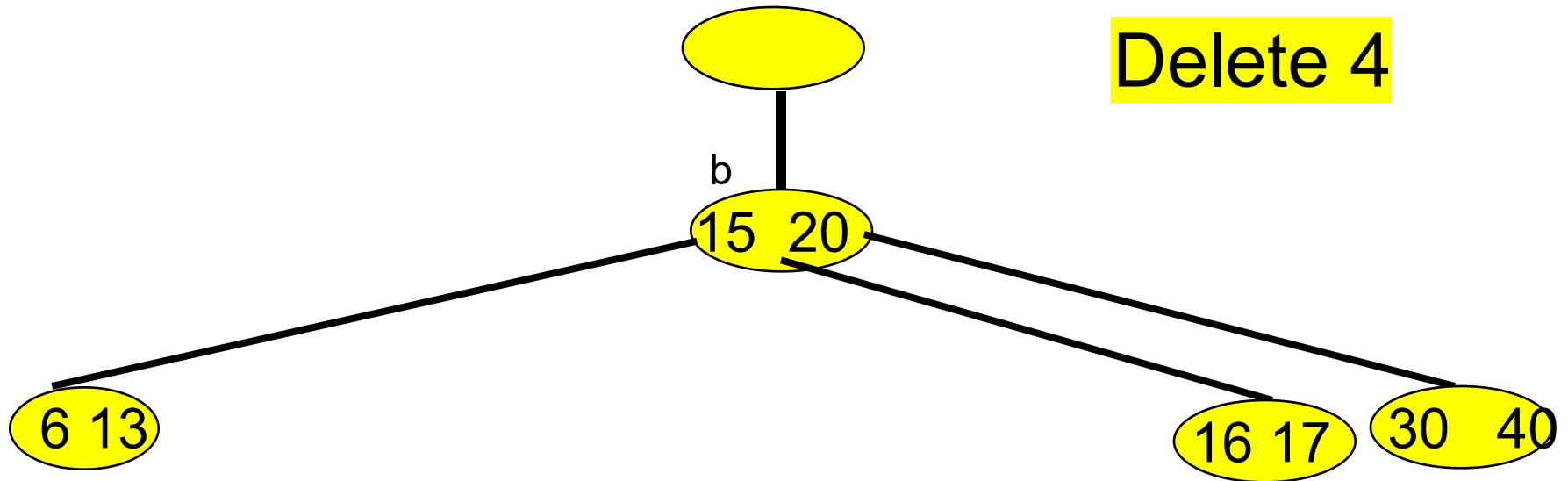
1) 最下层非叶子结点，删掉4后，和右兄弟合并；

B-Tree – Delete例子：3阶B-Tree



2) d需要继续和右兄弟合并;

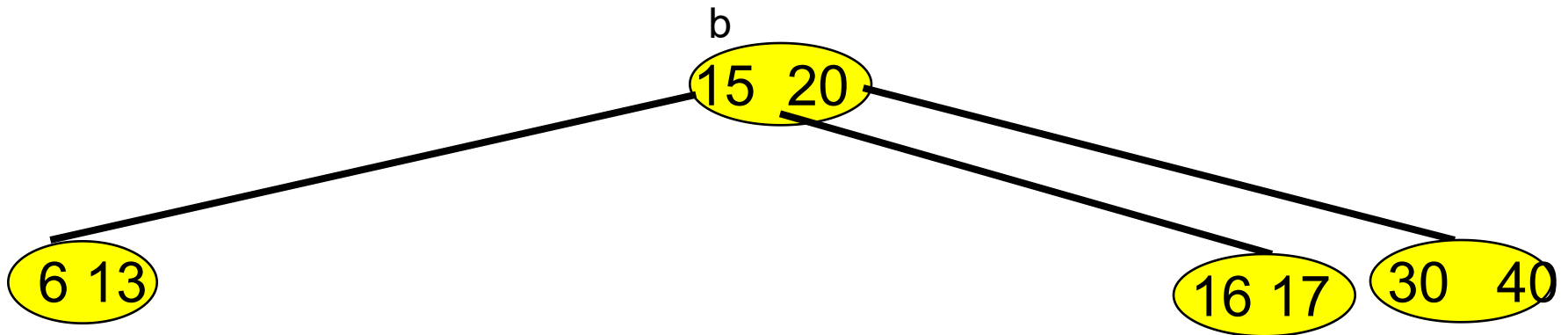
B-Tree – Delete例子：3阶B-Tree



3) 删掉空的根

B-Tree – Delete例子：3阶B-Tree

Delete 4



4) 完成